
MASTER THESIS

The Emergence of Internal Representations in Sequence Models:

Comparison of Architectures and Objectives

Program Name	MSc Artificial Intelligence
--------------	-----------------------------

Module Name	Postgraduate Dissertation – AG707
-------------	-----------------------------------

Supervisor	Athanasios Chasapis
------------	---------------------

Student Full Name	Charalampos Lazaridis Patsalias
-------------------	---------------------------------

Thessaloniki, October 2026

Copyright Statement

Copyright © 2026 Charalampos Lazaridis Patsalias.

All rights reserved. Copying, storing and distributing this work, in whole or in part, for commercial purposes is prohibited. Reprinting, storing and distributing it for non-profit, educational or research purposes is permitted, provided that the source is acknowledged and this notice is retained. Enquiries about use of the work for commercial purposes should be addressed to the author.

The views and conclusions contained in this document are those of the author and should not be interpreted as representing the official positions of Aegean College or the University of Essex.

Acknowledgements

Completing this dissertation would not have been possible without the support of the people who stood by me throughout.

I would like to warmly thank my supervisor, Athanasios Chasapis, for his invaluable guidance and insightful comments at every stage of the research.

I thank my family from the bottom of my heart for supporting and encouraging me in every possible way on this journey, with patience and love.

Finally, I am especially grateful to my partner, Marina, who stood by me at every moment, lifted my confidence in the difficult times and smiled with me in the happy moments of this journey.

Charalampos Lazaridis Patsalias

Thessaloniki, October 2026

Abstract

Can a sequence model form a world model of an environment it does not observe directly? In Othello, earlier studies showed that models trained solely on sequences of legal moves, with no supervision of the board state, develop representations from which that state can be extracted. This thesis examines whether the finding generalises beyond a Transformer trained on next-move prediction. It compares two architectures, Transformer and Mamba, and two training objectives, Autoregressive (AR) and Joint-Embedding Predictive Architecture (JEPA), on 8×8 , 12×12 and 16×16 boards: twelve conditions with shared data and a unified evaluation protocol.

The evaluation separates behavioural performance, board-state decoding and the causal effect of the representation. The results did not reveal a universal winner. Mamba-JEPA achieved the highest macro-balanced accuracy on the relative state with a linear probe at all three sizes, whereas Mamba-AR had the highest top-1 legality. Adapted causal interventions increased the probability mass of the legal moves of the modified board in all twelve conditions, both against the unchanged run and against a geometrically matched random control. This effect weakened on the larger boards.

Scale also changes the discriminative power of the comparison. The gap between the highest and lowest macro-balanced accuracy of the linear probe for the relative state grows from 1.32 percentage points at 8×8 to 5.39 at 16×16 . The established 8×8 board may therefore mask differences that emerge at greater complexity.

The findings indicate that decodable information, legal behaviour and causal use are not interchangeable criteria for the existence of a world model. Evaluation requires a combination of behavioural, representational and causal measurements across more than one scale.

Keywords: internal representations, interpretability, world models, Othello, Transformer, Mamba, JEPA, causal interventions.

Contents

Copyright Statement	i
Acknowledgements	ii
Abstract	iii
Abbreviations and Acronyms	xi
1 Introduction	1
2 Theoretical Background and Literature Review	4
2.1 Othello as an environment for studying world models	4
2.1.1 Rules and dynamics of the game	4
2.1.2 From the move sequence to the board state	5
2.2 Sequence modelling architectures	5
2.2.1 Transformer	6
2.2.2 State Space Models and Mamba	7
2.3 Training objectives	7
2.3.1 Autoregressive next-move prediction	8
2.3.2 JEPA-like prediction in representation space	8
2.3.3 InfoNCE and collapse avoidance	9
2.4 Internal representations, interpretability and functional behaviour	10
2.4.1 Board-state representation and decodability	10
2.4.2 Linear and nonlinear probes	10
2.4.3 Causal interventions and functional use of the representation	11
2.4.4 Functional behaviour	11
2.5 Related literature and research gap	11
3 Research Methodology	13
3.1 Research design and hypotheses	13
3.2 Dataset generation	13
3.3 Move vocabulary and sequence construction	14
3.4 Implementation of the architectures	14
3.5 Training protocol	15

3.6	Organisation of the evaluation	15
3.6.1	Next-move prediction and legality	15
3.6.2	Board-state representation	16
3.6.3	Causal intervention protocol	17
3.7	Uncertainty analysis and reproducibility	17
3.8	Computational environment	18
4	Experimental Exploration and Selection of the JEPA Configuration	19
4.1	Starting point and criteria of the exploration	19
4.2	From nested to hard-disjoint views	19
4.2.1	The initial hypothesis of a shared history	20
4.2.2	Increasing the time horizon	20
4.2.3	Contrastive learning and isolating the future	20
4.2.4	Systematic exploration of hard-disjoint views	21
4.3	Dense supervision as a second turning point	21
4.4	Selection and finalisation of the JEPA configuration	22
5	Results	24
5.1	Training completion and computational cost	24
5.2	Next-move prediction and legal behaviour	26
5.3	Decoding the board state	28
5.4	Variability across independent training runs	30
5.5	Results of the causal interventions	32
5.6	Effect of board size on functional, representational and causal performance . . .	34
6	Discussion	36
6.1	How do the architecture and the objective affect legality?	36
6.2	Do the models learn representations of the board?	37
6.3	Is the represented state used causally?	38
6.4	Do the three levels of analysis agree?	40
6.5	How does scale affect these findings?	41
6.6	Main findings and research contribution	42
6.7	Limitations and threats to validity	43
6.8	Future directions	44
7	Conclusions	45
7.1	Answers to the research questions	45
7.2	Overall contribution	46
7.3	Final assessment	46
	References	47
A	Full Hyperparameters and Configurations	49
A.1	Shared main-training protocol	49

A.2	Input dimensions per board size	49
A.3	Per-architecture settings	49
A.4	Per-objective settings	52
A.5	Parameter-count matching	52
A.6	Shared downstream evaluation protocol	53
B	Mathematical Details of the Architectures and Objectives	56
B.1	The attention mechanism in the Transformer	56
B.2	Continuous form and discretisation of the State Space Model	57
B.3	Relationship with RNNs and HiPPO	57
B.4	Selective scan and computational execution	58
B.5	Details of the Autoregressive objective	58
B.6	Details of the JEPA-like objective	58
C	Detailed Evolution of the JEPA Experiments	59
C.1	Scope and comparability	59
C.2	Early Transformer runs	59
C.3	All-position Transformer re-runs	61
C.4	Transfer to Mamba	61
C.5	Later exploratory directions	62
D	Details of the Causal Interventions	63
D.1	Construction and splitting of the counterfactual cases	63
D.2	The Li, Nanda and adapted intervention protocols	63
D.3	Head and strength selection for the adapted method	64
D.4	Metrics and limits of interpretation	65
D.5	Supplementary causal results	65
E	Reproducibility and Experimental Records	67
E.1	Computational environment and version records	67
E.2	Random seeds and independent repetitions	67
E.3	Run identity and compatibility checks	69
E.4	Organisation of experimental records and reproduction order	69
E.5	Resuming runs and remaining limits	70
F	Supplementary Analyses of the Results	71
F.1	Paired t -tests and Holm correction	71
F.2	Exact sign-flip tests and effect sizes	71
F.3	Absolute and relative encoding	73
F.4	Analysis of the relative linear probe	74
F.5	Comparison with random prediction	74
G	Core Code Definitions	76
G.1	Data generation and sequence construction	76

G.1.1	Deterministic game generation	76
G.1.2	Construction of input and target sequences	77
G.2	Definition of the Transformer	77
G.3	Definition of Mamba	78
G.4	Autoregressive training	79
G.5	JEPA training	80
G.5.1	InfoNCE and target-encoder update	81
G.5.2	Construction of nested and hard-disjoint views	81
G.5.3	Supervision at all valid positions	82
G.5.4	Historical VICReg and Smooth L1 objectives	82
G.5.5	Multimodal distance-margin loss of v6	83
G.5.6	Restricting the active implementation to the final setup	84
G.6	Frozen move-prediction heads	85
G.7	Board-state probes	86
G.8	Causal intervention in the residual stream	87
G.8.1	Li method: optimising the internal representation	87
G.8.2	Nanda method: intervention on selected residual branches	88
G.8.3	Adapted method: successive intervention after every block	89

List of Figures

2.1	The flow of a move in Othello	5
2.2	Two modes of information flow: Transformer and Mamba	6
2.3	Autoregressive and JEPA-like training objectives	8
2.4	Three evaluation paths for the internal representation	10
3.1	Fixed split of the 238 shards	16
4.1	Evolution of the JEPA experimental exploration	22
4.2	Best top-1 legality and relative-state accuracy during JEPA development	23
5.1	Training time per experimental condition	25
5.2	Evaluation time per experimental condition	25
5.3	Legal moves among the first k predictions	28
5.4	Top-1 legality by game phase (heatmap)	29
5.5	Relative-state linear-probe accuracy across layers	30
5.6	Variability over five independent training runs	31
5.7	Change in counterfactual legal mass for the three intervention methods	34
5.8	Legal mass and relative-state probe accuracy across board sizes	35

List of Tables

2.1	Comparative overview of the related literature and the position of this thesis . . .	12
3.1	Mapping of research questions to evaluation evidence	14
5.1	Final legal-move prediction performance	26
5.2	Mean proportion of legal moves among the first k predictions	27
5.3	Top-1 legality by normalised game phase	28
5.4	Decoding of the relative board state with a linear probe	30
5.5	Mean and standard deviation over five independent seeds	31
5.6	Matched effects in percentage points	32
5.7	Change in probability mass on the counterfactual legal set	33
A.1	Shared hyperparameters and operational settings of the main training	50
A.2	Vocabulary, sequence and main-training-set dimensions per board size	50
A.3	Final architectural settings of the Transformer and Mamba encoders	51
A.4	Objective-specific settings of the final Autoregressive and JEPA-like comparison .	52
A.5	Memory-chunking settings for the JEPA-like all-position objective	53
A.6	Exact parameter count of the base architecture, including the native output projection	53
A.7	Parameter-count structure of the final JEPA-like checkpoints	53
A.8	Fixed split of the 238 shards in the shared evaluation protocol	54
A.9	Fixed training and final-test settings of the frozen next-move heads	54
A.10	Hyperparameter grid of the frozen heads	55
A.11	Full hyperparameters of the board-state probes	55
C.1	Main configurations of the development sequence v1–v6 and of the final objective	60
C.2	Early Transformer development results on 8×8 under the historical evaluation protocol	60
C.3	All-position Transformer results on 8×8 with <code>unified_eval_v2</code>	61
C.4	Main Mamba-JEPA development results on 8×8	61
C.5	Later JEPA directions not included in the main experimental setup	62
D.1	Shared settings for constructing and evaluating the counterfactual cases	63
D.2	Defining elements of the three causal intervention protocols	64
D.3	Mapping of the intervention sites by normalised depth	64

D.4	Head and strength α selected without access to the 1,000 final-test cases	65
D.5	Change in top-1 legality on the counterfactual board	66
D.6	Change in false positives plus false negatives among the top N predictions	66
E.1	Software versions of the seeded environment locks and hardware for training and shared evaluation	68
E.2	Allocation of seeds to the successive stages of the experiment	68
E.3	Metadata that link each result to its source	69
E.4	Minimal execution order for reproducing the main results	70
F.1	Matched effects, t -based confidence intervals and paired t -test p -values	72
F.2	Exact sign-flip tests, direction agreement and Hedges' g_z	72
F.3	Linear and MLP probes of the absolute encoding	73
F.4	Linear and MLP probes of the relative encoding	73
F.5	Per-class and occupied-square performance of the relative linear probe	74
F.6	Selected and final layer of the relative linear probe	74
F.7	Legality against uniform random selection from the vocabulary	75

Abbreviations and Acronyms

Abbreviation	Meaning
AdamW	Adam with decoupled weight decay (optimiser)
AR	Autoregressive (training objective)
CE	Cross-entropy
EMA	Exponential moving average
GELU	Gaussian Error Linear Unit (activation function)
GPT	Generative Pre-trained Transformer
GPU	Graphics processing unit
HD	Hard-disjoint (separated input and target views)
HiPPO	High-order Polynomial Projection Operators
InfoNCE	Information Noise-Contrastive Estimation (contrastive objective)
JEPA	Joint-Embedding Predictive Architecture
LayerNorm	Layer normalisation
LLM	Large Language Model
MLP	Multi-layer perceptron
P@k	Precision at k (proportion of legal moves among the first k predictions)
RNN	Recurrent neural network
SSM	State space model
VICReg	Variance-Invariance-Covariance Regularization
VRAM	Video random access memory (GPU memory)

Chapter 1

Introduction

Has a model that predicts the next legal move in a game with high accuracy learned the rules of the game, or has it merely detected statistical regularities in the sequences it observed? The behaviour of the model alone is not enough to answer this question. A system can produce correct predictions without it being apparent what information it has represented internally, how it organises that information and to what extent it uses it to make its decisions.

This question lies at the core of interpretability in modern Artificial Intelligence. As neural models acquire ever more parameters and are applied to ever more complex tasks, evaluating them only by their final performance offers a limited picture of how they work. Analysing internal representations can reveal which properties of the data have been encoded (Alain and Bengio, 2018), but it does not prove that these representations take part in producing the predictions. Three properties must therefore be distinguished: behavioural competence, the decodability of information from the internal representations, and the causal use of that information.

Othello offers a particularly suitable environment for studying these questions. Its rules are fully specified and the board state can be reconstructed exactly from the sequence of moves. Every hypothesis about what the model has represented can thus be tested against a fully known reference state.

The first relevant study trained a variant of the Generative Pre-trained Transformer (GPT) to predict Othello moves (Li et al., 2023). Although the model received no explicit description of the board or of the rules, the board state was decoded from its internal representations with nonlinear probes, and interventions showed that modifying this state affected its predictions. A later re-examination showed that the apparently nonlinear structure depended to a large extent on how the state was described (Nanda, Lee and Wattenberg, 2023): when the pieces were encoded relative to the player whose turn it is, the state became largely linearly decodable. This allowed more direct interventions and provided stronger evidence of a causal link with behaviour.

These findings provide important evidence that a sequence model can develop an internal world model even when it is trained exclusively on move sequences. A broader question arises, however: is this behaviour a general property of learning from sequences, or does it depend on the conditions under which it was observed? Earlier studies focused on Transformers trained with an

AR objective on the classic 8×8 board.

The first dimension examined is the architecture. Transformers rely on self-attention, which allows direct interaction between the positions of a sequence, but at a cost that is quadratic in its length (Vaswani et al., 2017; Tay et al., 2022). This limitation has fuelled research into alternative architectures. One of these is State Space Models (SSMs), and in particular the Mamba architecture, which uses selective state mechanisms and scales linearly with sequence length (Gu and Dao, 2024). Beyond computational cost, the two architectures process and retain information through different mechanisms, which raises the question of whether they also organise the internal representation of the state differently.

The second dimension concerns the training objective. In AR training the model predicts the next move as a discrete token. JEPA instead predicts within the representation space, on the hypothesis that the model will thereby focus on more abstract and semantically useful properties of the data (LeCun, 2022). These approaches were developed for learning from images and video (Assran et al., 2023) and have recently been transferred to discrete language tokens (Huang, LeCun and Balestrieri, 2025). It remains an open question, however, how a JEPA-like objective affects the learning of discrete, dynamic states in a controlled environment such as Othello.

The third dimension concerns the board size. The classic 8×8 board is a small environment, both in terms of its state space and in terms of sequence length. In this thesis we extend the experiment to 12×12 and 16×16 boards, where the number of squares grows from 64 to 144 and 256 and the maximum length of a game from 60 to 140 and 252 moves. The rules remain unchanged, so that the kind of problem stays constant, but the information that must be retained and combined for each prediction increases. Scaling therefore makes it possible to examine whether the internal representations remain accurate as the game becomes more demanding, and whether differences that are marginal on 8×8 become visible at greater complexity.

We combine these three dimensions in a controlled factorial comparison: two architectures, two training objectives and three board sizes, with shared data and a single evaluation protocol. The evaluation is organised in three complementary levels. First, the behavioural performance of the models is examined, specifically their ability to predict legal moves. Next, using probes, we assess the extent to which the board state can be decoded from their internal representations. Finally, through controlled causal interventions, we investigate whether the decodable information actually takes part in producing the predictions. This combined evaluation makes it possible to examine whether a more accurate representation of the board is associated with better functional performance, and to what extent there is a causal relationship between the two.

Our aim is to investigate how the architecture, the training objective and their interaction affect these three properties as the complexity of the game increases. The research questions are as follows:

RQ1. How do the architecture and the training objective affect the prediction of legal moves?

RQ2. To what extent is the board state decodable from the internal representations of each model?

- RQ3.** Is the represented information used causally in selecting the next move?
- RQ4.** Do behavioural performance, the decoding accuracy of the board state and the results of the causal interventions agree with one another?
- RQ5.** How do these relationships change as the size and complexity of the game increase?

Chapter 2

Theoretical Background and Literature Review

This chapter organises the theoretical framework around three elements that connect the move sequence with the internal representation of the board: the environment, the architecture and the training objective. First, the relationship between the observable tokens and the not directly observable state of Othello is clarified. Next, the Transformer and Mamba are presented, Autoregressive and JEPA-like learning are compared, and the concepts of decodability, causal use of information and functional behaviour are introduced. Finally, these elements are connected to the relevant literature and the research gap is delineated.

2.1 Othello as an environment for studying world models

The research value of Othello lies in an asymmetry of knowledge: the researcher can fully reconstruct the board state at every step, whereas the model sees only sequences of discrete symbols. It is not given the rules, the existence of a board, or the geometric meaning of the tokens. Any information about the state must therefore be inferred from the statistical structure of the games. Because the true state is known, the internal representations can be compared against a clear reference point instead of relying on vague semantic interpretations. This distinction makes Othello a suitable environment for studying the emergence of world models (Li et al., 2023).

2.1.1 Rules and dynamics of the game

In Othello, two players alternately place black and white pieces on an 8×8 board. A move is legal when it encloses opposing pieces between the new piece and a piece of the same colour; the enclosed pieces are then flipped. Each move therefore changes the board, and the set of legal options, non-locally. If a player has no legal move, they lose their turn, and the game ends when neither player can move. The initial position and a legal change of the board are shown in Figure 2.1.

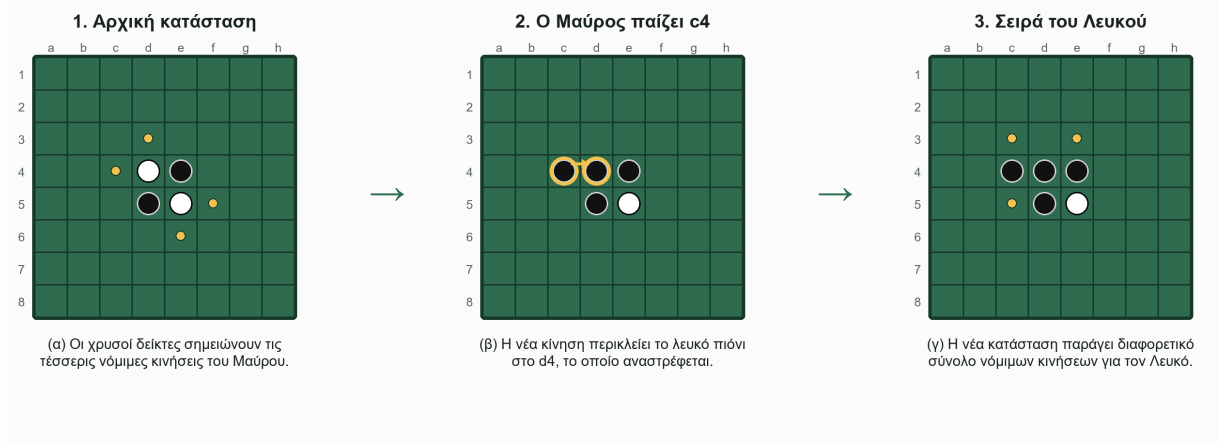


Figure 2.1: The flow of a move in Othello. Gold markers indicate legal squares, while the gold outline highlights the new and the flipped piece.

2.1.2 From the move sequence to the board state

In the data used in this thesis, each game is given only as a sequence of moves $a_1, a_2, \dots, a_t$, where each token denotes the square on which a piece is placed. The state s_t is not provided to the model; it is reconstructed deterministically from the initial position and the transition rules, together with the set of legal moves:

$$s_t = T(s_0, a_1, a_2, \dots, a_t) \quad (2.1)$$

Forced passes are not recorded as tokens; they follow from the absence of a legal move. Because the mapping is fully verifiable, the evaluation distinguishes two questions: whether the board state can be decoded from the internal representations, and whether the model’s predictions remain legal.

Random games do not provide a consistent strategic signal about which legal move is preferable. They do, however, provide a clear signal about which moves are allowed in each state. Legality therefore measures the learning of the constraints of the game and not strategic ability (Li et al., 2023). The same framework is applied to 8×8 , 12×12 and 16×16 : the rules remain fixed, while the available squares, the length of the game and the information that must be retained all increase.

2.2 Sequence modelling architectures

For an Othello model, the architecture determines how each new move is connected to the history and how the information required for the next prediction is carried forward. In the Transformer, each position draws information directly from all previous moves through causal self-attention. In Mamba, the history is carried sequentially through an internal state that is selectively updated at each step. The comparison therefore concerns two different inductive biases regarding memory and information flow, not merely two different computational costs. Figure 2.2 summarises the difference: neither architecture contains the board or the rules of Othello by construction.

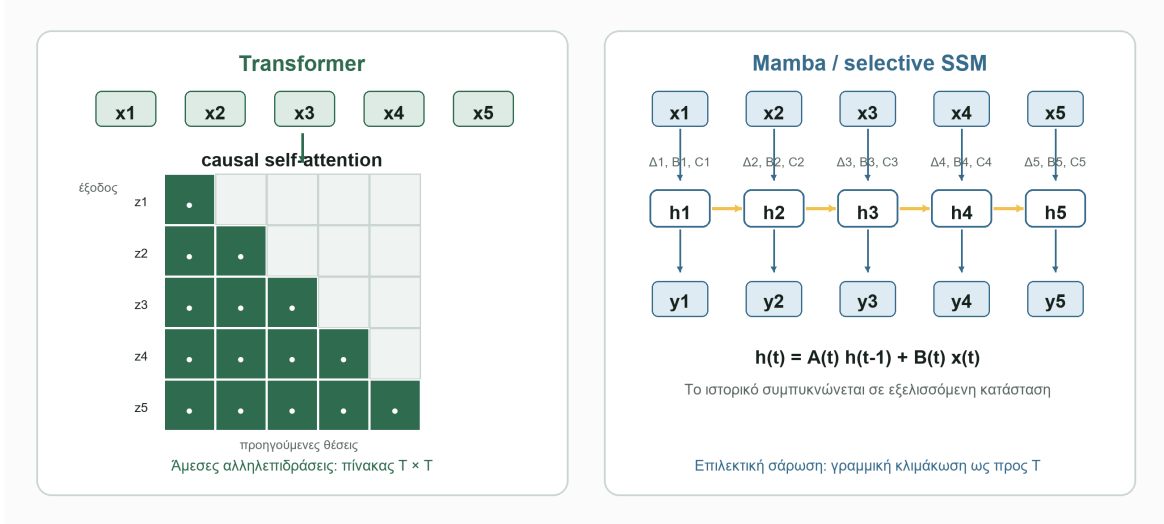


Figure 2.2: Two modes of information flow. In the Transformer, each position is connected directly to all permitted previous positions through causal self-attention. In Mamba, the history is carried through a selectively updated state. The diagram is schematic and does not imply that the state h_t coincides with the board state.

Despite their different internal operation, both architectures produce a vector representation for each position and layer. This shared interface allows prediction heads and probes of the same form to be used at corresponding points of the models. The comparison can thus focus on the mechanism that processes the history, without being confounded by the form of the input or of the required output.

2.2.1 Transformer

The Transformer replaces recurrence and convolutions with attention mechanisms, allowing the positions of a sequence to be processed in parallel during training (Vaswani et al., 2017; Tay et al., 2022). In the decoder-only form used for next-token prediction, each move a_t is converted into an embedding and combined with positional information, since self-attention on its own does not distinguish the order of the elements.

The core of each layer is scaled dot-product attention. Queries Q , keys K and values V are produced as linear projections of the input, and the output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V \quad (2.2)$$

The essential element is the product $QK^\top$, whose result expresses how important each position of the sequence is for every other position. The matrix M is a causal mask that blocks access to later moves, so that the representation at position t depends only on $a_1, \dots, a_t$. In Othello this is critical: without the mask, the model could exploit future moves to reconstruct the current state. The more detailed role of Q , K and V , multi-head attention and residual connections are presented in Appendix B.1.

Direct access to the history does, however, carry a substantial computational cost. For a sequence

of length T the matrix $QK^\top$ has dimensions $T \times T$, so the computational and memory costs grow quadratically with the length (Vaswani et al., 2017; Tay et al., 2022). In 8×8 Othello the maximum sequence length is 60 moves, whereas on 16×16 it reaches 252: the length increases by roughly 4.2 times, but the potential number of computations by roughly 17.6 times. The larger board thus also acts as a scaling test, which makes the comparison with a linear-complexity architecture meaningful.

2.2.2 State Space Models and Mamba

SSMs start from a different description of the sequence: instead of explicitly computing relationships between all pairs of positions, they model a dynamical system whose internal state evolves as new data arrive. In discrete form, each step combines the previous state with the new token:

$$h_t = A_d h_{t-1} + B_d x_t, \quad y_t = C h_t + D x_t \quad (2.3)$$

The matrix A controls what is retained from the past, B what is introduced from the current input and C what is read out from the state. The term D is a direct input-to-output connection that bypasses the state: it allows the current move to affect the output without passing through memory, leaving the state free for longer-range dependencies. The continuous formulation from which this form is derived, and the discretisation procedure with step Δ , are presented in Appendix B.2.

Because each step requires only the previous state and the new token, and not a $T \times T$ matrix, the total cost grows linearly with T for a fixed state dimension. The sequential update bears a functional resemblance to RNNs, although the two families differ substantially in their parameterisation and in how they retain the state (Appendix B.3). Structured variants such as S4 introduced a special parameterisation of the matrix A so that the dynamics are at once expressive, numerically tractable and efficient on long sequences (Gu, Goel and Ré, 2022).

Mamba extends earlier SSMs by introducing *selectivity*: the parameters Δ , B and C depend on the current input, allowing the model to decide which information to admit, retain or attenuate (Gu and Dao, 2024). The update is implemented with a selective scan and retains linear scaling; the details of the execution are presented in Appendix B.4.

2.3 Training objectives

The training objective determines which prediction the model is asked to make and, therefore, which error is used to update its parameters. The same move sequence can provide two different learning signals: the Autoregressive objective asks for the next move to be predicted as a specific token, whereas the JEPA-like objective asks for its representation to be predicted. The difference is summarised in Figure 2.3 and forms the second axis of the comparison.

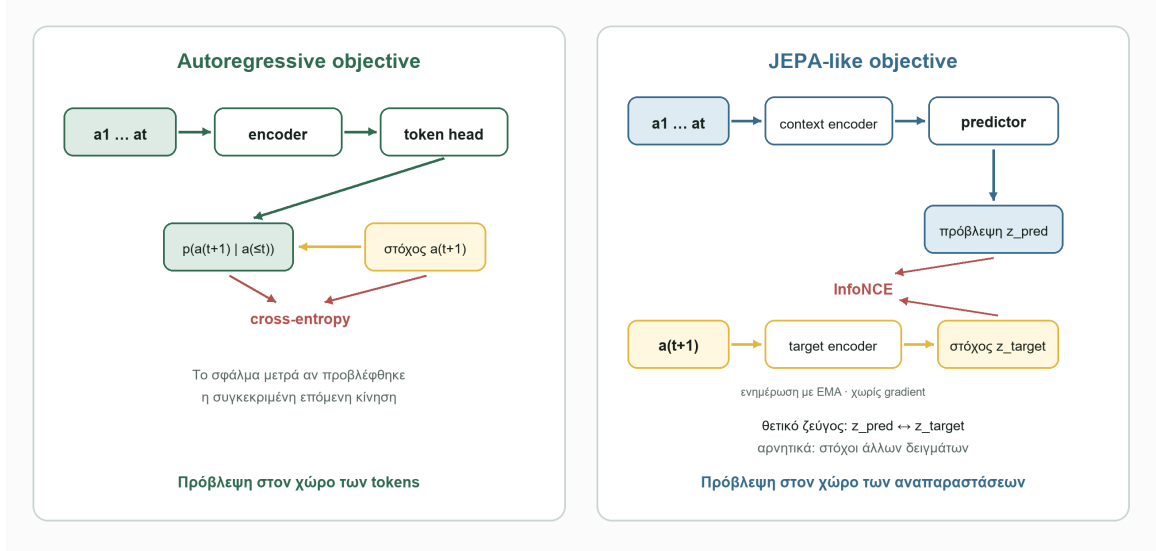


Figure 2.3: Different learning signals on the same sequence. The Autoregressive objective evaluates the probability of the next move, while the JEPA-like objective compares the predicted representation with the target representation.

2.3.1 Autoregressive next-move prediction

For a move sequence $a_1, \dots, a_T$, Autoregressive modelling factorises the probability of the whole sequence into successive probabilities, each of which depends only on the moves that precede it:

$$p(a_1, \dots, a_T) = \prod_{t=1}^T p(a_t | a_1, \dots, a_{t-1}) \quad (2.4)$$

During training, the model receives a prefix $a_1, \dots, a_t$ and produces logits over all possible board squares. After the softmax, the logits are converted into a probability distribution over the next move, and the cross-entropy loss rewards the probability assigned to the recorded move a_{t+1} :

$$\mathcal{L}_{AR} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log p_{\theta}(a_{t+1} | a_1, \dots, a_t) \quad (2.5)$$

Because the causal mask prevents each position from drawing information from future moves, all terms of the loss are computed in parallel with a single forward pass, without violating the causal direction of prediction. Details of the training procedure are presented in Appendix B.5.

2.3.2 JEPA-like prediction in representation space

The key idea of JEPA is to predict the target not in its original form but in representation space. Instead of reconstructing every detail of a token or an image, the model learns which representation of the target is predictable from the available context (LeCun, 2022; Assran et al., 2023). The idea has since been transferred to discrete tokens as well: LLM-JEPA adds a predictive term in representation space on top of the standard autoregressive objective of language models and reports improvements on four datasets (Huang, LeCun and Balestrieri, 2025).

In adapting the idea to Othello sequences, the context encoder processes the history $a_1, \dots, a_t$ and produces a representation z_c . A predictor q_φ transforms this representation into a prediction $\hat{z}_{t+1}$, while a target encoder produces the target representation z_{t+1} :

$$z_c = f_\theta(a_{1:t}), \quad \hat{z}_{t+1} = q_\varphi(z_c), \quad z_{t+1} = f_\xi(a_{t+1}) \quad (2.6)$$

The two encoders are not updated symmetrically. The context encoder and the predictor receive gradients from the loss, while the target encoder is kept outside the gradient path and its parameters are updated as an exponential moving average of the context encoder’s parameters, providing a more stable reference point (Appendix B.6).

The critical difference from the Autoregressive objective lies in what is being asked. The JEPA-like model is not penalised for failing to assign the highest probability to the correct square, but because the prediction $\hat{z}_{t+1}$ does not agree with the target representation z_{t+1} . The encoder is thus trained to make the future predictable within representation space. The exact construction of the context and target views is presented in the Methodology.

2.3.3 InfoNCE and collapse avoidance

If training only required two vectors to move closer together, there would be a trivial solution: the encoder could produce almost the same representation for every input. Prediction would then be easy, but the representations would not retain useful differences between moves or states. This phenomenon is called representation collapse (embedding collapse).

The InfoNCE loss addresses this risk by turning prediction into a discrimination problem (Oord, Li and Vinyals, 2019). For each predicted representation $\hat{z}_i$, the corresponding target representation z_i forms the positive pair, while the targets z_j of other samples act as negatives. After the vectors are normalised, their similarity is scaled by the temperature τ :

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{N} \sum_{i=1}^N \log \left[\frac{\exp(\text{sim}(\hat{z}_i, z_i)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(\hat{z}_i, z_j)/\tau)} \right] \quad (2.7)$$

Minimising the loss increases the relative similarity of the positive pair against the negatives. If all representations collapsed to the same vector, the model could not distinguish the correct target and the distribution over the candidate pairs would become almost uniform.

A clear terminological distinction is needed here. The original I-JEPA uses a JEPA structure without negative samples (Assran et al., 2023). This thesis retains the context encoder, the slowly updated target encoder and the predictor, but combines this structure with InfoNCE. For this reason the objective is described as JEPA-like and not as a verbatim application of I-JEPA.

2.4 Internal representations, interpretability and functional behaviour

Internal representations become valuable for interpretability when they can be linked to clear properties of the environment and, subsequently, to the behaviour of the model. In Othello this link can be examined at three levels: which information about the board is decodable, how directly it can be extracted, and whether a targeted change to it affects the predictions. Functional performance completes the picture by checking whether the final predictions respect the rules of the game. Figure 2.4 summarises the relationship between the three checks.

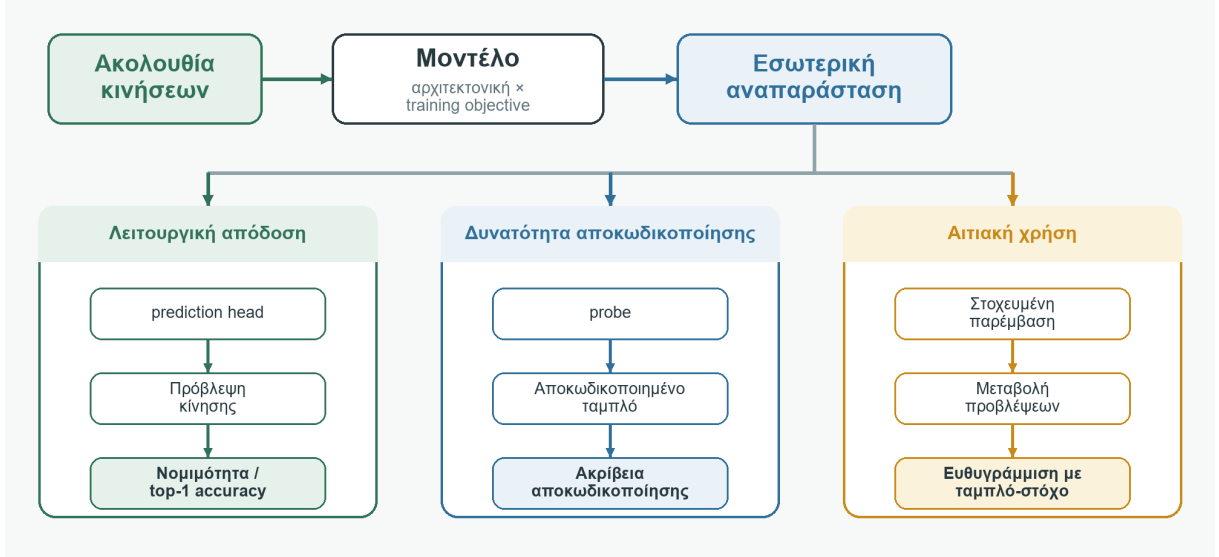


Figure 2.4: Three complementary paths for evaluating the internal representation: functional performance, decodability and causal use.

2.4.1 Board-state representation and decodability

The board state can be described in an absolute or a relative frame. In the absolute frame, each square is labelled black, white or empty. In the relative frame, it is described as mine, yours or empty, with the player to move as the reference point. The two descriptions contain the same information when the turn order is known, but they are not equivalent as decoding targets. The absolute description was recovered mainly with nonlinear probes (Li et al., 2023), whereas the relative description proved to be largely linearly decodable (Nanda, Lee and Wattenberg, 2023).

Saying that the state is decodable from a layer means that a classifier can predict the contents of the squares from that layer’s representations on data not used to train it. Accuracy must, however, be interpreted against a suitable baseline. In Othello many squares are empty, particularly early in the game, so a classifier that systematically predicts the majority class can show apparently high performance without making any substantive use of the representation.

2.4.2 Linear and nonlinear probes

Probes are diagnostic machine learning models trained on the internal representations while the base model is kept frozen. A linear probe tests whether the required information can be

recovered through a linear transformation. A nonlinear probe has greater expressive capacity and can recover more complex mappings (Alain and Bengio, 2018).

The strength of the probe affects the conclusion. A very expressive probe may learn part of the problem itself instead of simply revealing information that is already organised in the representation. For this reason, the success of a simple probe is stronger evidence that the information is directly accessible, but it is not on its own proof of overall interpretability or of functional use (Belinkov, 2022). The specific probe architectures and their training protocol are described in the Methodology.

2.4.3 Causal interventions and functional use of the representation

Causal interventions examine what happens when the representation is changed in a controlled way. If an intervention moves the represented state towards a target board and the predictions shift systematically towards the legal moves of that board, the result strengthens the evidence that the represented information takes part causally in the computation. The relevant literature has used both gradient-based changes to the activations (Li et al., 2023) and linear directions of the relative representation (Nanda, Lee and Wattenberg, 2023).

A mere change in the output is not sufficient, because any sufficiently large perturbation can affect the predictions. Controls are needed that separate the targeted semantic change from the general effect of the perturbation, such as a comparison with the unmodified model and with a random intervention of matched magnitude. If a particular intervention does not change behaviour, this does not automatically prove that the information is functionally inert; the result may also depend on the layer, the direction or the strength of the intervention. The exact protocol is presented in the Methodology.

2.4.4 Functional behaviour

The third level concerns the output of the model. When more than one move is legal, agreement with the single move recorded in a game does not fully capture knowledge of the rules. The legality of the prediction is a more direct criterion: it checks whether the model has restricted its output distribution to the options permitted by the current state.

Autoregressive models intrinsically have a probability distribution over the move vocabulary, whereas JEPA-like models predict in representation space. A fair behavioural comparison therefore requires a shared evaluation interface, without confounding the quality of the encoder with the training of the head. How this interface is constructed is presented in the Methodology.

2.5 Related literature and research gap

Othello was established as a controlled environment for studying world representations through a GPT-style model trained exclusively on move sequences. Nonlinear probes recovered the absolute board state, while gradient-based interventions on the activations shifted the predictions towards moves that were legal for the modified board (Li et al., 2023).

The same information proved to be largely linearly decodable when the state was expressed relative to the player to move. This linear description enabled interventions by adding directions to the residual stream (Nanda, Lee and Wattenberg, 2023). The relationship between the linear representation, its causal effect and the capacity of the Transformer was then examined in models of different depth. Here “depth” refers to the number of layers of the model and not to the board size, which remained 8×8 (Hazineh, Zhang and Chiu, 2023).

Later studies broadened the range of models and criteria. Seven pre-trained language models were examined for their ability to recover features of the board layout across different architectures and scales (Yuan and Søgaard, 2025). A different line of work compared RNNs, LSTMs, Transformers, Mamba and Mamba-2, showing that high performance on sequence prediction can coexist with incomplete alignment with the full world model (Vafa et al., 2025). This finding reinforces the need to evaluate behaviour and internal representation separately.

At the software level, a reproduction of OthelloGPT with Transformer, Mamba and Jamba, using linear probes on the relative board description, has also been presented (Torres, 2024). It is a useful technical precedent rather than a peer-reviewed study: it is limited to 8×8 , uses an Autoregressive objective and does not include causal interventions.

Despite the growth of the literature, two axes remain unconnected. Comparisons of architectures as a rule retain the Autoregressive objective, while studies of different representational objectives do not systematically examine Transformer and Mamba under shared conditions. At the same time, all of these works remain on the 8×8 board. This thesis addresses the gap by combining the axes of architecture, training objective and board size in a shared factorial design, with behavioural, representational and causal evaluation. Table 2.1 summarises the main questions and the research gap.

Table 2.1: Comparative overview of the related literature and the position of this thesis

Study	Architecture / objective	Board scale	Evaluation	Main focus
Li et al. (2023)	GPT-like Transformer / Autoregressive	Othello 8×8	Nonlinear probes; gradient-based interventions	Board representation and functional effect.
Nanda, Lee and Wattenberg (2023)	Transformer / Autoregressive	Othello 8×8	Linear probes; linear interventions	Relative encoding and linear structure.
Hazineh, Zhang and Chiu (2023)	Transformers of different depth / AR	Othello 8×8	Linear probes; causal steering	Effect of depth and capacity; not of board size.
Yuan and Søgaard (2025)	7 pre-trained language models of different scale	Othello 8×8	Recovery of layout features	Testing the world-model hypothesis on a broader set of models.
Vafa et al. (2025)	RNN, LSTM, Transformer, Mamba, Mamba-2	Othello 8×8	Behaviour versus alignment with the world model	Good prediction does not guarantee a complete internal representation.
Torres (2024)	Transformer, Mamba, Jamba / AR	Othello 8×8	Linear probes; no causal interventions	Technical reproduction; not peer reviewed.
This thesis	Transformer, Mamba / AR, JEPA-like	Othello 8×8 , 12×12 , 16×16	Performance; probes; causal interventions	Factorial design of architecture $\times$ objective $\times$ scale.

Chapter 3

Research Methodology

The central methodological challenge is to train the twelve main experimental conditions while keeping the comparison fair when the architecture, the training objective and the board size change at the same time. For this reason we use shared data generation, predefined splits and a single evaluation protocol. Where the systems necessarily differ, the difference is kept explicit, so that the results are interpreted at the correct level.

3.1 Research design and hypotheses

The study follows a $2 \times 2 \times 3$ factorial design. The three factors are the model architecture (Transformer or Mamba), the training objective (AR or JEPA) and the board size (8×8 , 12×12 or 16×16). The evaluation covers three aspects of how the models operate: the prediction of legal moves, the decodability of the board state and the change in predictions after a controlled causal intervention.

To limit the influence of other factors, we use shared data per board size, a shared `d_model`, the same core optimisation settings and a fixed evaluation protocol. Because a Transformer layer and a Mamba layer do not contain the same number of parameters, the number of layers is adjusted so that the compared models have a similar total number of trainable parameters. The exact configurations are collected in Appendix A.

The study is comparative and exploratory in character, examining the individual effects of the architecture and the objective, their interaction, and how the observed differences change as the board size increases. Table 3.1 summarises how each research question is connected to the corresponding evaluation level and to the evidence used to answer it.

3.2 Dataset generation

For each board size we used 238 consecutive shards of roughly 100,000 random but fully legal games each. For the original 8×8 board we used the 238 existing files of the synthetic dataset released with the implementation of Li et al. (2023). Following a similar core procedure, in which each move is chosen uniformly from the available set of legal moves, we generated new datasets

Table 3.1: Mapping of research questions to evaluation evidence

Question	Dimension	Main evidence
RQ1	Behaviour	Top-1/top- k legality and probability of legal moves
RQ2	Representation	Linear and MLP probes on absolute and relative encoding
RQ3	Causal use	Directed intervention against null and random controls
RQ4	Relationship between measurements	Joint analysis of functional performance, probes and interventions
RQ5	Scaling	Comparison of 8×8 , 12×12 and 16×16 with shared budgets

for the 12×12 and 16×16 boards. Their generation is deterministic, so that every game can be reproduced exactly (Appendix G.1.2). When the player to move has no legal move, the engine switches the active player internally; no pass token is stored. If the opponent has no legal move either, the game ends.

After generation, samples from different points of the corpus are replayed and checked by an independent rules engine for legality, length and termination. These checks guard against silent data corruption and make it possible to demonstrate that all models are compared on the same corpus.

3.3 Move vocabulary and sequence construction

Games are stored as sequences of integers that correspond to board positions numbered row by row, from left to right and from top to bottom. The four central squares of the initial state cannot be a move and are removed from the vocabulary. For a board of side n , the possible moves, the total vocabulary including the input padding token, and the maximum input length are defined as follows:

$$N_{\text{move}} = n^2 - 4, \quad |V| = n^2 - 3, \quad T_{\text{max}} = n^2 - 5 \quad (3.1)$$

Shorter sequences are filled with a padding token, which is excluded from the loss. The same tokenisation is used in the AR and JEPA models, so that the objective changes without changing the observed problem. The construction of input and target sequences is given in Appendix G.1.2.

3.4 Implementation of the architectures

The two architectures are implemented as separate PyTorch models, but they expose a shared external interface for inputting sequences and extracting internal representations. This allows core parts of the experimental infrastructure to be reused within the training loops. The forward pass and the processing of the sequence nevertheless remain different for the Transformer and Mamba, while the loss computation is adapted to the respective training objective. The definitions of the two models are given in Code Listings G.2 and G.3.

3.5 Training protocol

We use the experimental framework of Othello-GPT (Li et al., 2023) as a starting point and extend it with a single protocol for the comparative evaluation of different architectures, training objectives and board sizes. Each main training run consists of one full pass over the first 200 shards and is carried out in bf16 numerical precision (Appendix A.1). Optimisation is based on AdamW, with an initial linear warm-up period followed by cosine decay of the learning rate. The exact hyperparameters, the per-architecture settings and the parameter-matching procedure are collected in Appendix A.

In the AR models, the native head is trained jointly with the encoder through cross-entropy on the next move. In the JEPA-like models of this thesis, every valid prefix is used to predict the representation of the immediately following move, which is not included in the input of the context encoder. The context encoder and the linear predictor are updated through InfoNCE with temperature 0.1, while the target encoder receives no gradients and is updated through an Exponential Moving Average (EMA) with momentum 0.996. Supervision is applied at all valid time positions, with padding positions excluded. The InfoNCE negatives come from other games in the same training batch at the same absolute time position, so that position cannot serve as a trivial discrimination cue. This final configuration emerged from the development experiments presented in Chapter 4. The code for AR and JEPA training is given in Code Listings G.4 and G.5.

Training proceeds shard by shard and, after an interruption, is resumed from a checkpoint only if the numerical precision and the experiment configuration fully match the original ones (Appendix E.5).

3.6 Organisation of the evaluation

The evaluation is organised in three successive and complementary levels. Functional performance shows whether the model produces predictions compatible with the rules. Probes examine how cleanly the board state can be extracted from the internal representations. Causal interventions examine whether a selective change to the represented state causes a corresponding change in the prediction. To keep the final evaluation independent, the shards used for validation and for selecting the settings of head and probe training are different from those used in the final test. For this purpose, the 238 shards are divided from the outset into four predefined partitions. The flow, and the option of topping up the head-training shards, are shown in Figure 3.1.

3.6.1 Next-move prediction and legality

The main behavioural measurement is whether the move with the highest probability belongs to the true set of legal moves in the current state. The state is reconstructed by replaying the sequence, and the scorer takes the active player into account even when an implicit pass has occurred. We also report the fraction of legal moves within the top- k predictions for k equal to 1, 2, 3 and 5, the total probability assigned to all legal moves (probability mass), performance by game phase, and the normalised lift over random selection from the vocabulary.

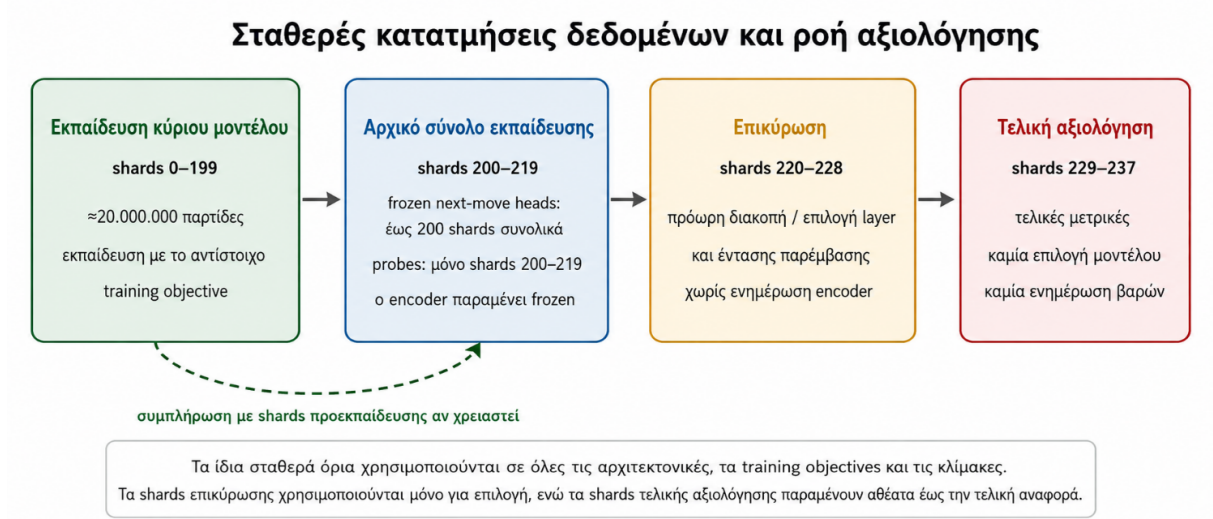


Figure 3.1: Fixed split of the 238 shards into pretraining, downstream training, validation and final test.

The AR models are evaluated with the native head they contain by construction. The JEPA encoders do not have one, so a linear and an MLP frozen head are fitted to them. The encoder remains unchanged and only the head is trained, on data that are never used for model selection or for the final evaluation. Final legality is computed on 50,000 games from the test split. The training details (data partitions, hyperparameter selection, early stopping) are given in Appendix A.6 and the code of the heads in Code Listing G.6.

The shared form of the metrics does not imply identical head training: in AR the head learns together with the encoder, whereas in JEPA it learns on top of an already trained, frozen encoder. The behavioural comparison therefore concerns the complete prediction systems.

3.6.2 Board-state representation

For each encoder, the representation of the last position of the prefix is extracted immediately after the embeddings, as well as after each layer of the model. Linear and MLP probes are trained independently on these frozen representations. Each probe predicts three classes for each square, for both the absolute and the relative encoding of the board as defined in Section 2.4. A LayerNorm is applied to the input of the main probes, so that differences in scale between architectures do not turn into an artificial advantage. The implementation of the probes is given in Code Listing G.7.

The sampling budget is fixed across board sizes: positions from all phases of each game are sampled, and the training, validation and test sets have the same size on every board (Table A.11). At each position, however, the state of every square is predicted. Thus 16×16 provides four times as many individual labels as 8×8 , in proportion to its fourfold area, while each square is supervised by the same number of examples.

Layer selection is carried out exclusively on the validation split, with macro-balanced accuracy as the criterion, i.e. the unweighted mean of the recall of the three classes. This metric limits the

dominance of the most frequent class, the empty squares.

The comparison of absolute and relative encoding with linear and MLP probes is presented as supplementary material in Appendix F. Because of limited computational resources, bootstrap intervals are not computed for the probe or causal-intervention metrics; uncertainty across training runs is examined separately.

3.6.3 Causal intervention protocol

The causal evaluation is applied to all twelve experimental conditions. In each case, a prefix and an occupied square are selected, and the piece on that square changes side. The case is kept only if the counterfactual board has at least one legal move and its set of legal moves differs from the original one. For each board size, two non-overlapping sets are constructed: 200 cases used only where configuration selection is required, and 1,000 cases for the final evaluation of all methods.

Three distinct protocols are compared. The Li method uses a nonlinear probe of the absolute state and directly optimises the internal representation at the selected depths so that the probe outputs the counterfactual state. The Nanda method takes, from a raw linear probe of the relative state, the normalised vector of the target class and adds it with a fixed strength to selected residual branches. The adapted method of this thesis uses the difference between the vectors of the target class and the original class, applies the intervention after every block and scales it by the standard deviation of the representation. Only in the adapted method is the strength α selected, on the 200 selection cases. The constants of the three methods are given in Table D.2.

In the Transformer-AR 8×8 condition, the published constants and intervention sites of the Li and Nanda methods are retained; in the remaining conditions the sites are mapped according to their relative position in the stack (Appendix D.2). The applications to JEPA, Mamba and larger boards are therefore treated as explicit extensions and not as verbatim reproductions of the original experiments. The AR models use the native head, while in JEPA the frozen head is selected in advance on the validation split and remains the same across all three methods (Appendix D.3).

In all three protocols, the intervention is evaluated on the same 1,000 cases and compared with the unmodified run; the adapted method additionally includes a geometrically matched random control. The same random permutation of coordinates and sign flip is applied to the directions of all blocks, so that the norms and the angles between them are preserved. The effect is assessed by the fraction of top-1 predictions that are legal on the counterfactual board, the total probability assigned to the new legal set, and the sum of false positives and false negatives among the top N predictions, where N is the number of new legal moves. The three exact procedures are presented in Code Listing G.8.1.

3.7 Uncertainty analysis and reproducibility

Uncertainty is examined at two levels. At the first level, we estimate how much the mean top-1 legality and probability mass are affected by the particular 50,000 games of the final test. For

this purpose, the games are resampled 1,000 times with replacement (bootstrap), and the central 95% of the resulting means is reported as the 95% confidence interval. These intervals capture the variability of the evaluation sample and not the variability that arises from training the model.

At the second level, we examine the variability across independent training runs. For each experimental condition, five full repetitions with different seeds are carried out at the 8×8 and 16×16 sizes. The seed is treated as a shared block in the 2×2 factorial design, so that comparisons between architectures and training objectives are based on matched repetitions. We compute the main effects of the architecture and the training objective, their interaction, 95% confidence intervals based on the t distribution, the effect size Hedges' g_z , paired t -tests and, as a supplement, exact sign-flip tests. The Holm correction is applied to families of multiple tests. The 12×12 size takes part in the comparison of the effect of board size but does not have a corresponding full multi-seed repetition; this limitation is taken into account when interpreting the results.

To reproduce the experiments, each run starts from an explicitly specified random seed and records its configuration, the identity of the data, of the checkpoint and of the code, as well as the versions of the core libraries. The evaluation rejects checkpoints that do not match the expected configuration. The metadata, checks and file organisation are presented in Appendix E.

Despite these checks, some GPU kernels are not fully deterministic. For this reason, independent training with a different seed is the basic unit for estimating training variability. Repeating the evaluation on the same checkpoint is not considered an independent experimental repetition.

3.8 Computational environment

All runs were carried out on Google Colab, with one GPU at a time. Training ran on an NVIDIA RTX PRO 6000 Blackwell with 96 GB VRAM and evaluation on an NVIDIA L4 with 24 GB VRAM. Because of the difference in hardware, training and evaluation times are examined separately, and comparisons are made only between runs carried out in the same computational environment. For each experimental condition, the duration of the main training and the times of the individual evaluation stages were recorded separately. The corresponding measurements and the comparison of computational cost are presented in Section 5.1.

Chapter 4

Experimental Exploration and Selection of the JEPA Configuration

The final configuration of the JEPA objective emerged from successive experiments that examined which information the context encoder should see, which information the target encoder should see, and from how many points of a game the training signal should be drawn. This chapter presents that path as a sequence of research problems and decisions. The full tables of the development runs are given in Appendix C and the relevant code in Code Listing G.5.

4.1 Starting point and criteria of the exploration

The main difficulty was that a low JEPA loss does not guarantee a useful internal representation. A predictor can approximate the target without the board state remaining easily decodable. In the extreme case of collapse, different inputs end up with almost identical embeddings. For this reason we monitored, beyond the loss, the variance and similarity of the representations of different examples, the legality of the top prediction through a frozen head, and the accuracy of probes at successive layers.

To keep the computational cost of the search manageable, the development comparisons of the different JEPA configurations were based on an evaluation of 5,000 games. After the final configuration was selected, the final Transformer-JEPA and Mamba-JEPA models were evaluated on 50,000 games. The initial results are therefore used to document the path of the design choices, while the definitive comparative conclusions rest on the more complete final evaluation.

4.2 From nested to hard-disjoint views

The first series of experiments is organised around one critical question: which information should the target encoder receive, so that the model learns to predict the future and does not merely reproduce the shared history? We first examine nested views, in which the two encoders see the same prefix while the target encoder additionally has access to future moves. We then explore different time horizons, predictors and loss functions, before the shared prefix is removed

completely through hard-disjoint views.

4.2.1 The initial hypothesis of a shared history

The first version (v1) was based on the hypothesis that the target encoder should know the same history as the context encoder plus the next move. In the nested view, therefore, the context branch processed the prefix up to time position t , while the target branch processed the same prefix extended to position $t + 1$. v1 used a shared encoder, a linear predictor and Variance-Invariance-Covariance Regularization (VICReg), which adds two terms to the loss to control the variance and covariance of the representations, so as to avoid the trivial solution and their collapse (Bardes, Ponce and LeCun, 2022).

The run did not show complete collapse, but the information became less accessible to this probe as depth increased. The relative board state was decoded with 62.4% accuracy from the initial representation and only 49.3% from the last layer, while top-1 legality reached 25.7%. The drop does not prove that the information was erased from the encoder; it does show, however, that the successive transformations did not organise it in a form the probe could exploit better. Avoiding collapse was therefore not sufficient as a criterion of success.

4.2.2 Increasing the time horizon

v2 introduced a separate target encoder, which received no gradients and was updated with EMA, a Transformer predictor and a horizon of four future positions (Grill et al., 2020; Assran et al., 2023). Top-1 accuracy rose to 38.6% and decoding of the relative state stayed at roughly 61–62% across layers. Training became more stable, but no representation comparable to that of the AR models emerged.

In v3 the capacity of the predictor was reduced and the horizon increased to eight positions, to test whether a powerful predictor was absorbing the work that the encoder should be doing. We also lowered the momentum to 0.99. Top-1 accuracy fell to 27.2% and the best relative-probe accuracy remained at the initial layer. This version showed embedding collapse and was therefore abandoned without further testing. v4 tried an MLP predictor that produced four predicted representations at once, one for each of the next four positions of the sequence (Baevski et al., 2022; Assran et al., 2023). The target encoder still saw the entire preceding history. The result improved marginally, with 33.6% top-1 and 62.7% best relative-state accuracy, without solving the underlying problem.

4.2.3 Contrastive learning and isolating the future

The next hypothesis was that the objective needed a more discriminative structure. Reading DINOv3 (Siméoni et al., 2025) was the starting point for exploring a contrastive JEPA setup in v5. We applied the InfoNCE loss, a contrastive objective introduced in the context of Contrastive Predictive Coding. This loss increases the similarity between the predicted representation and the correct target representation, while reducing its relative similarity to the representations of other examples (Oord, Li and Vinyals, 2019; LeCun, 2022). The model is thus asked to distinguish the correct future state from alternative cases. Initially v5 also kept the history on the target side.

Top-1 accuracy rose to 39.6%, with peak board-state accuracy at layer 5 of 63.01%, a value that remained very low.

For v6, games were organised into groups based on their shared move sequence, and each training batch contained more than one group. Within the same group, games shared the history but not necessarily the future moves. The target encoder received only token $t + 1$, while the predictor produced ten candidate representations, so that it could cover more than one possible continuation (Rupprecht et al., 2017). A distance-margin loss (Hadsell, Chopra and LeCun, 2006) pushed the most suitable of these towards the correct target representation and kept incompatible representations at a distance. With this configuration, top-1 legal-move accuracy rose to 55.9%, while the best relative board-state accuracy came to 60.7%. These values refer to the early linear evaluation of Table C.2. In the later evaluation included in Figure 4.2, the best available MLP results were 62.2% top-1 and 63.1% relative state. The functional improvement was not yet accompanied by strong decoding of the board.

4.2.4 Systematic exploration of hard-disjoint views

The noticeably higher performance of v6 was a turning point and led to re-running the earlier variants with hard-disjoint views. The results showed that removing the prefix from the input of the target encoder had a substantial effect. In v1 VICReg, for example, top-1 legality rose to 73.3% with a linear head and 79.1% with an MLP. With the views separated, the target encoder no longer has access to the prefix and forms the target representation only from the future move (Code Listing G.5.2). The predictor must therefore predict this representation using only the information that the context encoder has extracted from the history of the game. Figure 4.1 summarises the successive questions that led from a shared history to separated views and denser supervision.

Before supervision was applied at all positions, the most promising variants were transferred to Mamba, to check whether their behaviour depended on self-attention. v1 with hard-disjoint VICReg and single-position supervision reached 94.3% top-1 with a linear head and 84.4% overall relative linear-probe accuracy at L7. This improvement strengthened interest in a second change: that each game should provide a training signal not from just one random point but from all valid positions.

4.3 Dense supervision as a second turning point

In the initial versions, one random cut-off point was selected from each game, so that only one position of the sequence provided a direct training signal in each pass. In the all-position variant, a single causal pass of the context encoder produces representations for the entire sequence, and a context-target pair is formed for every valid transition. The InfoNCE loss is computed separately at each position and the individual values are combined into a weighted mean, according to the number of valid pairs. In this way, all available positions of the game contribute directly to updating the model.

Denser supervision was not a solution on its own. Nested v1 with all-position training remained

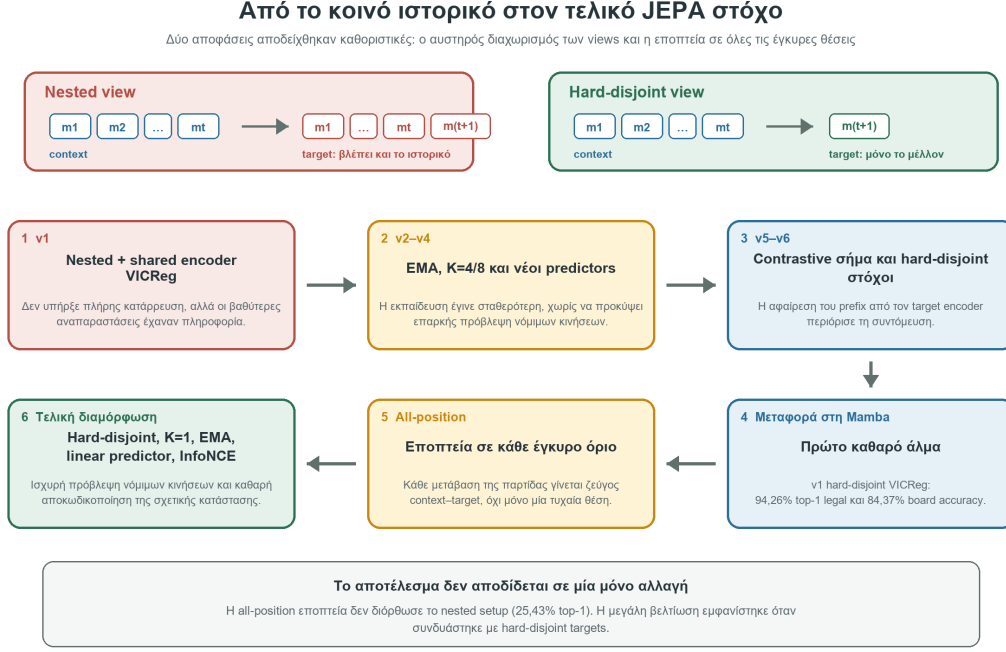


Figure 4.1: Evolution of the JEPA experimental exploration. The improvement came from combining a hard-disjoint target with supervision at all valid positions, not from a single isolated change.

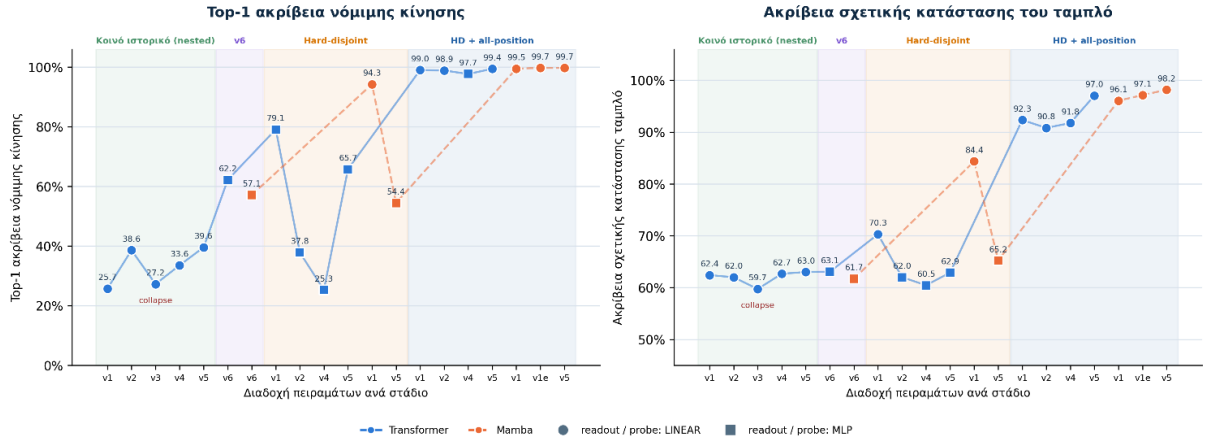
at 25.4% top-1 and 65.1% overall relative linear-probe accuracy. When the same signal was combined with hard-disjoint VICReg, the values rose to 99.0% and 92.3%, and with EMA they reached 99.4% and 94.5%. The remaining re-runs and the individual metrics are given in Table C.3. The path shows that separating the target from the history and using all valid positions act as complements. Figure 4.2 presents the evolution of the best top-1 legality and the best overall relative-state accuracy at the selected stages of the exploration.

4.4 Selection and finalisation of the JEPA configuration

Applying all-position supervision to Mamba brought hard-disjoint VICReg to 99.5% top-1 and 96.1% overall relative-state accuracy. With EMA added, the values reached 99.7% and 97.1%. The final hard-disjoint InfoNCE version retained 99.7% top-1 and achieved the highest overall relative linear-probe accuracy, 98.2%, with a macro-balanced accuracy of 97.7%. On the Transformer, the same configuration reached 99.4%, 97.0% and 96.2% respectively. The choice of InfoNCE was therefore not based on the almost saturated top-1 metric, but on the combination of high functional performance, clean linear decoding and a simple linear predictor.

The development exploration was designed to identify a promising and stable configuration, not to fully isolate the effect of each individual choice. Some early runs differed simultaneously in the predictor, the length of training or the evaluation protocol. For this reason, the differences between them are interpreted as indications that guided the search and not as controlled evidence for the effect of any single factor.

Εξέλιξη της αναπτυξιακής πορείας JEPA στο Othello 8×8



Σημείωση: Οι τιμές είναι ποσοστά. Η v3 εμφανίζεται μόνο στο αρχικό στάδιο· μετά την κατάρρευση των αναπαραστάσεων δεν περιλαμβάνεται στα επόμενα στάδια του διαγράμματος. Ο Mamba δεν εκτελέστηκε στα στάδια με κοινό ιστορικό. Η χρονογραμμή είναι περιγραφική: τα στάδια δεν έχουν όλα ίδιο budget και πρωτόκολλο αξιολόγησης, και οι τιμές των πρώιμων σταδίων προέρχονται από μικρότερες αξιολογήσεις.

Figure 4.2: Evolution of the best top-1 legality and the best overall relative-state accuracy during JEPA development. T and M denote Transformer and Mamba. v3 appears only in the initial stage of the diagram; its archived all-position re-run is listed separately in Table C.3. The values are descriptive, since the stages differ in training length, evaluation and available head.

Chapter 5

Results

This chapter presents the results of the final experimental setup. The development exploration of the JEPA objective was completed in Chapter 4; here we examine only the final models trained with the shared protocol of Chapter 3. We first compare the architectures and the training objectives while holding the board size fixed, separately for functional performance, decodability and the causal use of the representations. In the last section we examine how the same measures change when the board grows from 8×8 to 12×12 and 16×16 .

The main tables are based on the shared seed 42 and on the final test set. Training variability is examined separately through five independent seeds on 8×8 and 16×16 . All accuracy and probability-mass values are reported as percentages, while differences between conditions or interventions are expressed in percentage points.

5.1 Training completion and computational cost

All twelve main training runs completed the planned pass over 200 shards, i.e. roughly 20 million games per experimental condition, and produced valid bf16 checkpoints. No non-finite values or incompatibilities were detected in the runs used for the final evaluation.

The total time of the twelve main training runs came to roughly 54 GPU-hours. The duration per run rose from 0.68 hours for Transformer-AR on 8×8 to 11.40 hours for Mamba-JEPA on 16×16 . For every architecture and board size, the JEPA objective had a higher computational cost than the corresponding AR, since it includes a target encoder, a predictor and the application of InfoNCE at all valid positions. Training times per experimental condition are shown in Figure 5.1.

Evaluation was run on a lower-powered GPU and, therefore, its durations are not directly comparable with the training times. The time of the full evaluation ranged from 0.42 to 12.13 hours and increased noticeably with board size. For example, the evaluation of Transformer-AR on 16×16 took roughly five hours, most of which was spent computing the legality of the predictions on 50,000 games. The corresponding times for all experimental conditions are shown in Figure 5.2.

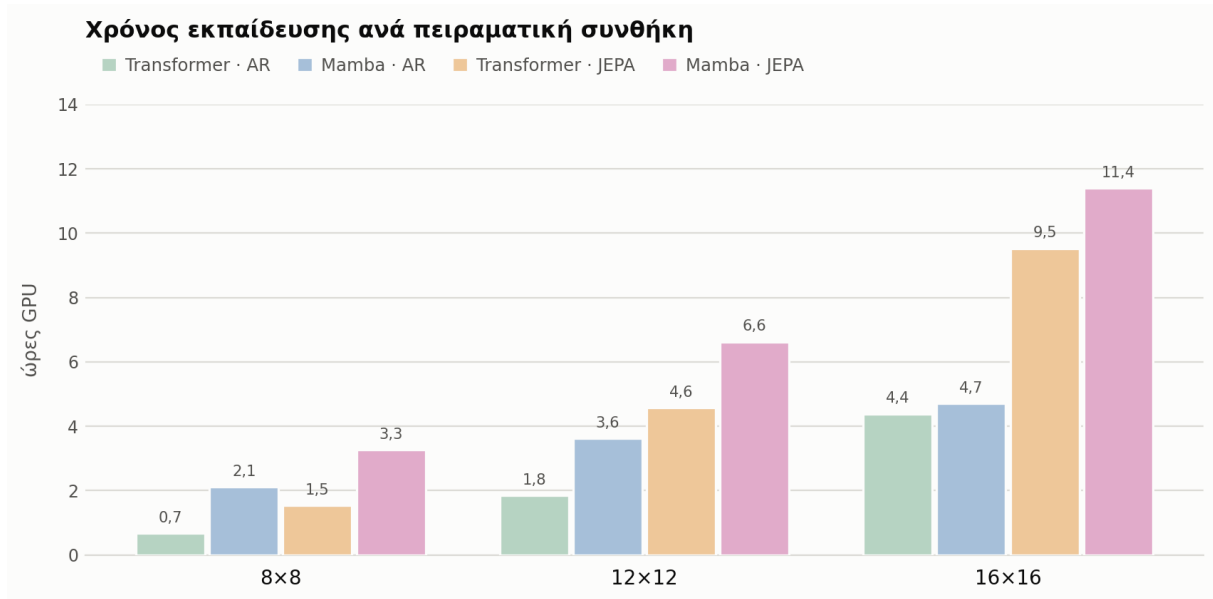


Figure 5.1: Training time per experimental condition. Shared budget of 20,000,000 games and 200 shards in every condition; the time is the sum of the per-shard measurements in `metrics.csv`.

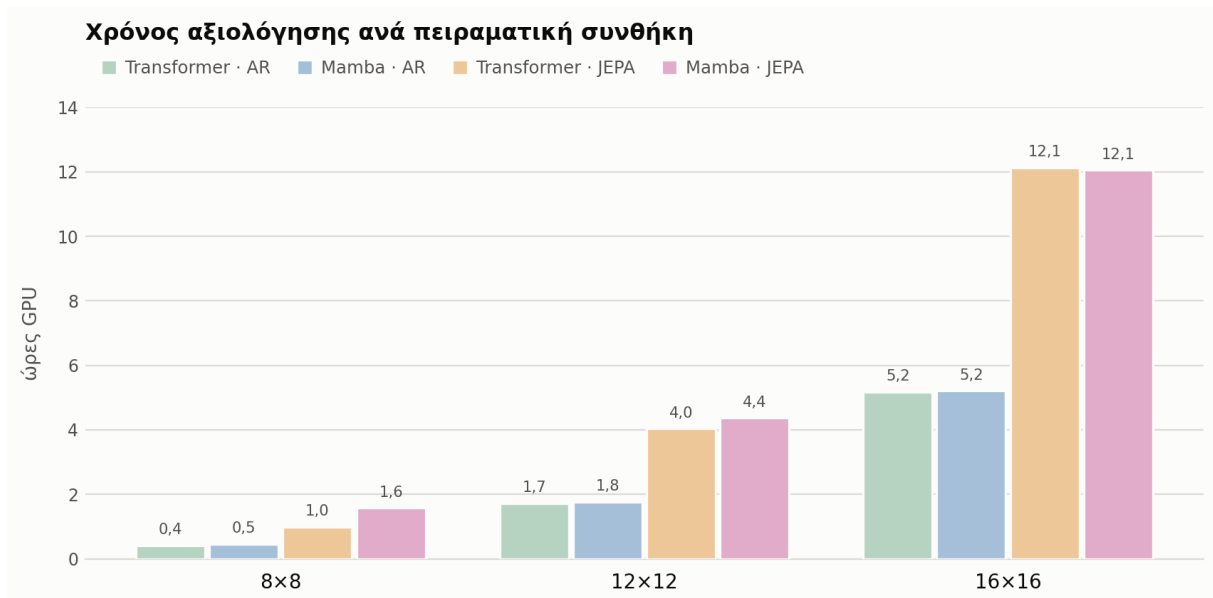


Figure 5.2: Evaluation time per experimental condition. Includes legal-move prediction, training of the frozen heads where required, feature extraction and the board-state probes.

5.2 Next-move prediction and legal behaviour

The next-move evaluation uses the native head in the AR models. In the JEPA models, the linear or MLP frozen head was selected exclusively on the validation split, with the probability mass on legal moves as the criterion. The procedure selected the linear head for all Transformer-JEPA conditions and for Mamba-JEPA on 12×12 and 16×16 , while on Mamba-JEPA 8×8 the MLP prevailed.

Table 5.1: Final legal-move prediction performance in the twelve main experimental conditions. Values are given as estimate [lower–upper bound of the 95% bootstrap confidence interval], from 1,000 resamples of the 50,000 test-set games. For JEPA, the frozen head selected on the validation split is reported.

Board	System	Head	Top-1 legal (%) [95% CI]	Legal mass (%) [95% CI]
8×8	Transformer-AR	Native	99.726 [99.717–99.734]	98.477 [98.465–98.490]
8×8	Transformer-JEPA	Linear	99.416 [99.404–99.428]	97.763 [97.752–97.774]
8×8	Mamba-AR	Native	99.847 [99.841–99.853]	99.345 [99.335–99.354]
8×8	Mamba-JEPA	MLP	99.719 [99.710–99.728]	98.904 [98.894–98.914]
12×12	Transformer-AR	Native	99.416 [99.408–99.423]	93.135 [93.124–93.147]
12×12	Transformer-JEPA	Linear	98.524 [98.511–98.537]	93.302 [93.292–93.313]
12×12	Mamba-AR	Native	99.512 [99.505–99.519]	95.412 [95.403–95.422]
12×12	Mamba-JEPA	Linear	98.993 [98.981–99.004]	95.472 [95.463–95.481]
16×16	Transformer-AR	Native	98.991 [98.983–98.999]	88.787 [88.776–88.798]
16×16	Transformer-JEPA	Linear	97.539 [97.526–97.553]	87.934 [87.923–87.946]
16×16	Mamba-AR	Native	99.049 [99.041–99.058]	90.739 [90.729–90.750]
16×16	Mamba-JEPA	Linear	98.413 [98.403–98.424]	90.425 [90.415–90.434]

The results of Table 5.1 reveal two consistent trends: Mamba systematically retains higher top-1 legality than the Transformer, while the AR objective consistently outperforms JEPA. The size of the gaps, however, is strongly determined by the interaction of the two factors with board scale.

In top-1 legality, the two AR models show marginal differences at all sizes (the architectural gap never exceeds 0.12 percentage points), whereas under the JEPA objective the difference between Mamba and the Transformer widens noticeably with complexity: it starts at 0.30 points on 8×8 and reaches 0.87 on 16×16 . Likewise, the lead of AR over JEPA widens with scale in both architectures (almost fivefold from 8×8 to 16×16), but the absolute size of the gap remains clearly larger in the Transformer, where the difference reaches 1.45 points, against 0.64 points in Mamba.

Unlike top-1 legality, which stays compressed near the ceiling, the analysis of the legal probability mass separates the systems with greater discriminative power. Examining the entire output distribution, and not only the dominant choice, shows that the architecture exerts a stronger and more consistent effect than the training objective: Mamba retains a steady lead of up to roughly 2 percentage points over the Transformer at all scales and for both objectives. By contrast, the difference between AR and JEPA remains limited (below 0.9 points at each size), with JEPA

marginally ahead only on the intermediate 12×12 board. The most important finding is that, while top-1 legality remains almost saturated (above 97.5%), the total legal mass falls noticeably on the larger boards (up to about 12% short of the ideal on 16×16), showing that a valid first choice often coexists with notable probability leakage onto illegal moves.

The 95% bootstrap intervals that accompany the values are very narrow: the largest total width was 0.027 percentage points for top-1 legality and 0.025 for legal mass. The uncertainty due to the 50,000 test-set games is therefore small; the variability that arises from independent training runs is examined in Section 5.4. The distance from random selection is additionally checked with the normalised lift in Table F.7, which takes into account that the fraction of legal tokens in the vocabulary differs by board size.

The next two analyses look more closely at the behaviour of the models by examining legality across the k most likely predictions, as well as its variation across the different phases of the game. $P@k$ is defined as the mean fraction of legal moves among the first k predictions, for k equal to 1, 2, 3 and 5.

Table 5.2 and Figure 5.3 show how the proportion of legal choices declines as the window of k predictions widens, revealing consistent behavioural patterns. At every board size, Mamba-AR and Transformer-JEPA consistently mark the highest and the lowest performance respectively, while Mamba-JEPA and Transformer-AR achieve comparable results. The architectural and training gap grows noticeably on the larger boards. The fact that AR systematically outperforms JEPA for every k confirms that its advantage is not an incidental advantage of the first choice but a structural property of its distribution. Because the number of legal moves differs by scale, the metric is compared only between systems of the same size.

Table 5.2: Mean proportion of legal moves among the first k predictions, $P@k$, for k equal to 1, 2, 3 and 5. Each evaluated position has the same weight in the mean. The metric does not require all k predictions of a position to be legal. In Table 5.1 the mean is computed first within each game, which is why $P@1$ may differ slightly from top-1 legality.

Board	System	Head	P@1 (%)	P@2 (%)	P@3 (%)	P@5 (%)
8×8	Transformer-AR	Native	99.72	98.28	96.78	92.45
8×8	Transformer-JEPA	Linear	99.42	98.01	96.55	92.26
8×8	Mamba-AR	Native	99.84	98.39	96.90	92.57
8×8	Mamba-JEPA	MLP	99.72	98.29	96.80	92.49
12×12	Transformer-AR	Native	99.42	98.76	98.12	96.34
12×12	Transformer-JEPA	Linear	98.52	97.89	97.27	95.54
12×12	Mamba-AR	Native	99.51	98.89	98.27	96.53
12×12	Mamba-JEPA	Linear	98.99	98.39	97.81	96.13
16×16	Transformer-AR	Native	98.99	98.55	98.12	97.01
16×16	Transformer-JEPA	Linear	97.54	96.99	96.49	95.30
16×16	Mamba-AR	Native	99.05	98.64	98.24	97.17
16×16	Mamba-JEPA	Linear	98.41	97.98	97.58	96.55

As shown in Table 5.3 and Figure 5.4, dividing the game into four normalised phases reveals that the differences between architectures and objectives appear mainly after the opening. While in

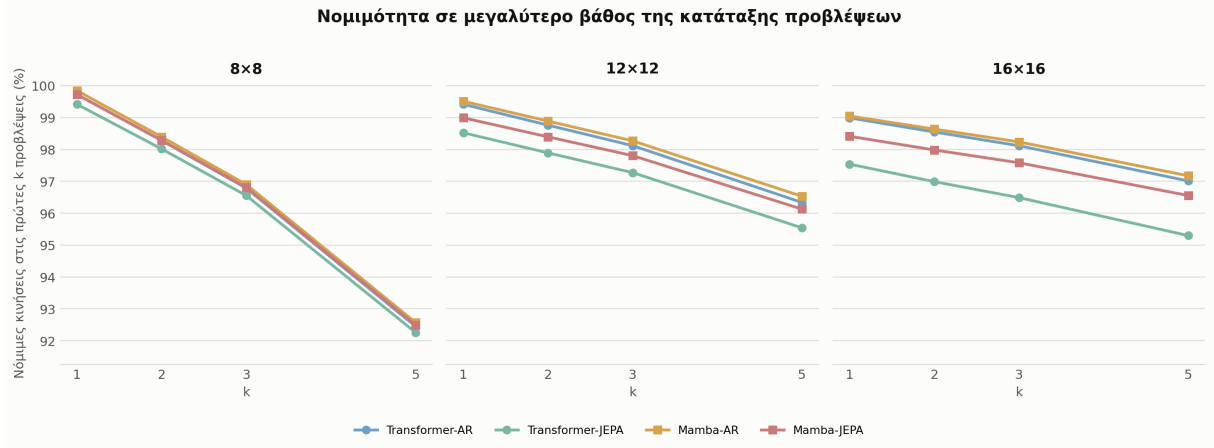


Figure 5.3: Percentage of legal moves among the first k predictions for the three board sizes. The same validation-selected heads as in Table 5.1 are used.

the first quarter all models are close to perfect, their paths then diverge sharply: the AR models maintain consistently high legality to the end, while the JEPA models show a clear decline as the complexity of the position increases. This temporal instability is strongest in the Transformer under JEPA training, which falls noticeably both in the endgame of the smaller boards and in the middle phases of 16×16 , remaining up to two points behind Mamba. This finding shows that the advantage of the AR objective concerns not only the overall success rate but also the robustness of its behaviour across the entire temporal depth of the game.

Table 5.3: Top-1 legality by normalised game phase. Each column corresponds to one quarter of the duration of the game.

Board	System	Head	0–25%	25–50%	50–75%	75–100%
8×8	Transformer-AR	Native	99.99	99.86	99.66	99.40
8×8	Transformer-JEPA	Linear	99.99	99.71	99.20	98.78
8×8	Mamba-AR	Native	100.00	99.96	99.81	99.63
8×8	Mamba-JEPA	MLP	100.00	99.93	99.66	99.30
12×12	Transformer-AR	Native	99.71	99.32	99.31	99.32
12×12	Transformer-JEPA	Linear	99.46	98.46	98.21	97.97
12×12	Mamba-AR	Native	99.89	99.45	99.39	99.32
12×12	Mamba-JEPA	Linear	99.79	98.91	98.73	98.54
16×16	Transformer-AR	Native	99.28	98.77	98.90	99.00
16×16	Transformer-JEPA	Linear	98.33	96.99	97.33	97.49
16×16	Mamba-AR	Native	99.53	98.95	98.87	98.84
16×16	Mamba-JEPA	Linear	99.18	98.26	98.15	98.05

5.3 Decoding the board state

The main comparison uses a linear probe of the relative encoding, with LayerNorm on the input, as defined in Section 3.6.2. Macro-balanced accuracy gives equal weight to the three classes, while overall accuracy shows the fraction of all squares predicted correctly. The values in Table 5.4 therefore concern the normalised representations and not the separate raw linear probe without

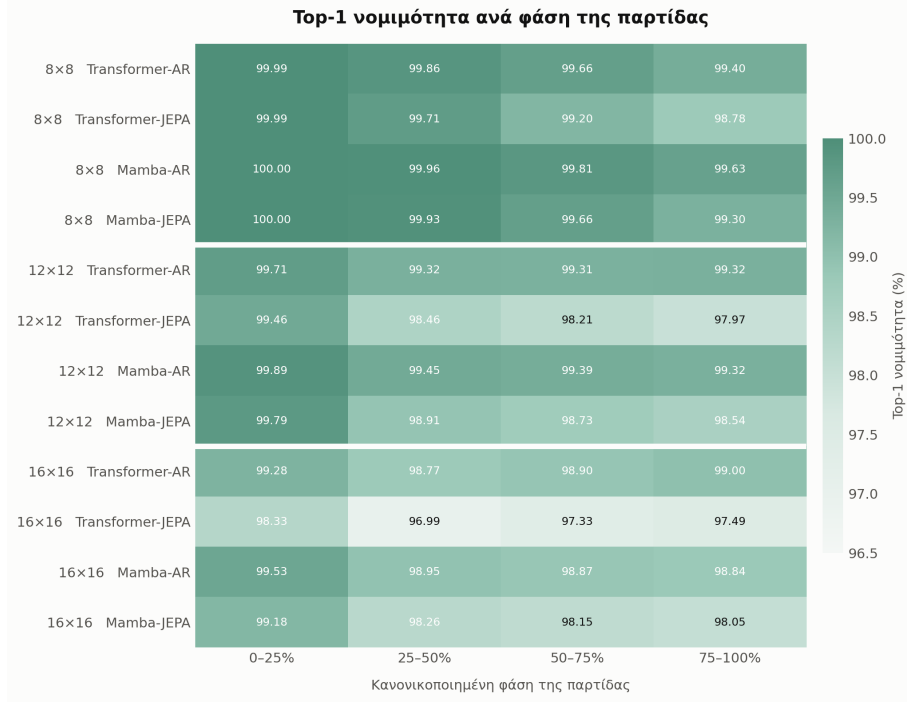


Figure 5.4: Top-1 legality by normalised game phase in the twelve main experimental conditions. The white horizontal lines separate the three board sizes.

normalisation that is used for the causal-intervention directions.

The comparisons within each scale reveal a strong interaction between architecture and training objective, as the effect of JEPA proves to be diametrically opposite in the two models. While in the Transformer the move from AR to JEPA consistently degrades linear decodability, in Mamba it is beneficial, further increasing macro-balanced accuracy at all three board sizes.

This contrast grows dramatically with the complexity of the environment. On 8×8 , the deviations caused by JEPA are still marginal, ± 0.19 percentage points in either direction. On 16×16 , however, the gap explodes: the performance of Transformer-JEPA collapses by 4.51 points relative to the corresponding AR, while Mamba-JEPA keeps its advantage, exceeding Mamba-AR by 0.42 points. As a result, whereas under AR training Mamba holds a modest lead over the Transformer (below 2.2 points at all scales), under the JEPA objective the architectural gap widens dramatically, jumping from 1.32 points on 8×8 to 5.40 points on 16×16 . Finally, in all conditions overall accuracy is systematically higher than macro-balanced accuracy, confirming that the high frequency of empty squares overstates the true ability to recover the occupied positions.

The comparison of the four probe variants in Tables F.3 and F.4 supports the use of the relative encoding: it achieved higher accuracy than the absolute encoding with both probe families, in all conditions. In addition, the linear probe of the relative encoding outperformed the corresponding MLP in all twelve conditions of the applied protocol. The main comparison is therefore based on the same, fixed probe variant for all systems. Per-class performance in Table F.5 shows why macro-balanced accuracy is also needed: empty squares are decoded more easily than the two occupied classes.

Table 5.4: Decoding of the relative board state with an input-normalised linear probe. The layer is selected exclusively on the validation split.

Board	System	Macro-balanced (%)	Overall (%)	Layer
8×8	Transformer-AR	96.91	97.59	6
8×8	Transformer-JEPA	96.72	97.45	6
8×8	Mamba-AR	97.85	98.32	12
8×8	Mamba-JEPA	98.04	98.48	13
12×12	Transformer-AR	89.17	91.74	7
12×12	Transformer-JEPA	88.48	91.21	6
12×12	Mamba-AR	91.37	93.41	12
12×12	Mamba-JEPA	92.85	94.54	12
16×16	Transformer-AR	82.88	87.02	8
16×16	Transformer-JEPA	78.37	83.49	7
16×16	Mamba-AR	83.35	87.37	11
16×16	Mamba-JEPA	83.77	87.56	12

In Figure 5.5, decodability changes with depth and is not always maximised at the final layer. Numerical comparisons of the selected and the final layer, together with normalised depth, are given in Table F.6. Normalisation is useful because the Transformer has eight blocks and Mamba fifteen: the same layer number does not correspond to the same position within the two architectures.

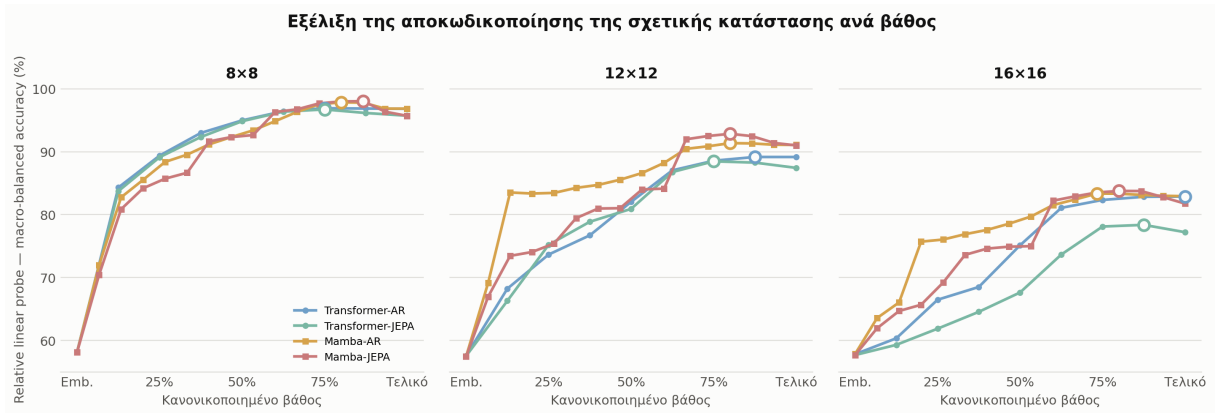


Figure 5.5: Macro-balanced accuracy of the input-normalised linear probe of the relative state at successive layers. The larger markers indicate the layer selected on the validation split for each condition.

5.4 Variability across independent training runs

The five independent training runs on 8×8 and 16×16 examine whether the comparisons within each size depend on seed 42. On 8×8, Mamba-AR retained the highest mean top-1 legality and Mamba-JEPA the highest mean relative-state accuracy. The same ranking was repeated on 16×16. Table 5.5 presents the mean and the standard deviation, while Figure 5.6 keeps all five individual runs visible.

Table 5.6 summarises the matched effects of the factors at the two ends of the scale, confirming

Table 5.5: Mean $\pm$ standard deviation over five independent seeds. The relative state is assessed with the macro-balanced accuracy of the input-normalised linear probe.

Board	System	Top-1 legal (%)	Relative state (%)
8 $\times$ 8	Transformer-AR	99.71 $\pm$ 0.01	96.95 $\pm$ 0.07
8 $\times$ 8	Transformer-JEPA	99.42 $\pm$ 0.01	96.62 $\pm$ 0.24
8 $\times$ 8	Mamba-AR	99.84 $\pm$ 0.01	97.92 $\pm$ 0.14
8 $\times$ 8	Mamba-JEPA	99.72 $\pm$ 0.00	98.17 $\pm$ 0.08
16 $\times$ 16	Transformer-AR	98.99 $\pm$ 0.02	82.96 $\pm$ 0.08
16 $\times$ 16	Transformer-JEPA	97.53 $\pm$ 0.06	78.34 $\pm$ 0.33
16 $\times$ 16	Mamba-AR	99.03 $\pm$ 0.05	83.43 $\pm$ 0.43
16 $\times$ 16	Mamba-JEPA	98.46 $\pm$ 0.08	83.89 $\pm$ 0.13

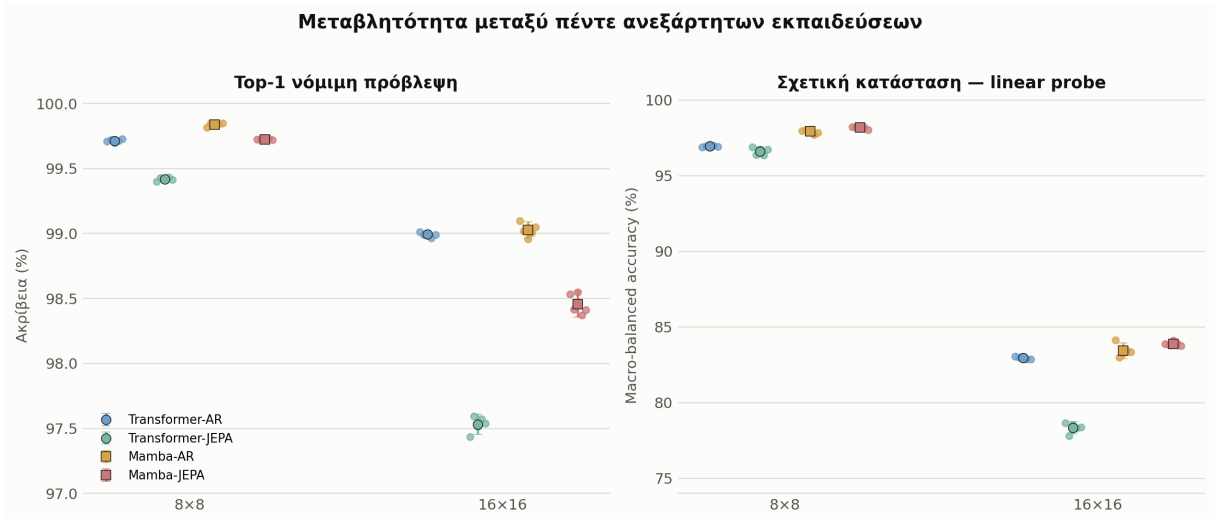


Figure 5.6: Variability of top-1 legality and of linear decoding of the relative state over five independent training runs. The large markers show the mean and the bars the 95% interval based on the t distribution.

that the complexity of the environment sharpens the differences between the systems.

Table 5.6: Matched effects in percentage points. A positive main effect of architecture favours Mamba, a positive objective effect favours JEPA, and a positive interaction indicates that the change AR→JEPA is more favourable in Mamba than in the Transformer. Confidence intervals are based on the t distribution; the corresponding p -values before and after the Holm correction are given in Table F.1.

Metric	Effect	8×8: difference [95% t -CI]	16×16: difference [95% t -CI]
Top-1	Architecture	+0.21 [+0.21; +0.22]	+0.48 [+0.39; +0.57]
Top-1	Objective	−0.20 [−0.22; −0.19]	−1.02 [−1.07; −0.96]
Top-1	Interaction	+0.18 [+0.15; +0.21]	+0.89 [+0.71; +1.07]
Relative state	Architecture	+1.26 [+1.10; +1.42]	+3.01 [+2.89; +3.13]
Relative state	Objective	−0.04 [−0.18; +0.10]	−2.08 [−2.32; −1.85]
Relative state	Interaction	+0.59 [+0.14; +1.04]	+5.09 [+4.23; +5.95]

The main effect of architecture is consistently in favour of Mamba and more than doubles as the board grows: its lead in top-1 legality rises from +0.21 to +0.48 points, while in linear decoding of the relative state it jumps from +1.26 to +3.01 percentage points on 16×16.

By contrast, the mean effect of the objective cannot be interpreted on its own. While on 8×8 it is essentially negligible for the representation, on 16×16 it appears strongly negative and reaches −2.08 points. This apparent shortfall of JEPA is a consequence of the strong interaction (+5.09 points on 16×16): it does not reflect a universal decline, but the fact that the heavy drop of Transformer-JEPA more than offsets the positive performance of Mamba-JEPA. This statistical behaviour reveals that the effect of the training objective is asymmetric: it imposes a disproportionately heavy cost on the Transformer, while Mamba remains robust, recording a small improvement in linear decoding of the state. How these effects change with size is compared in Section 5.6.

The paired t -tests after the Holm correction agree with the main pattern of Table 5.6. The only non-significant effect was the mean of the AR→JEPA change on the relative state at 8×8, while the interaction at the same scale remained marginal. Table F.1 gives the p -values before and after the correction. Table F.2 completes the picture with the exact sign-flip tests and Hedges’ g_z . With five seeds, the exact tests cannot give a two-sided p smaller than 0.0625, while the t -based estimates assume an approximately normal distribution of the matched differences. For the practical significance of the results, the size of the difference in percentage points and its consistency across independent training runs therefore take precedence.

5.5 Results of the causal interventions

The final causal evaluation covers all twelve conditions, the same 1,000 counterfactual cases per size and the three methods of Chapter 3. In AR the native head is used. In JEPA, the head previously selected on the validation split is used: MLP for Mamba-JEPA 8×8 and linear for the other JEPA conditions.

Table 5.7: Change in probability mass on the counterfactual set of legal moves, in percentage points. The three Δ columns compare the intervention with the unmodified run; the last column compares the adapted intervention with the geometrically matched random control.

Board	System	Li Δ	Nanda Δ	Adapted Δ	Adapted–random
8×8	Transformer-AR	2.29	0.39	6.37	6.51
8×8	Transformer-JEPA	2.30	0.57	5.97	7.28
8×8	Mamba-AR	0.01	0.01	6.35	7.12
8×8	Mamba-JEPA	0.10	0.01	6.05	6.72
12×12	Transformer-AR	−0.49	0.39	2.72	3.29
12×12	Transformer-JEPA	−1.69	0.70	2.19	2.77
12×12	Mamba-AR	0.23	0.01	2.71	3.47
12×12	Mamba-JEPA	0.05	0.03	2.55	3.18
16×16	Transformer-AR	0.11	0.07	1.06	1.22
16×16	Transformer-JEPA	0.49	0.11	0.78	1.41
16×16	Mamba-AR	0.57	0.01	1.48	1.73
16×16	Mamba-JEPA	−0.08	0.01	0.81	1.95

The adapted intervention emerged as the only method with a consistent and universal effect across all architectures and training objectives, achieving the largest increase in legal mass at every board size. The size of this effect nevertheless declined with scale, starting at 5.97 to 6.37 percentage points on 8×8, narrowing to between 2.19 and 2.72 points on 12×12, and to between 0.78 and 1.48 points on 16×16. The other two approaches clearly lagged behind. The Nanda method was limited to uniformly weak positive changes, with a maximum of 0.7 points, while the Li method was markedly unstable: it was effective essentially only on the two 8×8 Transformers, with an increase of about 2.3 points, remained almost inert on Mamba and even produced negative changes on the larger boards. Figure 5.7 shows this distinct behaviour of the three approaches clearly.

These findings are reinforced by the supplementary metrics of Tables D.5 and D.6. The adapted method systematically reduced the sum of false positives and false negatives by 0.45 to 2.17 per case relative to the unmodified run, while also improving top-1 legality in eleven of the twelve conditions, the only exception being a marginal shortfall of 0.10 points on Transformer-JEPA 16×16. Relative to the geometrically matched random control, the shift of the top prediction remained positive in all twelve cases.

The fact that the legal mass shifts favourably even when the first choice stays unchanged confirms that the overall probability distribution and the top prediction do not always respond in the same way. At the same time, the consistent advantage of the adapted intervention over the geometrically matched random control at every size is the most substantive causal finding: it shows that the directed modification of the representation actively affects the model’s decision process, going beyond mere statistical decodability through a probe. Even so, the observed ranking of the approaches must be interpreted with care, since only the adapted method used strength selection on the validation split, and the three techniques differ both in the number of intervention sites and in the tuning resources they require.

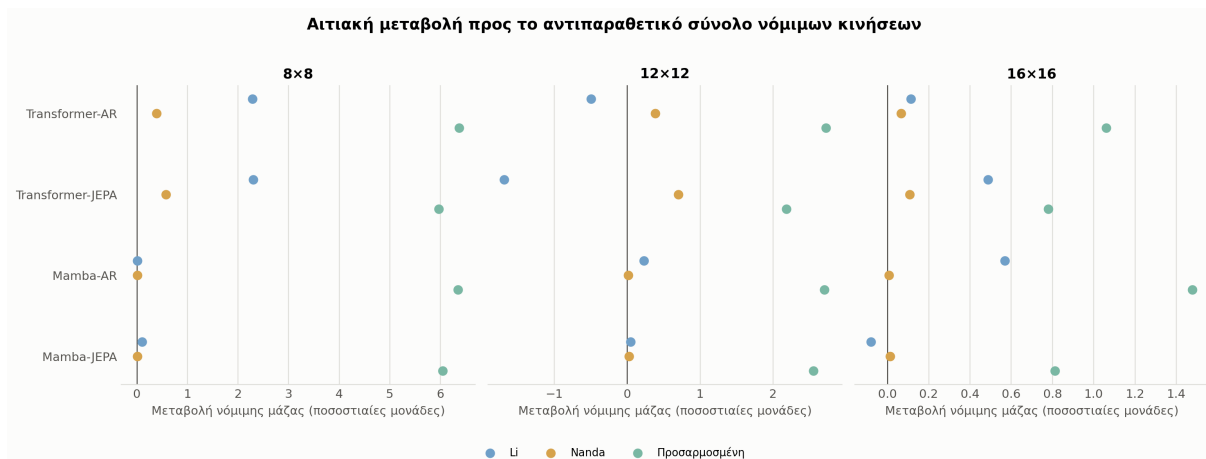


Figure 5.7: Change in probability mass on the counterfactual set of legal moves for the Li, Nanda and adapted interventions. Positive values indicate a shift of the prediction towards the new board state.

5.6 Effect of board size on functional, representational and causal performance

Increasing the board scale made both the functional behaviour and the internal organisation of the models more difficult. This degradation, however, is not reflected in the surface-level top-1 legality, which remains almost saturated above 97.5% in all conditions, but is clearly revealed in Figure 5.8. There, both the legal probability mass and the macro-balanced accuracy on the relative state show a parallel and substantial decline, demonstrating that the top choice masks a deeper degradation of the output distribution and of the board representation. At the same time, scale acts as a powerful magnifying glass for the differences between systems: the gap between the best and the worst performance more than triples in top-1 legality, starting at 0.43 points and reaching 1.51, and quadruples in board decoding, widening from 1.32 to 5.39 points. The top prediction can therefore remain valid while the overall distribution and the quality of the representation have already deteriorated significantly.

The effect of the objective did not act in the same way in the two architectures as the board size increased. In the Transformer, it magnified the shortfall of JEPA relative to AR both in behaviour and in internal organisation, with the top-1 legality deficit quadrupling and board recoverability dropping sharply. Mamba, by contrast, showed notable robustness: its functional decline under JEPA remained tightly limited, while the linear structure of its internal states remained consistently superior to that of AR at all scales. Increasing complexity therefore amplified the cost of JEPA almost exclusively in the Transformer, which explains the statistically strong interaction of the two factors recorded in Table 5.6.

A similar picture of weakening with increasing size was recorded in the causal interventions. The increase in legal mass through the adapted method narrowed gradually from roughly 6 points on 8x8 to roughly 2.5 on 12x12 and around 1 point on 16x16. Despite this reduction, the effect remained positive in all systems, consistently exceeding the geometrically matched random control even on the most complex board.

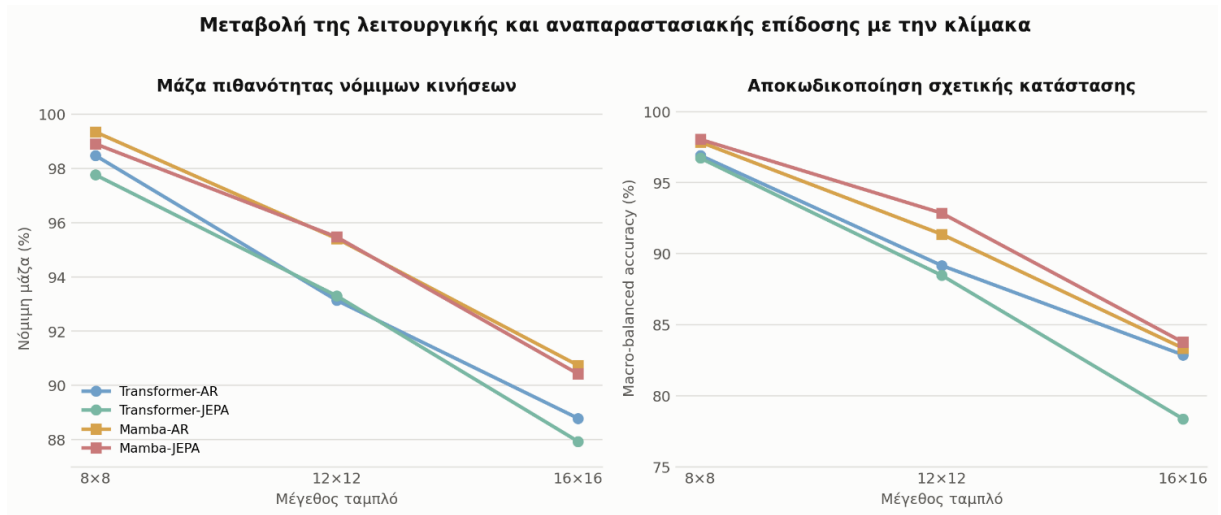


Figure 5.8: Change in the probability mass on legal moves and in the macro-balanced accuracy of the input-normalised relative-state linear probe across the three board sizes.

Chapter 6

Discussion

The purpose of this chapter is to move from presenting the results to interpreting them, guided by the five research questions posed at the start of the thesis. We first examine how the architecture and the training objective affect the prediction of legal moves, and then to what extent the models represent the board state internally. Next we investigate whether this represented information is actually used during prediction or remains detectable but inert. The three levels of evaluation (behaviour, probes and interventions) are then compared, to determine where the findings agree and where they diverge. Finally, we examine how all of the above are affected as the board size increases. The chapter closes with the three main findings and the contribution of the thesis, before the limitations of the study and the proposals for future research are presented.

6.1 How do the architecture and the objective affect legality?

All twelve models achieve top-1 legality above 97%, showing that they successfully learn to produce permissible moves. The architecture and the objective do, however, differentiate the performance of the systems substantially. Overall, Mamba proves consistently superior to the Transformer, while within the same architecture the Autoregressive (AR) objective gives systematically higher top-1 legality than JEPA.

The impact of the objective is not the same in the two architectures. As long as AR is used, the differences between Mamba and the Transformer in the top choice remain negligible at all scales. When we move to JEPA, however, the gap in favour of Mamba opens a little further, starting at 0.30 percentage points on 8×8 and reaching 0.87 on 16×16 . On the largest board, switching from AR to JEPA lowers the performance of the Transformer by 1.45 points, while in Mamba the drop is limited to 0.64. The measurements over multiple independent training runs (seeds) record a measurable interaction in top-1 legality, of +0.18 points on 8×8 and +0.89 on 16×16 . This figure shows that the JEPA objective places a disproportionate burden on the Transformer, while Mamba remains clearly more robust.

This difference is not distributed evenly within the game. In the opening, legality is very high for all systems, while after the first quarter the JEPA variants lose more ground than the corresponding AR ones. This is particularly visible for Transformer-JEPA in the middle phases

of 16×16 , where Mamba-JEPA keeps higher legality. The overall mean therefore hides the fact that the difference between objectives also concerns the stability of the predictions as the game unfolds.

The total legal probability mass gives a fuller picture of the effect of the two factors. While top-1 legality declines only slightly as scale increases (by just 0.73 to 1.88 points from 8×8 to 16×16), legal mass drops substantially, by 8.48 to 9.83 points. In other words, the models continue as a rule to select a legal move as their first choice, but the rest of the probability distribution leaks towards illegal squares as the board grows. At the same time, in legal mass the advantage of AR is not absolute: on the intermediate 12×12 board, JEPA concentrates marginally more legal probability in both architectures (+0.17 points in the Transformer and +0.06 in Mamba).

Finally, this diversity of behaviour must be assessed in the light of how the two systems were trained. In the AR models, producing the move directly guides the whole network from the start. In JEPA, by contrast, the encoder was trained exclusively in representation space, and the head was trained afterwards on frozen weights of the main model. The fact that a head trained after the fact comes so close to the performance of AR (differences of just 0.1–0.3% on 8×8) suggests that the JEPA representations organise the rules of the game to a comparable degree, without having been optimised directly for this purpose.

6.2 Do the models learn representations of the board?

To examine whether the internal representations contain information about the board state, linear probes were trained on the frozen representations of each system. On 8×8 , the relative state of the squares was linearly decodable with macro-balanced accuracy from 96.72% to 98.04%. A simple linear classifier could thus recover with high accuracy whether each square was “mine”, “yours” or “empty”, even though no explicit board-state labels were used when training the encoders.

The choice of macro-balanced accuracy matters for interpreting the result. Overall accuracy was higher in all twelve systems, because it is affected more by the most frequent class, the empty squares. This class is easier to identify, since for a square to be empty it is enough that the corresponding move has not appeared in the sequence, without the need to track flips. On 16×16 , the gap between the two metrics reached 3.79 to 5.12 percentage points. Overall accuracy can therefore give an overly favourable picture, concealing more errors in the less frequent categories.

How well this information is learned depends strongly on scale, and also on the combination of architecture and objective. In Mamba-JEPA, macro-balanced accuracy falls monotonically from 98.04% on 8×8 to 92.85% on 12×12 and 83.77% on 16×16 , i.e. by 14.27 percentage points. In terms of decoding error, the deterioration is more than eightfold, from 1.96% to 16.23%. At the same time, JEPA affected the two architectures in opposite ways: it reduced decodability in the Transformer but improved it in Mamba at all three sizes. On 8×8 the changes were still marginal, -0.19 points in the Transformer and $+0.19$ in Mamba. On 16×16 , however, Transformer-JEPA lagged Transformer-AR by 4.51 points, while Mamba-JEPA kept a lead of 0.42 points over Mamba-AR. As a result, the difference between the two architectures under JEPA grew from 1.32

points on 8×8 to 5.40 on 16×16 . The almost complete decoding on the small board therefore compresses an interaction between architecture and training objective that becomes clear as scale increases.

The general drop on 16×16 does not mean that board information disappears from the representations, since performance remains far above the chance level of 33.3%. The decoding task does, however, become more demanding. From a representation of fixed dimension ($d_{\text{model}} = 512$), the linear probe must recover the state of 256 squares instead of 64. Nevertheless, the same requirements and the same probe protocol are applied to all four systems of each size, so the comparisons between them within the same scale remain valid (Section 6.5).

The form of the representation is equally telling. As the comparison of Tables F.3 and F.4 shows, describing each square as “mine”, “yours” or “empty” reveals structure that is more easily accessible to a linear probe than the absolute colour encoding (Li et al., 2023; Nanda, Lee and Wattenberg, 2023). The depth curves in Figure 5.5 also show that decodability is shaped during the processing of the sequence and does not appear only at the final layer.

On the criterion of linear decodability, the results ultimately give an affirmative answer to the question of this section: all twelve systems form internal representations from which the relative board state can be recovered. From here on, a “better representation” means higher linear decodability of the relative state. The clarity of this information, however, is not the same in all conditions. It depends on the architecture, the training objective and, above all, the scale of the problem. Moreover, a probe shows that information can be extracted from the representations, not that the model uses it during prediction (Belinkov, 2022). This second question is examined in Section 6.3.

6.3 Is the represented state used causally?

To test whether the board state plays a real role in the models’ decisions, targeted interventions were applied to the internal representations. In each case a counterfactual board was formed by switching the side of the piece on one square. The representation was then moved towards that of the new state, and we checked whether the prediction shifted towards the moves that are legal on the new board. Three different ways of modifying the vectors were examined in total: the established Li and Nanda methods as reference points from the literature, and an adapted variant, which was additionally compared with a geometrically matched random control. The differences between the three protocols are described in Section 3.6.3.

The three methods did not give equally stable results. The adapted intervention was the only one with a clear positive effect on the probability mass of the new legal moves in all twelve systems (Table 5.7). The Nanda method had a weaker but positive effect, while the Li method was more unstable and in some conditions caused a negative change. In addition, the adapted intervention outperformed the geometrically matched random control in every case.

The size of this effect depended mainly on scale. The advantage of the adapted intervention over the random control ranged from 6.5 to 7.3 percentage points on 8×8 , from 2.8 to 3.5 on 12×12 and from 1.2 to 2.0 on 16×16 . Within each size, the four systems were at most 0.8 points apart.

The mean effect, by contrast, shrank from 6.91 points on 8×8 to 1.58 on 16×16 , i.e. to less than a quarter. Descriptively, and unlike decoding, where a clear interaction between architecture and objective appeared, the causal effect seems to be differentiated more by board size. Because the interventions are not accompanied by multi-seed repetitions, this pattern is recorded descriptively and is examined further in Section 6.5.

The behaviour of the Li method mainly indicates limited transferability outside its original experimental condition. The method was developed for a Transformer on 8×8 and, in this thesis, had a clear positive effect only on the two systems with the same architecture and the same board size. On Mamba, where the intervention sites were mapped according to the relative depth of the network, its effect almost vanished. On the larger boards it also became weak or even negative. This pattern suggests that intervention sites and settings do not transfer automatically across architectures and scales. It does not prove a general failure of the Li method.

The comparison with the Nanda method is highly instructive. This method, like the adapted one, draws its directions from the same linear probe of the relative state. Their main difference is that the former uses only the vector corresponding to the new side of the piece, whereas the adapted method subtracts from it the vector of the original side, moving the representation simultaneously towards the new state and away from the previous one. In the adapted method, in addition, the intervention is applied after every block, is adapted to the scale of the activations and has its strength tuned on the validation set. These choices lead to a large difference in effectiveness: whereas the Nanda method had only a marginal effect (at most 0.7 points), the adapted method achieved a clearly larger shift in all cases. Because these individual modifications were applied in combination, it cannot be determined in isolation which contributed most to this advantage.

The causal-intervention approach has specific methodological limits. Each intervention examines one local change and only the immediately following prediction, not the consequences of an alternative game over time. In JEPA, the result concerns the complete system of the frozen encoder and the head trained after the fact. Finally, the strength of the adapted intervention was selected on the validation set. Its comparison with the Li and Nanda methods therefore concerns the settings applied here, not the maximum achievable performance of each method under an equal hyperparameter search.

Overall, the results give a positive but bounded answer to the question of this section. The systematic advantage of the adapted intervention over the geometrically matched random control shows that just any perturbation of the activations is not enough. When the representation is changed towards a specific counterfactual board state, the prediction moves towards the moves that are legal in that state. At least part of the information recovered by the probes is therefore not merely detectable but takes part functionally in shaping the immediately following prediction. The finding does not prove that the model maintains a complete and coherent model of the game, but it goes beyond a mere correlation between the activations and the board state.

6.4 Do the three levels of analysis agree?

Comparing the results first shows that which system comes out ahead depends on the evaluation criterion. At the behavioural level, Mamba-AR achieved the highest top-1 legality at all three sizes. Legal mass, which examines the whole probability distribution and not only the first choice, gives a more complex picture. Mamba kept its lead over the Transformer with both objectives, but the order of AR and JEPA was not stable: on 12×12 , JEPA had marginally higher legal mass in both architectures, with Mamba-JEPA recording the highest value. In decoding, Mamba-JEPA showed the best macro-balanced accuracy at all three sizes. The same divergence appeared in the causal interventions. Although Mamba-JEPA had the strongest probes, the advantage of the adapted intervention over the random control was smaller than that of Mamba-AR on 8×8 and 12×12 and slightly larger on 16×16 . These differences come from one training run per condition, so they are descriptive in nature.

Despite the different rankings, the three levels share a common core picture. All twelve systems select legal moves with high frequency and assign most of the probability to the legal set, form representations from which the relative board state can be recovered, and change their prediction when the represented state is modified towards a counterfactual board. Behavioural performance shows that the models have learned to operate according to the rules. The probes show that information about the state of the game is accessible in their activations. The advantage of the adapted intervention over the random control adds the evidence that part of this information also takes part in shaping the prediction.

The different rankings show that the clarity of the internal representation and its use in producing the next move are not the same thing. Mamba-JEPA organises the board state in a more cleanly linearly decodable form, but this advantage does not carry over intact to behaviour or to the strength of the causal response. One possible interpretation is that the head trained after the fact does not exploit all of the information available in the frozen encoder. In the AR models, by contrast, the encoder and the native head are optimised jointly for the next move, which may make the representations more aligned with the path that leads to the output.

This interpretation, however, concerns Mamba and does not explain the pattern of the Transformer. Transformer-JEPA lags Transformer-AR in linear decoding at all three sizes, where the evaluation is applied to the encoder’s representations and does not depend on the next-move head. The shortfall therefore already appears in the linear accessibility of the information and cannot be attributed only to the head trained after the fact. For the same reason, the opposite effect of JEPA on the two architectures with respect to decoding is not an artefact of the asymmetry between the heads. The divergences ultimately show that the presence of a piece of information, the form in which it is encoded and its functional use are distinct properties.

A further common trend appears as the board size increases, but it is not reflected with the same strength in all metrics. Top-1 legality declines relatively little, while legal mass, probe accuracy and the effect of the interventions fall much more. The models can therefore keep a correct top choice while the rest of the distribution, the clarity of the internal state and the sensitivity of the prediction to local interventions have already weakened. The significance of this divergence

is examined in more detail in Section 6.5.

Overall, the three levels agree qualitatively but not quantitatively. They converge on the evidence that the models form useful information about the board state and that part of it functionally affects the prediction. They do not agree, however, on which system has the best representation or on the size of its advantage. The most coherent interpretation therefore comes from combining behaviour, probes and causal interventions, and not from any single metric in isolation.

6.5 How does scale affect these findings?

The previous sections examined the effect of size at each level of analysis separately. When the results are considered together, it emerges that the change of scale is the largest source of observed variation in the study. In legal mass, in decoding and in the interventions, moving to the larger board shifts each system more than the four systems differ from one another, even on 16×16 , where the distances between them are larger. In decoding, the largest interaction of the factorial design, the +5.09 points of Table 5.6, corresponds to roughly one third of the drop that accompanies the increase in scale. This trend cannot be attributed only to the difficulty of the probe, since it also appears in legal mass, where no decoding procedure is involved.

Top-1 legality appears more robust, as it declines only slightly in absolute percentage points and could create the impression that behaviour is preserved almost unchanged. If the metrics are expressed as the distance from 100%, however, the picture changes. From 8×8 to 16×16 , the top-1 legality error is multiplied by 3.7 to 6.2 times, the share of probability assigned to illegal moves by 5.4 to 14.1 times and the decoding error by 5.5 to 8.3 times. These ratios are not self-contained effect sizes, since they are inflated by the very small initial errors. They do show, however, that the apparent robustness of top-1 is largely due to the fact that it already starts very close to the ceiling, i.e. a saturation of the metric. This partly explains why the four systems appear almost equivalent on 8×8 . A complementary explanation is that the small board is a simple enough problem to be learned effectively by all of them. The data do not allow the contributions of the two explanations to be fully separated.

The burden imposed by scale is not the same for all systems. In decoding, three of the four lose 14.0 to 14.5 percentage points, while Transformer-JEPA loses 18.4. The same system shows the largest drop in top-1 legality and, marginally, in legal mass. In decoding, therefore, the strengthening of the interaction between architecture and objective is linked mainly to the greater sensitivity of Transformer-JEPA to increasing complexity. The interaction is already measurable on 8×8 , but becomes clearer on the larger board.

In the interventions, the difference concerns mainly the architecture. On 16×16 , the two Transformers retain roughly 19% of the advantage they had over the random control on 8×8 , while the two Mamba models retain from 24% to 29%. In Mamba-JEPA, however, the higher percentage comes mainly from the larger drop caused by the random control on 16×16 and not from a larger shift towards the new legal moves. The evidence of greater robustness of Mamba is therefore clearer under AR.

Scale is not, however, a single isolated change. Together with the board size, the number of

positions, the move vocabulary, the length of the games and the number of possible states all increase, while the dimension of the representations remains fixed at $d_{\text{model}} = 512$. The results therefore capture the overall effect of this increase in complexity, without identifying which individual change is most responsible. Because the four systems are evaluated under the same conditions within each size, however, the comparisons between them remain valid.

Scale thus acts both as a source of difficulty and as a means of discriminating between the systems. 8×8 is not sufficient on its own to bring out their differences clearly. Evaluation on larger boards is therefore an essential part of the comparison, because it shows how well the performance of each system is preserved as the demands of the problem increase.

6.6 Main findings and research contribution

The first finding is that a JEPA-like objective can stand as a competitor to AR, without any next-move prediction term. The JEPA encoders approach the legality of AR through a simple head trained after the fact, and in Mamba they give the most easily decodable board representation at every scale. No system, however, is best at all levels. Mamba-AR is best at prediction and Mamba-JEPA at representation, while the ranking in the interventions depends on the board size (Sections 6.1, 6.2 and 6.4).

The second finding confirms and extends earlier research. Models trained only on move sequences learn not only to play legal moves but also to form an internal representation of the board state, which takes part in the prediction (Li et al., 2023; Nanda, Lee and Wattenberg, 2023; Torres, 2024; Vafa et al., 2025). This thesis extends these findings to 12×12 and 16×16 boards, as well as to an additional JEPA-like objective. It applies causal interventions in conditions where they had not been applied: in Mamba, in JEPA and on the larger boards. For this purpose an adapted intervention was used which, unlike the Nanda method, moves the representation not only towards the new state of the square but also away from the previous one. This intervention outperformed the geometrically matched random control in all conditions (Sections 6.1 to 6.3).

The third and most important finding is that the effect of the objective depends on the architecture, and that this becomes clearly visible only as the board grows. Scale is the largest source of variation in the study. Top-1 legality stays above 97%, but legal mass, state decoding and the causal effect fall sharply, and the differences between the systems grow. On 8×8 the models are very close to each other, either because the metrics approach saturation or because the environment is simple enough for all of them to learn it equally well. On 16×16 it becomes apparent that JEPA slightly improves the representation of Mamba while noticeably worsening that of the Transformer. This sensitivity of Transformer-JEPA, which is barely discernible on 8×8 , is the main source of the interaction between architecture and objective, which grows from +0.59 on 8×8 to +5.09 points on 16×16 (Table 5.6, Sections 6.2 and 6.5).

The research contribution of the thesis follows from these findings and has three parts. First, it designs and evaluates a standalone JEPA-like objective on discrete move sequences, without a parallel next-token prediction term, and shows that it can support behaviour close to AR. Second, it extends earlier results on prediction legality, the internal representation of the state and its

functional use, examining them on 12×12 and 16×16 and under a JEPA-like objective, with an adapted intervention method shared across all conditions. Third, and most importantly, it shows that the JEPA-like objective affects the linear decodability of the state differently depending on the architecture: it strengthens it in Mamba and weakens it in the Transformer. Under AR, the distance between the two architectures does not grow systematically with scale, whereas under JEPA it grows as the board grows. A comparison of architectures under AR alone, as in the earlier literature (Vafa et al., 2025), would therefore not reveal the interaction, while an evaluation on 8×8 alone would show it as a very small difference. Overall, the internal representation of a model is not a property of the architecture alone but is shaped by the interaction of architecture, objective and environmental complexity, and its evaluation requires all three levels of analysis.

6.7 Limitations and threats to validity

The data consist of random legal games. The evaluation concerns the learning of the constraints of the game, not the quality of strategic decisions or the ability to win. Moreover, Othello is a synthetic environment with a fully known and deterministic state. The ability to check every move and every counterfactual change exactly strengthens internal validity, but does not guarantee that the same representations will appear in partially observable or stochastic environments.

The factorial design includes two architectures, two training objectives and one roughly shared parameter scale. Matching the parameter count required different depths for the Transformer and Mamba, so it does not isolate the processing mechanism from the number of successive layers. Only one final JEPA configuration was examined, not the whole family of possible objectives. In the functional comparison, the native head of AR and the frozen head of JEPA, trained afterwards, are not equivalent learning paths. The differences between the complete systems in functional behaviour therefore do not measure exclusively the effect of the objective on the encoder.

Probes show that information can be recovered from the activations, not that the model uses it (Belinkov, 2022). On the large boards, part of the drop in decoding may be due to the capacity of the linear probe itself. Because the intervention directions come from the probes, the smaller causal effect on 16×16 may also reflect less accurate directions and not only less use of the information. In addition, the interventions test how a local change in the representation affects the immediately following prediction. They do not examine whether the model can consistently track the modified board state across a sequence of subsequent moves.

Finally, the five independent repetitions were carried out on 8×8 and 16×16 , not on 12×12 . The small number of seeds limits the precision of the estimates of training variability, while the bootstrap intervals describe only the variability of the test sample. Increasing the size changes more than one property of the problem at the same time, so the study records the overall effect of scale without isolating the contribution of each individual change to the drop in performance (Section 6.5).

6.8 Future directions

The most important open question concerns the interaction between architecture and objective: why the same JEPA-like objective improves the representation of Mamba and worsens that of the Transformer. A first direction is to examine more architectures, to see which feature of the mechanism is linked to the benefit. A second is to combine the two objectives, with the JEPA term added to next-move prediction (Huang, LeCun and Balestrieri, 2025), to test whether the shortfall of the Transformer under JEPA is offset.

A further direction would be to extend the independent repetitions to 12×12 and to examine more parameter scales. Varying width and depth independently would help separate the effect of the architecture from the effect of the number of layers. For a fairer comparison of the training objectives, the same kind of frozen head could also be trained on top of an AR encoder, to estimate how much of the observed gap is due to the readout path. Systematic re-runs of JEPA changing one choice at a time would clarify the effect of the view, the loss and the density of supervision.

Causal interventions need controls that separate the usefulness of a direction from its range of application. A shared budget for strength selection and number of intervention sites would make the comparison of the methods fairer. It would also be valuable to examine interventions at selected layers or in representation subspaces, and to check whether changing one square leaves the others unaffected. Testing several successive counterfactual moves would show whether the new state remains consistent beyond the immediately following prediction.

Finally, the same three-level evaluation framework can be tested in environments with partial observability or stochastic transitions. There, the reference state will not always be as clear as in Othello. This extension would test which of the relationships between behaviour, decoding and causal use depend on the specific game and which carry over to more complex dynamical systems.

Chapter 7

Conclusions

This thesis examined how the architecture, the objective and the scale of the environment affect learning from Othello move sequences. The conclusions concern the experimental conditions that were studied.

7.1 Answers to the research questions

- *Legal-move prediction (RQ1).* All systems exceeded 97% in top-1 legality, with Mamba-AR leading at all three sizes. Within each architecture AR outperformed JEPA, but with differences of at most 1.45 points: without any next-move prediction term, the JEPA systems stayed close to AR. Legal mass gave a more complex picture, as Mamba led the Transformer with both objectives while the order of AR and JEPA was not stable. The comparison concerns the complete systems, with a native head in AR and a frozen head in JEPA.
- *Decoding of the state (RQ2).* The board state was decodable in all conditions, with the relative description more easily accessible than the absolute one. Mamba-JEPA achieved the highest macro-balanced accuracy in linear decoding at all three sizes. JEPA improved decoding in Mamba and worsened it in the Transformer, with the interaction growing from +0.59 points on 8×8 to +5.09 on 16×16 , based on five independent repetitions at each size.
- *Causal effect on prediction (RQ3).* The adapted intervention increased the probability of the legal moves of the counterfactual board and outperformed the geometrically matched random control in all conditions. The Li and Nanda methods gave weaker or less stable results under the settings examined. On the basis of the findings, part of the decodable information is used in the immediately following prediction.
- *Relationships between the measurements (RQ4).* Better decoding was not always accompanied by better prediction or a stronger causal response. In Mamba, AR gave the best prediction and JEPA the most easily decodable representation, while the ranking in the interventions changed with size. Success at prediction, recoverable information and its functional use are therefore different aspects of a model and need to be evaluated together.
- *Effect of scale (RQ5).* Increasing the board was the largest source of observed variation,

with a sharp drop in legal mass, in decoding and in the causal effects. The larger boards revealed differences that were small on 8×8 , in particular the sensitivity of Transformer-JEPA. Evaluation on the smallest board alone is therefore not sufficient for comparing the systems.

7.2 Overall contribution

The contribution of the thesis follows from these answers and has three parts. First, it designs and evaluates a standalone JEPA-like objective for discrete move sequences, with a hard-disjoint prediction target, an EMA target encoder, InfoNCE and supervision at all valid positions. It shows that this objective can stand as a competitor to AR without a next-move prediction term, unlike LLM-JEPA (Huang, LeCun and Balestrieri, 2025), which combines the two terms. Second, it extends the known findings on legality, the representation of the state and its functional use to 12×12 and 16×16 boards and to a JEPA-like objective, with a single three-level evaluation framework and an adapted intervention method shared across all conditions. Third, and most importantly, it shows that the JEPA-like objective affects the linear decodability of the state in opposite directions in the two architectures. This interaction would not be visible in a comparison of architectures under AR alone, as in the earlier literature. The code, configurations and artefacts were organised so that every number and figure in the results can be traced back to the corresponding run.

7.3 Final assessment

The Othello environment shows that the evaluation of a sequence model remains incomplete when it is limited to the surface observation of almost saturated functional legality. Behind apparently equivalent external behaviour there may lie internal representations with fundamentally different geometric organisation and divergent sensitivity to causal interventions. The main idea of this thesis lies in quantifying and cross-checking these differences across two distinct architectural families and three levels of environmental scale. The next critical step is to isolate the deeper computational mechanisms that drive these divergences, and to investigate their robustness in broader fields of application of Artificial Intelligence.

Bibliography

- Alain, G. and Bengio, Y. (2018) ‘Understanding intermediate layers using linear classifier probes’. arXiv. Available at: <https://doi.org/10.48550/arXiv.1610.01644>.
- Assran, M. *et al.* (2023) ‘Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2301.08243>.
- Baevski, A. *et al.* (2022) ‘data2vec: A General Framework for Self-supervised Learning in Speech, Vision and Language’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2202.03555>.
- Bardes, A., Ponce, J. and LeCun, Y. (2022) ‘VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2105.04906>.
- Belinkov, Y. (2022) ‘Probing Classifiers: Promises, Shortcomings, and Advances’, *Computational Linguistics*, 48(1), pp. 207–219. Available at: https://doi.org/10.1162/coli_a_00422.
- Grill, J.-B. *et al.* (2020) ‘Bootstrap your own latent: A new approach to self-supervised learning’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2006.07733>.
- Gu, A. *et al.* (2020) ‘HiPPO: Recurrent Memory with Optimal Polynomial Projections’, *Advances in Neural Information Processing Systems*. Available at: <https://proceedings.neurips.cc/paper/2020/hash/102f0bb6efb3a6128a3c750dd16729be-Abstract.html>.
- Gu, A. and Dao, T. (2024) ‘Mamba: Linear-Time Sequence Modeling with Selective State Spaces’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2312.00752>.
- Gu, A., Goel, K. and Ré, C. (2022) ‘Efficiently Modeling Long Sequences with Structured State Spaces’.
- Hadsell, R., Chopra, S. and LeCun, Y. (2006) ‘Dimensionality Reduction by Learning an Invariant Mapping’, *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, Volume 2 (CVPR’06)*. New York, NY, USA: IEEE, pp. 1735–1742. Available at: <https://doi.org/10.1109/CVPR.2006.100>.
- Hazineh, D.S., Zhang, Z. and Chiu, J. (2023) ‘Linear Latent World Models in Simple Transformers: A Case Study on Othello-GPT’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2310.07582>.
- Huang, H., LeCun, Y. and Balestrieri, R. (2025) ‘LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2509.14252>.
- LeCun, Y. (2022) ‘A Path Towards Autonomous Machine Intelligence’, Version 0.9.2, 2022-06-27.
- Li, K. *et al.* (2023) ‘Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2210.13382>.
- Nanda, N., Lee, A. and Wattenberg, M. (2023) ‘Emergent Linear Representations in World Models of Self-Supervised Sequence Models’, *Proceedings of the 6th BlackboxNLP Workshop: Analyzing and*

- Interpreting Neural Networks for NLP*. Singapore: Association for Computational Linguistics, pp. 16–30. Available at: <https://doi.org/10.18653/v1/2023.blackboxnlp-1.2>.
- Oord, A. van den, Li, Y. and Vinyals, O. (2019) ‘Representation Learning with Contrastive Predictive Coding’. arXiv. Available at: <https://doi.org/10.48550/arXiv.1807.03748>.
- Rupprecht, C. *et al.* (2017) ‘Learning in an Uncertain World: Representing Ambiguity Through Multiple Hypotheses’. arXiv. Available at: <https://doi.org/10.48550/arXiv.1612.00197>.
- Siméoni, O. *et al.* (2025) ‘DINOv3’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2508.10104>.
- Tay, Y. *et al.* (2022) ‘Efficient Transformers: A Survey’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2009.06732>.
- Torres, A. (2024) ‘Othello Mamba: Evaluating the Mamba architecture on the OthelloGPT’. Available at: https://github.com/alxndrTL/othello_mamba.
- Vafa, K. *et al.* (2025) ‘What Has a Foundation Model Found? Using Inductive Bias to Probe for World Models’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2507.06952>.
- Vaswani, A. *et al.* (2017) ‘Attention Is All You Need’. arXiv. Available at: <https://doi.org/10.48550/arXiv.1706.03762>.
- Yuan, Y. and Søgaard, A. (2025) ‘Revisiting the Othello World Model Hypothesis’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2503.04421>.

Appendix A

Full Hyperparameters and Configurations

This appendix collects the settings that define the main training runs and the shared downstream evaluation protocol. Unless stated otherwise, they apply to all twelve combinations of architecture, training objective and board size.

A.1 Shared main-training protocol

Each main model is trained on one full pass over shards 0–199. The nominal size of each shard is 100,000 games. The 8×8 corpus contains 19,999,840 games in total across these shards, while the 12×12 and 16×16 corpora contain 20,000,000 each. With a batch size of 256, the completed main training runs record 78,200 optimisation steps. Main training is carried out in bf16 (Brain Floating Point), which reduces memory requirements while keeping the dynamic range of fp32, thus preventing numerical instability.

A.2 Input dimensions per board size

The vocabulary contains one token for each of the squares that can appear as a move after the initial placement of the four pieces, plus one padding token. A pass is handled by the game-generation logic and does not introduce a separate token. The block size is one position shorter than the maximum game length, because the input and target sequences are shifted by one step.

A.3 Per-architecture settings

The two model families share the same representation width, the same learned token and positional embeddings, a final LayerNorm and a bias-free native output projection. The settings that differentiate the sequence-processing mechanism are collected in Table A.3.

The linear layers and embeddings defined by the implementation are initialised from a normal

Table A.1: Shared hyperparameters and operational settings of the main training

Setting	Value and application
Main training data	Shards 0–199
Passes over the data	1
Nominal shard size	100,000 games
Optimisation steps	78,200, computed automatically from the data
Batch size	256 games
Sample order	Shuffled independently within each shard
Optimiser	AdamW
Initial learning rate	0.0003
AdamW β_1, β_2	0.9 and 0.95
AdamW ϵ	10^{-8} , PyTorch default
Weight decay	0.01, with no parameter-group exclusions
Gradient clipping	Maximum total norm 1.0
Linear warm-up	1,000 optimisation steps
Learning-rate schedule	Cosine decay after warm-up
Minimum learning rate	10% of the initial value, i.e. 0.00003
Numerical precision	bf16 autocast; no GradScaler
Main seed	42
Additional repetition seeds	0, 1, 2 and 3
Data-loader workers	0
Evaluation during training	1 evaluation shard per 5 training shards
Additional shard restriction	None beyond the first 200
Milestone checkpoints	After 1, 5, 10, 15 and 20 million games
Additional JEPA checkpoints	At 10%, 25% and 50% for the 12×12 and 16×16 runs
Periodic per-shard checkpoint	Disabled; the predefined milestones are kept
Model compilation	Disabled

Table A.2: Vocabulary, sequence and main-training-set dimensions per board size

Board	Max moves	vocab_size	Padding token	block_size	Games in shards 0–199
8×8	60	61	60	59	19,999,840
12×12	140	141	140	139	20,000,000
16×16	252	253	252	251	20,000,000

Table A.3: Final architectural settings of the Transformer and Mamba encoders

Setting	Transformer	Mamba
<code>d_model</code>	512	512
Number of layers	8 Transformer blocks	15 Mamba blocks
Sequence mechanism	Causal multi-head self-attention	Mamba selective state space
Attention heads	8	Not applicable
Dimension per head	64	Not applicable
Attention mask	Lower-triangular causal mask	Not applicable
Transformer MLP	Hidden dimension 2,048	Disabled
MLP activation	GELU	Not applicable
<code>d_state</code>	Not applicable	16
<code>d_conv</code>	Not applicable	4
<code>expand</code>	Not applicable	2
Mamba inner dimension	Not applicable	1,024
<code>dt_rank</code>	Not applicable	32, derived from <code>d_model</code>
<code>mlp_hidden_mult</code>	Not applicable	0
Normalisation	Pre-norm on attention and MLP; final LayerNorm	Pre-norm before the Mamba mixer; final LayerNorm
Dropout	0.1 on embeddings, attention, residual projection and MLP	0.1 on embeddings and residual path
Native output projection	Linear, no bias	Linear, no bias
Backend	Causal Transformer implementation of this thesis	Strictly <code>mamba_ssm</code> , no automatic fallback
Mamba runtime versions	Not applicable	<code>mamba_ssm</code> 2.3.2.post1; <code>causal_conv1d</code> 1.6.2.post1

distribution with mean 0 and standard deviation 0.02, while the available biases are set to zero. In the Transformer, the weights of the residual projections `c_proj` use a standard deviation of 0.005, adjusted to the depth of eight layers. The internal parameters of the official Mamba mixer keep the initialisation of the `mamba_ssm` library.

A.4 Per-objective settings

The architecture of the main encoder remains unchanged between the Autoregressive and the JEPA-like arms. What changes is the learning signal and the objective-specific parts of the model. In the final JEPA-like condition, each valid prefix predicts the representation of the immediately following move, without the target move being included in the context.

Table A.4: Objective-specific settings of the final Autoregressive and JEPA-like comparison

Setting	Autoregressive	JEPA-like
Objective	Next move token	Representation of the next move
Loss	Cross-entropy on valid tokens	InfoNCE
Supervised positions	All valid positions of the shifted sequence	All valid prefix boundaries
Prediction horizon	1 token	1 move
Context-target separation	Input and target shifted by one step	<code>hard_disjoint_future</code> ; the target does not enter the context
Prediction head	Native output projection to the vocabulary	Separate linear predictor $512 \rightarrow 512$ with bias
Predictor parameters	Already included in the base architecture	262,656
Target encoder	Not used	Frozen copy of the context encoder
Target-encoder update	Not applicable	Exponential moving average
EMA momentum	Not applicable	0.996
Contrastive temperature	Not applicable	0.1
Negatives	Not applicable	Other games at the same absolute sequence position
Normalisation before similarity	Not applicable	L2 normalisation of the embeddings

The chunk sizes in Table A.5 only limit the maximum temporary memory use while computing the JEPA-like loss. They do not change the context-target pairs, the negatives or the mathematical value of the objective.

The run configuration also stores fields that belong to earlier or alternative JEPA variants. Settings for VICReg, action-conditioned or multi-action objectives are not active in the final comparison and are therefore not treated as hyperparameters of the models analysed.

A.5 Parameter-count matching

Matching was carried out on the shared base architecture and not on the total size of the saved JEPA-like checkpoint. A shared `d_model` of 512 was fixed first, and the same encoder geometry

Table A.5: Memory-chunking settings for the JEPA-like all-position objective

Architecture	Board	Target chunk	Statistics chunk
Transformer	8×8	65,536	64
Transformer	12×12	4,096	8
Transformer	16×16	4,096	8
Mamba	8×8	65,536	64
Mamba	12×12	65,536	138
Mamba	16×16	65,536	250

was kept for the two training objectives. Eight Transformer blocks and fifteen Mamba blocks were then selected. The count was repeated for each board size, because the token embeddings, the positional embeddings and the native output projection change. With this procedure, the deviation remains below 0.86% at all scales.

Table A.6: Exact parameter count of the base architecture, including the native output projection

Board	Transformer	Mamba	Difference	Mamba excess
8×8	25,312,768	25,529,856	217,088	0.858%
12×12	25,435,648	25,652,736	217,088	0.854%
16×16	25,607,680	25,824,768	217,088	0.848%

In the Autoregressive arm, the number in Table A.6 is also the total size of the model. In the JEPA-like arm, the linear predictor and a full target copy of the encoder are stored in addition. The target copy is excluded from the optimiser and is updated only through EMA. Furthermore, the native output projection of the live encoder is bypassed by the `encode_hidden` path and receives no gradient from the JEPA-like loss. For this reason, the total checkpoint size is not a suitable measure of trainable capacity.

Table A.7: Parameter-count structure of the final JEPA-like checkpoints. The active path includes the 262,656-parameter predictor and excludes the inactive native output projection.

Model	Base	Active gradient path	Frozen target	Checkpoint total
Transformer JEPA 8×8	25,312,768	25,544,192	25,312,768	50,888,192
Transformer JEPA 12×12	25,435,648	25,626,112	25,435,648	51,133,952
Transformer JEPA 16×16	25,607,680	25,740,800	25,607,680	51,478,016
Mamba JEPA 8×8	25,529,856	25,761,280	25,529,856	51,322,368
Mamba JEPA 12×12	25,652,736	25,843,200	25,652,736	51,568,128
Mamba JEPA 16×16	25,824,768	25,957,888	25,824,768	51,912,192

A.6 Shared downstream evaluation protocol

`unified_eval_v4` requires exactly 238 sorted shards and creates the split before any downstream training or selection. The selection and final-test shards take no part in any weight update. For training the frozen heads, the pool starts from shards 200–219 and, if needed, continues with pretraining shards. The upper limit is 200 shards in total, so the 20 initial downstream shards and up to 180 pretraining shards can be used.

Table A.8: Fixed split of the 238 shards in the shared evaluation protocol

Role	Shards	Count	Use
Main-model training	0–199	200	Weight updates of the main encoder
Initial downstream training	200–219	20	First training pool for frozen heads and probes
Validation	220–228	9	Selection, early stopping and intervention tuning; no encoder update
Final evaluation	229–237	9	Final metrics; no weight update

The frozen next-move heads use the final representation of the frozen encoder at each sequence position. The architecture-neutral comparison includes a linear head and a one-hidden-layer MLP. Before the final streaming refit, the head’s hyperparameters are selected with grouped cross-validation exclusively within the head’s training pool.

Table A.9: Fixed training and final-test settings of the frozen next-move heads

Frozen-head setting	Value
Encoder state	Frozen, in evaluation mode
Encoder precision	bf16
Head precision	fp32
Head families	Linear and MLP
MLP architecture	Hidden layer 512, GELU, dropout selected from the grid
Loss	Next-move cross-entropy
Optimiser	AdamW
Streaming-refit batch size	512 games
Gradient clipping	Maximum total norm 1.0
Maximum training pool	200 shards
Training-pool order	200–219, then 0–199
Selection sample	2,000 games from shards 220–228
Selection frequency	After every training shard
Early-stopping patience	5 consecutive checks
Early-stopping <code>min_delta</code>	0.001 in legal probability mass
Linear-head seed	42
MLP-head seed	10,042
Final legal-move test	50,000 games from shards 229–237
Final-test batch size	64
Values of k	1, 2, 3 and 5

The board-state probes are trained independently for each depth and for both board encodings. Level 0 corresponds to the representation before the successive blocks, followed by one probe after each block: nine depths for the Transformer and sixteen for Mamba. In all cases, only the representation of the last available position of the prefix is used.

The parameters of the causal interventions are not repeated here, because they are described together with the checks for selecting the strength, layer and direction in Appendix D.

Table A.10: Hyperparameter grid of the frozen heads. Selection uses 5,000 games, 3 folds grouped by shard, 3 epochs per fit and a token batch of 8,192, with mean legal probability mass as the criterion.

Head	Learning rates	Regularisation / dropout	Hidden	Candidates
Linear	0.0003; 0.001	None; L2 weight decay 0.001; L1 strength 0.00001	n/a	6
MLP	0.0003; 0.001	No weight penalty; dropout 0 or 0.1	512	4

Table A.11: Full hyperparameters of the board-state probes

Board-state probe setting	Value
Probe families	Linear and MLP
Targets	Absolute and relative state of each square; 3 classes
Depths	All depths: 0–8 for the Transformer, 0–15 for Mamba
Sequence position	Last available position of each prefix
Input normalisation	LayerNorm without learnable parameters
MLP architecture	Hidden layer 512, GELU, dropout 0.1
Optimiser	AdamW
Linear learning rate	0.001
MLP learning rate	0.0003
Weight decay	0
Gradient clipping	Maximum total norm 1.0
Feature-extraction batch	64 games
Probe-training batch	256 position examples
Maximum epochs	5
Early-stopping patience	2 epochs
Early-stopping <code>min_delta</code>	0.0005 in macro-balanced accuracy
Positions per game	16 normalised positions
Training sample	3,125 games; 50,000 positions
Selection sample	512 games; 8,192 positions
Test sample	1,000 games; 16,000 positions
Linear-probe seed	42
MLP-probe seed	20,042

Appendix B

Mathematical Details of the Architectures and Objectives

This appendix collects the mathematical and architectural details that complement the theoretical background without interrupting the main narrative. It covers self-attention, the transition from continuous to discrete SSMs, and the core relations of the two training objectives.

B.1 The attention mechanism in the Transformer

The core of each Transformer layer is the attention mechanism. From the input matrix X , which contains the current representation of each position of the sequence, three linear projections are produced with learnable weight matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (\text{B.1})$$

The three projections have distinct roles. The query q_i of a position expresses what that position is looking for in the history. The key k_j of each previous position expresses what that position offers as information. The product $q_i \cdot k_j$ scores the relevance of the two positions and, after normalisation with softmax, gives the weight with which position i is influenced by j . The value v_j is the content actually carried from j to i : the total output of position i is the weighted mean of the values of all permitted positions. Dividing by the square root of the dimension d_k keeps the size of the dot products in check, so that the softmax does not saturate in high dimensions.

Multi-head attention repeats the computation in parallel lower-dimensional subspaces, with separate projection matrices per head. The individual outputs are concatenated and projected back to the model dimension. Different heads can thus organise different relationships within the same history.

The output of attention is added through a residual connection to the representation of the position and is followed by a feed-forward network and a further residual connection, with layer normalisation in each sub-layer. By repeating this structure, the residual stream of each layer gradually gathers information from the entire available prefix. This is precisely the quantity

examined with probes: not the attention weights in isolation, but the representations formed after the successive updates of each layer.

B.2 Continuous form and discretisation of the State Space Model

In its continuous form, a linear SSM describes the evolution of a hidden state $h(t)$ under the influence of an input $x(t)$:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t) \quad (\text{B.2})$$

$h(t)$ condenses the history, $x(t)$ is the current input and $y(t)$ the resulting output. To apply the model to discrete sequences, the system is discretised with a step Δ , which converts the continuous matrices A and B into their discrete counterparts and leads to the recurrent form used in Section 2.2. The step Δ acts as a time scale: smaller values correspond to denser sampling and slower decay of older information.

With constant input over each time interval of length Δ , the exact discretisation is given by:

$$A_d = e^{\Delta A}, \quad B_d = \int_0^\Delta e^{sA} B ds \quad (\text{B.3})$$

These relations concern the linear SSM. The selectivity of Mamba allows key parameters of the update to depend on the input, as explained in Section B.4.

B.3 Relationship with RNNs and HiPPO

RNNs and discrete SSMs share one feature: they update their internal state by combining the previous state with the current input. A key difference lies in how this update is performed. In a classic RNN, the combination passes through a nonlinear activation function, such as $\tanh$, at every time step. In the core of a linear structured SSM, by contrast, the state is updated without such a function inside the recurrence. This does not mean that the whole layer is linear: nonlinear functions and gating mechanisms are applied around the SSM core. This separation facilitates the analysis of memory and parallel computation, without on its own guaranteeing the elimination of vanishing or exploding gradients.

In Mamba there is an additional important difference: the parameters of the update depend on the input. Therefore, although no activation function is applied to the recurrent state itself as in a classic RNN, the overall input-output relation of the selective SSM is not linear. The model can thus adapt which information it keeps and which it discards depending on the content of the sequence (Gu and Dao, 2024).

High-order Polynomial Projection Operators (HiPPO) formulate memory as a continuously updated projection of the past onto a basis of orthogonal polynomials. The construction provides specific transition matrices with controlled memory properties and was an important step towards S4 (Gu et al., 2020; Gu, Goel and Ré, 2022). This does not mean that every SSM eliminates the

vanishing gradient, nor that every Mamba layer uses a HiPPO matrix verbatim. The connection is historical and a matter of design: long memory is built into the structure of the dynamics instead of being left to emerge solely from a generic recurrent update.

B.4 Selective scan and computational execution

In the first structured SSMs, the transition parameters were the same at every position, which allowed the sequence to be equivalently formulated as a convolution. Mamba makes the parameters Δ , B and C functions of the input, so that each token affects differently what is admitted into the state, what is retained and what is attenuated (Gu and Dao, 2024). Because this selectivity rules out the use of a single fixed convolution, the architecture uses a hardware-aware selective scan for parallel execution during training and a recurrent update during generation.

In a Mamba block, a short causal convolution captures local patterns before the selective SSM, while a second branch acts as a gate before the final projection and the residual connection. The local convolution, the selective state and the gating work together without forming a $T \times T$ interaction matrix. This organisation explains the linear scaling of Mamba and the different mechanism through which a representation of the Othello state can be formed.

B.5 Details of the Autoregressive objective

During training, the input of the model at each position is the actual prefix of the game and not its own previous predictions, a practice known as teacher forcing. The loss terms for all valid positions are computed with a single forward pass. Access to future moves is blocked by the causal mask in the Transformer and by the causal structure of the processing in Mamba.

The details of the present implementation regarding padding, masking of padded positions and the prediction head are presented in Sections 3.3 and 3.5 and in Code Listing G.1.2.

B.6 Details of the JEPA-like objective

The target encoder is kept outside the gradient path. The parameters ξ of the target encoder are updated as an Exponential Moving Average (EMA) of the parameters θ of the context encoder:

$$\xi \leftarrow m \xi + (1 - m) \theta \tag{B.4}$$

With m close to one, the target encoder changes more slowly and provides a more stable reference point for prediction. Being frozen with respect to the gradient therefore does not mean that its parameters remain permanently fixed; they are updated by the EMA and not by the optimiser.

The InfoNCE temperature controls how sharply the loss separates small differences in similarity: lower values intensify the competition between candidate targets. How the positive and negative pairs are formed, and the choice of time positions for the context and target views, are controlled choices of the experimental design and are described in the Methodology.

Appendix C

Detailed Evolution of the JEPA Experiments

This appendix collects the configurations and metrics that supported the decisions of Chapter 4. It separates the early development results from the later re-runs under the unified protocol, because not all values are directly comparable.

C.1 Scope and comparability

v1–v6 formed the main development sequence, while v7–v9 and the multi-positive InfoNCE family were later explorations. The run names preserve the history of the search: the name v5 covers both early prefix-grouped branches and the final InfoNCE configuration of Section 4.4. The tables distinguish them by view, predictor and supervised positions. The development measurements of `unified_eval_v2` do not coincide with the final re-evaluation of `unified_eval_v4` in Chapter 5. This is why the final configuration appears here with 96.20% macro-balanced accuracy on the Transformer and 97.65% on Mamba, instead of the 96.72% and 98.04% of Table 5.4. These are neither additional independent training runs nor values that should be combined into the same mean.

C.2 Early Transformer runs

Table C.2 uses the archived results of the initial evaluation. The top-1 value refers to the legality of the top prediction. The relative-probe path reports the accuracy at the initial layer L0, at the best layer and at the last layer. These values do not come from the final test protocol.

In a later single-boundary check, hard-disjoint v1 VICReg with the full budget reached 73.25% top-1 with a linear head and 79.11% with an MLP, with a best relative linear-probe accuracy of 70.29% at L5. The hard-disjoint re-runs of v2 and v4 had only 5 million games and are not used for a clean numerical comparison. The older prefix-grouped hard-disjoint v5 InfoNCE reached 57.78% top-1 with a linear head and 65.7% with an MLP, and remained far below the final all-position version.

Table C.1: Main configurations of the development sequence v1–v6 and of the final objective

Version	View and target	Loss	Predictor / horizon	Research question
v1	Nested; shared context/target history	VICReg	Linear, $K = 1$	Baseline and collapse check
v2	Nested; EMA target	Smooth L1	Transformer, $K = 4$	Stabilising the target
v3	Nested; EMA target	Smooth L1	Small MLP, $K = 8$	Weaker predictor and longer horizon
v4	<code>disjoint_future</code> ; the target keeps the history	Smooth L1	Multi-position MLP, $K = 4$	Predicting four future representations
v5 exploratory	<code>disjoint_future</code> and prefix-grouped hard-disjoint branches	InfoNCE	MLP, $K = 1$	Contrastive discrimination; not the final implementation
v6	Hard-disjoint next action token	Distance-margin	MLP, 10 modes	Multiple candidate representations
Final InfoNCE	Hard-disjoint next token; EMA	InfoNCE per absolute position	Linear, $K = 1$	All-position canonical objective

Table C.2: Early Transformer development results on 8×8 under the historical evaluation protocol

Version	Top-1 legal	Relative probe: L0 → best → final	Main observation
v1	25.71%	62.41% → 62.41% (L0) → 49.29%	Decodability decreases with depth
v2	38.60%	60.91% → 62.00% (L6) → 61.93%	More stable but weak representation
v3	27.24%	59.73% → 59.73% (L0) → 57.61%	$K = 8$ and the smaller predictor did not help
v4	33.55%	60.54% → 62.67% (L1) → 59.36%	The target still saw the history
v5 exploratory	39.58%	61.96% → 63.01% (L5) → 62.39%	InfoNCE alone was not enough
v6 preliminary	55.88%	60.72% → 60.72% (L0) → 58.66%	Top-1 improved, the representation did not

C.3 All-position Transformer re-runs

The runs in Table C.3 were trained on 19,999,840 games and evaluated with `unified_eval_v2`. For consistency, the metrics of the linear head and of the relative linear probe at the layer selected exclusively on the selection split are reported.

Table C.3: All-position Transformer results on 8×8 with `unified_eval_v2`

Configuration	Top-1 legal	Relative macro	Relative overall	Layer
v1 nested VICReg	25.43%	58.35%	65.09%	L0
v1 HD VICReg	98.99%	90.17%	92.33%	L2
v1 HD VICReg + EMA	99.38%	92.89%	94.45%	L3
v2 HD Smooth L1, $K = 4$	98.87%	88.24%	90.81%	L2
v3 HD Smooth L1, $K = 8$	96.48%	87.69%	90.40%	L6
v4 HD Smooth L1, $K = 4$	97.48%	89.43%	91.76%	L3
Final HD InfoNCE	99.42%	96.20%	97.04%	L6
v6 HD distance-margin	60.62%	59.64%	66.75%	L7

Setting the first row against the second highlights the essential finding: all-position supervision did not fix the nested view, while strict separation of the target led to a jump of 73.56 percentage points in top-1 legal accuracy. EMA further improved the representation. The final InfoNCE gave the highest relative macro and overall accuracy among the Transformer-JEPA variants.

v3 appears here because an archived all-position re-run exists. It is not included in the later stages of the selected diagram in Chapter 4, where the shorter path v1, v2, v4 and v5 is kept. Its exclusion from the diagram does not mean that the run is absent from the results archive.

C.4 Transfer to Mamba

Mamba was used as a second check of the generality of the conclusions. The first row of Table C.4 comes from the older `normalized_v2` validation protocol and is listed because it marked the turning point in the development timeline. The remaining rows come from `unified_eval_v2` and from the fixed test split.

Table C.4: Main Mamba-JEPA development results on 8×8

Configuration	Top-1 legal	Relative macro	Relative overall	Layer / split
v1 HD VICReg; single position	94.26%	n/a	84.37%	L7; validation
v1 HD VICReg; all-position	99.47%	94.97%	96.07%	L7; test
v1 HD VICReg + EMA; all-position	99.72%	96.30%	97.13%	L7; test
Final HD InfoNCE; all-position	99.72%	97.65%	98.17%	L12; test

The most recent valid evaluation of Mamba-v5 hard-disjoint, before all-position supervision, replaces the older evaluation that had been affected by numerical overflow. With an MLP head it gave 54.42% top-1 legality and with an MLP probe a best overall relative-state accuracy of 65.21% at L0. The values come from 5,000 evaluation games and are shown in Figure 4.2. They remain clearly lower than the performance of the final all-position configuration and must not be confused with the 94.26% of hard-disjoint v1 VICReg.

C.5 Later exploratory directions

The following branches examined more complex forms of prediction and grouping of positive examples. They are listed to delimit the scope of the exploration, not as additional conditions of the final comparison.

Table C.5: Later JEPA directions not included in the main experimental setup

Branch	Main configuration	Research question
v7	Action-conditioned Smooth L1, grouped InfoNCE and hybrid CE	Explicitly conditioning the predictor on the move
v8	Cross-order positives, same-set hard negatives and hybrid variants	Learning invariance to permutations that lead to the same state
v9	Multi-action rollout with Smooth L1 and InfoNCE	Predicting several successive actions in representation space
Family InfoNCE	Offline permutation families and multi-positive InfoNCE	Using equivalent prefixes as contrastive classes

These branches are kept in `JEPA_Experimentation` as a record of the research, but they are not treated as additional cells of the main factorial design. The final choice remained the simpler hard-disjoint, $K = 1$, EMA, linear-predictor InfoNCE setup, based on the criteria of Section 4.4. These historical branches are not used to claim superiority under fully controlled conditions.

Appendix D

Details of the Causal Interventions

This appendix records the exact constants, depth mappings and checks of the three causal protocols. The Transformer-AR 8×8 run is the only setting in which the published sites and constants can be applied directly. The JEPA and Mamba conditions and the larger boards are declared explicitly as extensions. In all twelve conditions the three methods use the same counterfactual cases and the same prediction head, so that the comparison is not affected by a different sample.

D.1 Construction and splitting of the counterfactual cases

For each case, a prefix with a length from 4 up to two positions before the end of the game, and an occupied square, are selected deterministically. The colour of the piece is flipped, and the case is kept only if the new board has at least one legal move and its legal set differs from the original one. Strength selection is performed on 200 cases from the validation shards, while the final metrics are computed once on 1,000 different cases from the final-evaluation shards.

Table D.1: Shared settings for constructing and evaluating the counterfactual cases

Element	Setting
Board change	Flipping the colour of one occupied square
Prefix length	Random but deterministic choice from 4 to $\text{len}(\text{game})-2$
Acceptance criterion	Non-empty and different set of legal moves
Selection cases	200 from shards 220–228; seed 50,042
Final-test cases	1,000 from shards 229–237; seed 60,042
Batch size	64 cases
Shared control	Unmodified run on the same 1,000 cases
Random control	Shared random coordinate permutation and sign flip across all layers, preserving the norms and the angles between them; seed 70,042

D.2 The Li, Nanda and adapted intervention protocols

The Li method trains a nonlinear probe of the absolute state and modifies the representation directly with 1,000 Adam steps. The nonlinear probe is trained separately from the intervention,

on the same 50,000 probe-training positions, for five epochs with Adam, learning rate 0.001, batch size 256 and gradient clipping 1.0. The epoch with the highest mean validation accuracy over the selected layers is kept. The Nanda method uses a raw linear probe of the relative state and adds the normalised vector of the target class to selected residual branches. The adapted method uses the difference target class minus source class, applies it after every block and scales the change by the standard deviation of the current representation. The first two methods follow the respective directions in the literature (Li et al., 2023; Nanda, Lee and Wattenberg, 2023), while the third is the unified extension of this thesis.

Table D.2: Defining elements of the three causal intervention protocols

Method	Probe / direction	Application	Constants and selection
Li	Nonlinear absolute probe, Linear-ReLU-Linear, hidden 128	Optimisation of the representation at the selected post-block depths	Adam, lr 0.001, 1,000 steps, preservation weight 0.2
Nanda	Raw linear relative probe; target-class vector	Attention output or Mamba mixer residual branch at the selected blocks	Fixed strength 2.3; no selection on validation
Adapted	Raw linear relative probe; target minus source	Last position after every block, with residual-std scaling	$\alpha \in \{1, 2, 4, 8\}$, selected on the 200 cases; geometrically matched random control

The published sites of the 8×8 Transformer are mapped onto the fifteen Mamba blocks according to normalised depth. In the Li method, layers are numbered after each block. In the Nanda method, the residual branches keep the zero-based numbering of the implementation, while the probe layer is expressed as the output after a block. The mapping preserves the relative position within the stack, not a functional identity between attention and the selective SSM.

Table D.3: Mapping of the intervention sites by normalised depth. The Nanda indices are zero-based.

Architecture	Blocks	Li: post-block layers	Nanda: branch indices	Nanda probe layer
Transformer	8	L4, L5, L6, L7, L8	2, 3, 4, 5, 6, 7	L6
Mamba	15	L8, L9, L11, L13, L15	4, 6, 8, 10, 12, 14	L11

D.3 Head and strength selection for the adapted method

In the AR models the native head is used. In JEPA, the linear or MLP frozen head is selected before the causal interventions with the probability mass on legal moves in the validation split as the sole criterion; the test set takes no part in the decision. In the adapted method, the value of α is chosen that minimises false positives plus false negatives among the top N predictions of the 200 cases. Ties are broken in favour of the larger probability mass on the counterfactual legal set and, if the tie persists, the smaller α .

Table D.4: Head and strength α selected without access to the 1,000 final-test cases

System	8×8	12×12	16×16
Transformer-AR	Native; $\alpha = 2$	Native; $\alpha = 2$	Native; $\alpha = 1$
Transformer-JEPA	Linear; $\alpha = 4$	Linear; $\alpha = 2$	Linear; $\alpha = 2$
Mamba-AR	Native; $\alpha = 4$	Native; $\alpha = 2$	Native; $\alpha = 1$
Mamba-JEPA	MLP; $\alpha = 4$	Linear; $\alpha = 2$	Linear; $\alpha = 2$

D.4 Metrics and limits of interpretation

Top-1 legality measures whether the most likely prediction belongs to the legal set of the counterfactual board. Legal mass is the sum of the probabilities of all moves in that set. For the third metric, N is set equal to the number of counterfactual legal moves, the N highest predictions are taken, and the illegal predictions that were introduced and the legal moves that were omitted are summed. Higher values on the first two metrics and a lower value on the third therefore indicate better adaptation to the modified state.

Each effect is reported as the difference from the unmodified run on the same cases. For the adapted method, the difference from the geometrically matched random control is also computed. This control is critical, because it tests whether the effect depends on the target-oriented direction and not merely on introducing a perturbation of corresponding geometry into the residual stream. A positive change against the geometrically matched random control supports a causal link, but does not prove that the linear direction is the only or the complete internal representation of the board.

D.5 Supplementary causal results

The following tables complement the legal mass of Table 5.7 with top-1 legality and the errors of the predicted legal set. The first three value columns compare each intervention with the unmodified run. The last compares the adapted method with the geometrically matched random control, on the same 1,000 cases. T denotes Transformer and M denotes Mamba.

The comparison between the three methods is descriptive with respect to the specific settings. Only the adapted method selects its strength on validation, and it also uses more intervention sites. The random control does not equalise these differences and does not establish a general superiority under equal tuning resources.

Table D.5: Change in top-1 legality on the counterfactual board, in percentage points. Positive values indicate improvement.

Board	System	Li Δ	Nanda Δ	Adapted Δ	Adapted-random
8×8	T-AR	+5.40	+1.20	+6.70	+7.20
8×8	T-JEPA	+4.80	+1.40	+6.50	+7.50
8×8	M-AR	+0.70	+0.20	+6.40	+8.60
8×8	M-JEPA	+1.10	+0.10	+7.90	+6.40
12×12	T-AR	−1.00	+0.90	+2.70	+4.20
12×12	T-JEPA	+0.20	+1.20	+3.20	+3.40
12×12	M-AR	+0.90	+0.40	+3.50	+4.80
12×12	M-JEPA	+1.60	+0.20	+4.70	+4.80
16×16	T-AR	−0.90	+2.40	+3.00	+2.80
16×16	T-JEPA	−0.10	−0.30	−0.10	+0.40
16×16	M-AR	+0.40	+0.40	+2.60	+5.10
16×16	M-JEPA	−0.30	+0.10	+1.30	+2.10

Table D.6: Change in the sum of false positives and false negatives among the top N predictions, as the mean number of errors per case. Negative values indicate improvement.

Board	System	Li Δ	Nanda Δ	Adapted Δ	Adapted-random
8×8	T-AR	−0.93	−0.51	−1.86	−1.90
8×8	T-JEPA	−0.69	−0.32	−2.12	−2.11
8×8	M-AR	−0.14	−0.02	−2.03	−2.08
8×8	M-JEPA	−0.07	−0.01	−2.17	−2.10
12×12	T-AR	+2.67	−0.63	−1.24	−1.59
12×12	T-JEPA	+3.14	−0.69	−1.25	−1.32
12×12	M-AR	+0.02	−0.02	−1.50	−1.82
12×12	M-JEPA	+0.01	−0.02	−1.63	−1.73
16×16	T-AR	+5.05	−0.43	−0.57	−0.89
16×16	T-JEPA	+4.67	−0.24	−0.45	−0.90
16×16	M-AR	+0.16	−0.01	−0.81	−1.26
16×16	M-JEPA	+0.15	−0.04	−0.80	−1.47

Appendix E

Reproducibility and Experimental Records

Reproducibility is organised at four levels: a recorded execution environment, explicit random seeds, data and checkpoint identity, and self-contained experimental records per stage. This information makes it possible to check exactly which training run, data split and protocol version produced each result. It does not remove the not fully deterministic behaviour of some GPU kernels, which is why training variability is estimated from independent runs and not from repeating the evaluation on the same checkpoint.

E.1 Computational environment and version records

The main training runs were executed on Google Colab with an NVIDIA RTX PRO 6000 Blackwell 96 GB, and the final shared evaluations of all twelve conditions on an NVIDIA L4 24 GB. The encoder uses bf16, while the frozen heads and probes are trained in fp32. The software versions in Table E.1 are those of the `replication_environment_lock.json` files of the seeded families. Because Google Colab periodically updates its runtime software, runs on different dates in some cases recorded different versions; for example, the shared evaluations of the two AR models on 8×8 record Python 3.13.15. For the exact reproduction of each run, its own metadata are used.

E.2 Random seeds and independent repetitions

Seed 42 defines the twelve canonical models. Seeds 0, 1, 2 and 3 complete the family of five independent training runs on 8×8 and 16×16. The downstream stages use fixed offsets of the base seed, so that different tasks do not share the same sequence of pseudo-random choices. The offsets are recorded in the code and summarised in Table E.2.

Table E.1: Software versions of the seeded environment locks and hardware for training and shared evaluation

Item	Locked value
Python	3.12.13
PyTorch	2.11.0+cu128
CUDA runtime	12.8
cuDNN	91900
mamba-ssm	2.3.2.post1
causal-conv1d	1.6.2.post1
Main-encoder precision	bf16 autocast, no GradScaler
Downstream-head precision	fp32
Main-training GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition, 96 GB
Final-evaluation GPU	NVIDIA L4, 24 GB

Table E.2: Allocation of seeds to the successive stages of the experiment

Stage	Seed when base seed = 42
Canonical main training	42
Additional independent training runs	0, 1, 2, 3
Linear frozen head	42
MLP frozen head	10,042
Linear board-state probe	42
MLP board-state probe	20,042
Raw linear intervention probe	40,042
Construction of selection cases	50,042
Construction of final-test cases	60,042
Random intervention directions	70,042
Nonlinear probe of the Li method	80,042

E.3 Run identity and compatibility checks

Each result is linked to its checkpoint through a SHA-256 hash and to the data through a versioned manifest. The manifest stores the exact identity of the shards and, where required, of the selected games or positions. The evaluation protocol has its own version, `unified_eval_v4`, while the causal suite is stored separately as `causal_intervention_suite_v2`. Keeping separate namespaces prevents a newer run from silently overwriting older results.

Table E.3: Metadata that link each result to its source

Identity record	Role
Source and resolved configuration	Record both the original setting and the final values after defaults are resolved
<code>train_config.json</code> / <code>metrics.csv</code>	Document the configuration and the per-shard progress of the main training
Checkpoint SHA-256	Links each downstream result to specific weights
Source hash	Detects changes to the source code; does not on its own prove functional equivalence
Corpus and split manifest hashes	Pin down the shards, games and positions of each split
Protocol / schema version	Distinguishes incompatible versions of metrics and result structures
Precision and library versions	Prevent mixing fp16, bf16 or a different Mamba backend
Causal-intervention case manifest	Records the prefix, square, and original and counterfactual legality for each case

Before evaluation, it is checked that the checkpoint contains no non-finite values and that it matches the expected board size, block size, vocabulary, precision and source identity. The artefacts of the frozen heads and probes are additionally checked against the checkpoint hash and the split manifest before being reused. Incompatibilities are rejected. A change in the source hash in particular can be explicitly permitted with `ALLOW_SOURCE_DRIFT` and is recorded in the metadata. This exception does not remove the need to check that the code changes do not affect the reused results.

E.4 Organisation of experimental records and reproduction order

`Master_Thesis_Code_Final` contains the notebooks and the locked configurations, while the large corpora and checkpoints remain in `Master_Thesis_Artifacts`. The selected JSON files, Markdown summaries, CSV files and figures used in the thesis are collected in `Main_Experimentation_Results` without constituting a second source of truth: `provenance/results_manifest.json` records the original path of each copy. The notebooks in Table E.4 are located in `Main_Experimental_Setup/notebooks`. The table describes the reproduction of the twelve main conditions and the generation of the aggregate results from the stored evaluations.

The notebooks explicitly select the architecture, objective and board size and resolve the remaining values from the canonical configurations. The shared evaluation must be completed before the causal suite, because the JEPA heads and the intervention probes reuse its checked artefacts. `chapter_5_results.ipynb` does not retrain models: it only reads completed results and produces the aggregate tables and figures of Chapter 5, as well as the supplementary tables

Table E.4: Minimal execution order for reproducing the main results

Step	Notebook / location	Output
1. Main training	<code>train_transformer_ar.ipynb</code> , <code>train_transformer_jepa.ipynb</code> , <code>train_mamba_ar.ipynb</code> , <code>train_mamba_jepa.ipynb</code>	<code>latest.pt</code> , planned milestones, <code>final.pt</code> , <code>metrics.csv</code> , <code>train_config.json</code>
2. Shared evaluation	<code>common_evaluation.ipynb</code>	Frozen heads, legal-move metrics, bootstrap intervals, board-state probes and <code>results__unified_eval_v4.json</code>
3. Causal evaluation	<code>causal_intervention.ipynb</code>	Shared case manifest, Li/Nanda/adapted results and summary of <code>causal_intervention_suite_v2</code>
4. Comparative analysis	<code>comparative_analysis_v3.ipynb</code>	Controlled comparisons between the twelve conditions
5. Chapter 5 figures and tables	<code>chapter_5_results.ipynb</code>	CSV/JSON tables and the final result figures
6. Computational cost	<code>compute_cost_figures.ipynb</code>	Training and evaluation times and the corresponding bar plots

of Appendices D and F. The multi-seed analysis additionally uses the already stored evaluations of the five seeds on 8×8 and 16×16 ; running only the twelve canonical models is not sufficient to reproduce it.

E.5 Resuming runs and remaining limits

The data, checkpoints and result files are stored on a mounted Google Drive, so that each run can be resumed from the last saved checkpoint. Training proceeds shard by shard, so the whole set of games never needs to be loaded into memory. Each checkpoint contains the model and optimiser parameters, as well as the run state: step number, shard index and total volume of processed data. Resuming is allowed only if the numerical precision and the configuration of the checkpoint fully match those of the original run.

The main training updates `latest.pt` at shard boundaries, saves additional checkpoints at the planned milestones and creates `final.pt` on completion. Keeping a separate checkpoint for every shard depends on the relevant setting and is not a universal requirement. Resuming restores the saved weights and optimiser state, but the older checkpoints do not contain the full state of all random-number generators. Bitwise identity with an uninterrupted run is therefore not guaranteed.

For the same reason, reproducing the exact final decimals is not the only criterion. Priority is given to the identity of data and code, completion of the same number of shards, use of the same splits and repetition of the qualitative pattern across independent seeds. The stored hashes and manifests make it possible to distinguish a genuine experimental discrepancy from the use of a different checkpoint, corpus or protocol.

Appendix F

Supplementary Analyses of the Results

This appendix complements Chapter 5 with the statistical tests of the independent training runs, the comparison of encodings and probe families, per-class performance and the comparison with random prediction. In the first two tables, each effect is first computed separately for each seed and then summarised over the five matched values. The remaining tables concern the twelve main models of seed 42.

F.1 Paired t -tests and Holm correction

For each seed, three contrasts are computed: the mean Mamba–Transformer difference across the two objectives, the mean JEPA–AR difference across the two architectures, and the difference of these two AR→JEPA changes between architectures. Each contrast yields five numbers. The 95% interval is obtained from their mean and standard error, with a Student t distribution and four degrees of freedom. The Holm correction adjusts the family of three p -values separately for each metric and size; the displayed intervals remain individual 95% intervals, not simultaneous intervals corrected for multiplicity.

F.2 Exact sign-flip tests and effect sizes

The exact sign-flip test examines all 32 possible sign combinations of the five matched differences. The two-sided p is the fraction of combinations that give a mean with absolute value at least equal to the observed one. The column $+ / 0 / -$ records separately how many seeds gave a positive, zero or negative effect. Hedges’ g_z divides the mean matched difference by the standard deviation of the same differences and applies the approximate small-sample correction used here, with a factor of 0.8 for five seeds. It does not use the standard deviation of all models pooled together.

The very large absolute values of Hedges’ g_z on the almost saturated top-1 metrics are due mainly to the extremely small variance across seeds. For this reason, the standardised sizes are reported for completeness, but the practical significance of the differences is interpreted mainly

Table F.1: Matched effects, 95% confidence intervals based on the t distribution and two-sided p -values of the paired t -tests. The Holm correction is applied separately to the family of three effects for each combination of board size and metric.

Board	Metric	Effect	Difference (pp) [95% t -CI]	p_t	p_{Holm}
8×8	Top-1 legality	Architecture	+0.21 [+0.21 to +0.22]	< 0.001	< 0.001
8×8	Top-1 legality	Objective	−0.20 [−0.22 to −0.19]	< 0.001	< 0.001
8×8	Top-1 legality	Interaction	+0.18 [+0.15 to +0.21]	< 0.001	< 0.001
8×8	Legal mass	Architecture	+1.07 [+0.95 to +1.19]	< 0.001	< 0.001
8×8	Legal mass	Objective	−0.54 [−0.59 to −0.48]	< 0.001	< 0.001
8×8	Legal mass	Interaction	+0.28 [+0.22 to +0.35]	< 0.001	< 0.001
8×8	Relative state	Architecture	+1.26 [+1.10 to +1.42]	< 0.001	< 0.001
8×8	Relative state	Objective	−0.04 [−0.18 to +0.10]	0.474	0.474
8×8	Relative state	Interaction	+0.59 [+0.14 to +1.04]	0.021	0.043
16×16	Top-1 legality	Architecture	+0.48 [+0.39 to +0.57]	< 0.001	< 0.001
16×16	Top-1 legality	Objective	−1.02 [−1.07 to −0.96]	< 0.001	< 0.001
16×16	Top-1 legality	Interaction	+0.89 [+0.71 to +1.07]	< 0.001	< 0.001
16×16	Legal mass	Architecture	+2.32 [+2.02 to +2.62]	< 0.001	< 0.001
16×16	Legal mass	Objective	−0.56 [−0.77 to −0.35]	0.002	0.004
16×16	Legal mass	Interaction	+0.98 [+0.50 to +1.45]	0.005	0.005
16×16	Relative state	Architecture	+3.01 [+2.89 to +3.13]	< 0.001	< 0.001
16×16	Relative state	Objective	−2.08 [−2.32 to −1.85]	< 0.001	< 0.001
16×16	Relative state	Interaction	+5.09 [+4.23 to +5.95]	< 0.001	< 0.001

Table F.2: Results of the exact sign-flip tests, agreement of direction across the five seeds and Hedges’ g_z effect sizes. With five seeds, the smallest possible two-sided p -value is 0.0625.

Board	Metric	Effect	p_{exact}	Seeds +/0/−	Hedges’ g_z
8×8	Top-1 legality	Architecture	0.0625	5/0/0	+45.07
8×8	Top-1 legality	Objective	0.0625	0/0/5	−14.43
8×8	Top-1 legality	Interaction	0.0625	5/0/0	+6.62
8×8	Legal mass	Architecture	0.0625	5/0/0	+9.07
8×8	Legal mass	Objective	0.0625	0/0/5	−9.71
8×8	Legal mass	Interaction	0.0625	5/0/0	+4.27
8×8	Relative state	Architecture	0.0625	5/0/0	+7.88
8×8	Relative state	Objective	0.5000	2/0/3	−0.28
8×8	Relative state	Interaction	0.0625	5/0/0	+1.31
16×16	Top-1 legality	Architecture	0.0625	5/0/0	+5.48
16×16	Top-1 legality	Objective	0.0625	0/0/5	−17.85
16×16	Top-1 legality	Interaction	0.0625	5/0/0	+4.93
16×16	Legal mass	Architecture	0.0625	5/0/0	+7.65
16×16	Legal mass	Objective	0.0625	0/0/5	−2.61
16×16	Legal mass	Interaction	0.0625	5/0/0	+2.05
16×16	Relative state	Architecture	0.0625	5/0/0	+24.62
16×16	Relative state	Objective	0.0625	0/0/5	−8.73
16×16	Relative state	Interaction	0.0625	5/0/0	+5.87

in percentage points.

F.3 Absolute and relative encoding

Tables F.3 and F.4 compare overall accuracy and macro-balanced accuracy for linear and MLP probes. The layer is selected independently for each family and encoding, exclusively on validation. All probes use LayerNorm on the input. For brevity, T denotes Transformer and M denotes Mamba.

Table F.3: Comparison of linear and MLP probes of the absolute encoding. Accuracies are given in %, at the layer L selected on validation.

Board	System	Linear overall	Linear macro	L	MLP overall	MLP macro	L
8×8	T-AR	75.57	68.88	3	95.20	93.88	5
8×8	T-JEPA	75.69	69.02	4	95.94	94.83	5
8×8	M-AR	75.50	68.78	11	89.43	86.55	10
8×8	M-JEPA	75.60	68.92	12	91.41	89.31	9
12×12	T-AR	74.96	67.30	5	79.00	72.59	5
12×12	T-JEPA	74.90	67.23	5	81.49	75.84	6
12×12	M-AR	74.55	66.77	10	74.48	66.68	11
12×12	M-JEPA	74.78	67.07	12	75.24	67.79	9
16×16	T-AR	74.79	66.81	5	74.94	67.03	7
16×16	T-JEPA	74.36	66.26	6	73.82	65.70	7
16×16	M-AR	74.62	66.58	11	74.33	66.26	11
16×16	M-JEPA	74.28	66.18	13	73.82	65.66	13

Table F.4: Comparison of linear and MLP probes of the relative encoding. Accuracies are given in %, at the layer L selected on validation.

Board	System	Linear overall	Linear macro	L	MLP overall	MLP macro	L
8×8	T-AR	97.59	96.91	6	97.38	96.61	6
8×8	T-JEPA	97.45	96.72	6	97.22	96.41	6
8×8	M-AR	98.32	97.85	12	98.16	97.62	12
8×8	M-JEPA	98.48	98.04	13	98.30	97.82	12
12×12	T-AR	91.74	89.17	7	90.86	88.01	7
12×12	T-JEPA	91.21	88.48	6	90.38	87.40	6
12×12	M-AR	93.41	91.37	12	92.42	90.06	12
12×12	M-JEPA	94.54	92.85	12	93.77	91.85	12
16×16	T-AR	87.02	82.88	8	85.16	80.49	7
16×16	T-JEPA	83.49	78.37	7	81.56	75.99	7
16×16	M-AR	87.37	83.35	11	85.80	81.41	11
16×16	M-JEPA	87.56	83.77	12	85.58	81.26	13

F.4 Analysis of the relative linear probe

Table F.5 separates the recognition of empty squares from the recognition of the side of the pieces. The three per-class columns are recalls; the occupied column is the overall accuracy on squares with a piece only. Table F.6 complements the depth comparison without any new selection on the test set.

Table F.5: Per-class and occupied-square performance, in %, of the relative linear probe at the selected layer. Each condition is evaluated on the same 16,000 positions of the corresponding size.

Board	System	Empty	Mine	Opponent's	Occupied
8×8	T-AR	99.99	94.91	95.85	95.41
8×8	T-JEPA	99.99	94.40	95.78	95.15
8×8	M-AR	99.99	96.39	97.16	96.81
8×8	M-JEPA	99.97	96.51	97.64	97.12
12×12	T-AR	99.98	82.23	85.29	83.83
12×12	T-JEPA	99.97	80.96	84.50	82.81
12×12	M-AR	99.97	85.77	88.36	87.13
12×12	M-JEPA	99.98	88.49	90.09	89.33
16×16	T-AR	99.97	72.22	76.45	74.39
16×16	T-JEPA	99.46	65.16	70.50	67.91
16×16	M-AR	99.95	73.21	76.89	75.10
16×16	M-JEPA	99.37	73.45	78.48	76.04

Table F.6: Selected and final layer of the relative linear probe. Relative depth is the ratio of the selected layer to the number of blocks. Accuracies are given in %.

Board	System	L / depth	Relative depth	Selected macro	Final macro	Final overall
8×8	T-AR	6/8	0.750	96.91	96.82	97.51
8×8	T-JEPA	6/8	0.750	96.72	95.72	96.60
8×8	M-AR	12/15	0.800	97.85	96.86	97.50
8×8	M-JEPA	13/15	0.867	98.04	95.70	96.54
12×12	T-AR	7/8	0.875	89.17	89.16	91.73
12×12	T-JEPA	6/8	0.750	88.48	87.44	90.25
12×12	M-AR	12/15	0.800	91.37	91.09	93.19
12×12	M-JEPA	12/15	0.800	92.85	91.01	93.03
16×16	T-AR	8/8	1.000	82.88	82.88	87.02
16×16	T-JEPA	7/8	0.875	78.37	77.21	82.47
16×16	M-AR	11/15	0.733	83.35	82.93	87.06
16×16	M-JEPA	12/15	0.800	83.77	81.75	85.96

F.5 Comparison with random prediction

The random baseline corresponds to uniform selection of a token from the vocabulary, not to selection among the legal moves only. The normalised lift expresses what fraction of the distance from this baseline to perfect legality the model covers. Means per evaluated position are used, as the definition of the metric requires.

Table F.7: Legality against uniform random selection from the vocabulary. The normalised lift is 0% at the baseline and 100% at perfect legality.

Board	System	Random baseline (%)	Top-1 per position (%)	Normalised lift (%)
8×8	T-AR	14.13	99.73	99.68
8×8	T-JEPA	14.13	99.42	99.32
8×8	M-AR	14.13	99.85	99.82
8×8	M-JEPA	14.13	99.72	99.67
12×12	T-AR	11.86	99.42	99.34
12×12	T-JEPA	11.86	98.52	98.32
12×12	M-AR	11.86	99.51	99.45
12×12	M-JEPA	11.86	98.99	98.86
16×16	T-AR	10.29	98.99	98.87
16×16	T-JEPA	10.29	97.54	97.26
16×16	M-AR	10.29	99.05	98.94
16×16	M-JEPA	10.29	98.41	98.23

Appendix G

Core Code Definitions

Abridged excerpts of the implementation are listed below, with auxiliary statements omitted and line wrapping adjusted. The captions give the paths and the individual line ranges in `Master_Thesis_Code_Final`, as checked on 11/09/2026. The full executable form is in the corresponding files.

G.1 Data generation and sequence construction

The excerpts in this section concern the data used by all models: the generation of the games and the construction of the input and target sequences.

G.1.1 Deterministic game generation

The 8×8 dataset of Li et al. (2023) contains 23,799,812 records in total across the 238 shards. In the new 12×12 and 16×16 datasets, to ensure reproducibility, each game is generated from a pseudo-random sequence determined solely by the shared random seed and its unique serial number in the dataset. Code Listing G.1.1 shows the generation of one game and the implicit handling of a pass.

```
game_index = np.uint64(global_start_index + local_index)
rng_state = _splitmix64(seed ^ _splitmix64(game_index))

for _ply in range(max_moves):
    n_legal = _collect_legal(
        cells, board_size, color, legal_buffer
    )
    if n_legal == 0:
        color = -color
        n_legal = _collect_legal(
            cells, board_size, color, legal_buffer
        )
    if n_legal == 0:
        break

    rng_state, random_value = _next_random(rng_state)
    selected = int(random_value % np.uint64(n_legal))
    move = int(legal_buffer[selected])
```

```

_apply_move(cells, board_size, move, color)
color = -color

```

Code Listing G.1.1. Deterministic game generation and implicit handling of a pass (`src/othello_thesis/game_engine/generator.py`, selected lines 145–170).

G.1.2 Construction of input and target sequences

The mapping from board squares to consecutive token ids is computed once for each size. Shorter sequences are filled with a valid input padding token, while the corresponding target positions receive the value -100 and are ignored by the cross-entropy. For Autoregressive training, the input x contains all tokens except the last and the target y all tokens except the first (Code Listing G.1.2).

```

seq = seq[: self.block_size + 1]
pad_len = (self.block_size + 1) - len(seq)
if pad_len > 0:
    seq = seq + [TARGET_PAD] * pad_len

seq_t = torch.tensor(seq, dtype=torch.long)
x, y = seq_t[:-1], seq_t[1:]
x = x.masked_fill(x == TARGET_PAD, self.input_pad_token)
return x, y

```

Code Listing G.1.2. Construction of shifted input and target sequences (`src/othello_thesis/data/chunk_dataset.py`, selected lines 128–143).

G.2 Definition of the Transformer

The definition includes the pre-norm Transformer block, the embeddings, the stack of blocks and the native move-prediction head.

```

class TransformerBlock(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.d_model)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.d_model)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x

class OthelloGPT(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        self.config = config
        self.wte = nn.Embedding(config.vocab_size, config.d_model)
        self.wpe = nn.Embedding(config.block_size, config.d_model)

```

```

self.drop = nn.Dropout(config.dropout)
self.blocks = nn.ModuleList(
    [TransformerBlock(config) for _ in range(config.n_layers)]
)
self.ln_f = nn.LayerNorm(config.d_model)
self.lm_head = nn.Linear(
    config.d_model, config.vocab_size, bias=False
)
self.apply(self._init_weights)

def forward(self, idx, targets=None):
    _, seq_len = idx.size()
    pos = torch.arange(0, seq_len, device=idx.device)
    x = self.drop(self.wte(idx) + self.wpe(pos))
    for block in self.blocks:
        x = block(x)
    x = self.ln_f(x)
    logits = self.lm_head(x)
    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(-1, logits.size(-1)),
            targets.view(-1),
        )
    return logits, loss

```

Code Listing G.2. Definition of the Transformer model (src/othello_thesis/models/transformer.py, selected lines 176–197, 206–231 and 243–276).

G.3 Definition of Mamba

Mamba exposes the same external interface as the Transformer, but each block is updated through the selective SSM mixer.

```

class MambaARBlock(nn.Module):
    def __init__(self, config: MambaARConfig, layer_idx=None):
        super().__init__()
        self.layer_idx = layer_idx
        self.ln_1 = nn.LayerNorm(config.d_model)
        self.mixer = build_mamba_mixer(config)
        self.resid_dropout = nn.Dropout(config.dropout)
        self.use_mlp = int(config.mlp_hidden_mult) > 0
        if self.use_mlp:
            self.ln_2 = nn.LayerNorm(config.d_model)
            self.mlp = MambaMLP(config)

    def forward(self, x):
        x = x + self.resid_dropout(self._mix(self.ln_1(x)))
        if self.use_mlp:
            x = x + self.mlp(self.ln_2(x))
        return x

class OthelloMambaAR(nn.Module):
    def __init__(self, config: MambaARConfig):
        super().__init__()
        self.config = config

```

```

self.wte = nn.Embedding(config.vocab_size, config.d_model)
self.wpe = nn.Embedding(config.block_size, config.d_model)
self.drop = nn.Dropout(config.dropout)
self.blocks = nn.ModuleList(
    [MambaARBlock(config, layer_idx=i)
     for i in range(config.n_layers)]
)
self.ln_f = nn.LayerNorm(config.d_model)
self.lm_head = nn.Linear(
    config.d_model, config.vocab_size, bias=False
)
self.apply(self._init_weights)

def forward(self, idx, targets=None):
    _, seq_len = idx.shape
    pos = torch.arange(seq_len, device=idx.device)
    x = self.drop(self.wte(idx) + self.wpe(pos))
    for block in self.blocks:
        x = block(x)
    x = self.ln_f(x)
    logits = self.lm_head(x)
    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.reshape(-1, logits.size(-1)),
            targets.reshape(-1),
            ignore_index=TARGET_PAD,
        )
    return logits, loss

```

Code Listing G.3. Definition of the Mamba model (src/othello_thesis/models/mamba.py, selected lines 322–337, 376–380, 386–398 and 408–434).

G.4 Autoregressive training

The excerpt shows the shared AR training loop for the Transformer and Mamba, together with the learning-rate schedule, mixed precision and gradient clipping.

```

def train_one_chunk(model, optimizer, scaler, chunk_path,
                    cfg, model_cfg, device, step, total_steps,
                    *, style, progress_desc=None):
    loader, games, dataset = make_chunk_loader(
        chunk_path, block_size=model_cfg.block_size,
        board_size=cfg.board_size, batch_size=cfg.batch_size,
        num_workers=cfg.num_workers, shuffle=True,
    )
    model.train()
    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)
        learning_rate = cosine_learning_rate(step, cfg, total_steps)
        for group in optimizer.param_groups:
            group["lr"] = learning_rate

        optimizer.zero_grad(set_to_none=True)
        with autocast_context(device, cfg.precision):
            logits, loss = model(x, y)

```

```

    scaler.scale(loss).backward()
    if cfg.grad_clip > 0:
        if scaler.is_enabled():
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(
                model.parameters(), cfg.grad_clip
            )
        scaler.step(optimizer)
        scaler.update()
    step += 1

```

Code Listing G.4. Autoregressive training loop (`src/othello_thesis/training/ar_steps.py`, selected lines 20–73).

G.5 JEPA training

The final implementation explicitly restricts the active branch to the hard-disjoint all-position InfoNCE objective. The historical versions remain in `JEPA_Experimentation` and are not silently introduced into the main training. The excerpts that follow come from the verified implementation and isolate the decisions that differentiated the experiments.

Code Listing G.5 shows the shared core of one training step for the Transformer and Mamba: the all-position loss is computed, the context encoder and the predictor are updated and, next, the target encoder is updated with EMA.

```

lr = get_lr(step, cfg, total_steps)
for group in optimizer.param_groups:
    group["lr"] = lr

optimizer.zero_grad(set_to_none=True)
with torch.autocast(
    device_type=device.type, dtype=amp_dtype, enabled=amp_enabled
):
    out, effective_batch_size = forward_all_position_v1(
        model, x, pad_token=pad_token,
        target_chunk_size=cfg.all_position_target_chunk_size,
        stats_chunk_size=cfg.all_position_stats_chunk_size,
        profile=profile_step,
    )
    loss = out["loss"]

scaler.scale(loss).backward()
if cfg.grad_clip > 0:
    if scaler.is_enabled():
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(
            model.parameters(), cfg.grad_clip
        )
    scaler.step(optimizer)
    scaler.update()
    update_target_encoder(model)

```

Code Listing G.5. Shared core of JEPA all-position training for the Transformer and Mamba (`src/othello_thesis/training/_jepa_loop.py`, selected lines 98–138).

G.5.1 InfoNCE and target-encoder update

The predictor output and the target embedding are normalised before the similarity is computed. The correct pair lies on the diagonal of the logits matrix. The target encoder is initially copied from the context encoder, remains frozen with respect to gradients and is updated after every optimisation step with EMA.

```
def infonce_jepa_loss(z_pred, z_tgt, *, temperature):
    z_pred = z_pred.float()
    z_tgt = z_tgt.float()
    pred_n = F.normalize(z_pred, dim=-1)
    tgt_n = F.normalize(z_tgt, dim=-1)
    logits = pred_n @ tgt_n.T / temperature
    labels = torch.arange(logits.size(0), device=logits.device)
    loss = F.cross_entropy(logits, labels)
    return {"loss": loss, "infonce_loss": loss.detach()}

@torch.no_grad()
def update_target_encoder(self, momentum=None):
    m = self.jepa_config.ema_momentum if momentum is None else momentum
    for context_param, target_param in zip(
        self.context_encoder.parameters(),
        self.target_encoder.parameters(),
    ):
        target_param.data.mul_(m).add_(
            context_param.data, alpha=1.0 - m
        )
```

Code Listing G.5.1. InfoNCE objective and EMA update of the target encoder (src/othello_thesis/objectives/jepa.py, selected lines 132–140 and 281–292).

G.5.2 Construction of nested and hard-disjoint views

The critical difference lies in the tensor given to the target encoder. In `hard_disjoint_future`, only the future moves are included. In the historical `nested/disjoint_future` path, the target receives the prefix together with the future.

```
def build_jepa_views(cfg, x_valid, *, context_length, horizon):
    x_ctx = x_valid[:, :context_length].contiguous()
    view_mode = jepa_view_mode(cfg)
    batch_size = x_valid.size(0)
    device = x_valid.device

    if view_mode == "hard_disjoint_future":
        x_tgt = x_valid[
            :, context_length: context_length + horizon
        ].contiguous()
        target_positions = build_target_positions(
            context_length=context_length,
            horizon=horizon,
            batch_size=batch_size,
            device=device,
        )
    return x_ctx, x_tgt, target_positions
```

```

x_tgt = x_valid[:, : context_length + horizon].contiguous()
target_positions = None
if view_mode == "disjoint_future":
    target_positions = build_target_positions(
        context_length=context_length,
        horizon=horizon,
        batch_size=batch_size,
        device=device,
    )
return x_ctx, x_tgt, target_positions

```

Code Listing G.5.2. Construction of the target views (`src/othello_thesis/training/_jepa_all_positions.py`, selected lines 103–151).

G.5.3 Supervision at all valid positions

The causal representation of the context is computed once. Each non-padding token after the first position then forms a pair between the representation of the preceding prefix and the next token. In the InfoNCE branch, the logits are computed separately per absolute position.

```

context_hidden = raw_model.encode_hidden(
    raw_model.context_encoder, x
)
valid = x[:, 1:] != pad_token
pair_indices = valid.nonzero(as_tuple=False)
game_idx = pair_indices[:, 0]
boundary_idx = pair_indices[:, 1]
absolute_positions = boundary_idx + 1

context_summary = context_hidden[game_idx, boundary_idx, :]
target_tokens = x[game_idx, absolute_positions].unsqueeze(1)
target_positions = absolute_positions.unsqueeze(1)
z_pred = raw_model._predict_embeddings(
    context_summary.unsqueeze(1), target_positions
)

for j in range(n_chunk):
    if not bool(eligible[j]):
        continue
    sel = mask[:, j]
    pred_n = torch.nn.functional.normalize(pred[sel, j, :], dim=-1)
    tgt_n = torch.nn.functional.normalize(target[sel, j, :], dim=-1)
    logits = pred_n @ tgt_n.T / temperature
    labels = torch.arange(logits.size(0), device=logits.device)
    loss_at_position[j] = torch.nn.functional.cross_entropy(
        logits, labels
    )

```

Code Listing G.5.3. Construction of all-position pairs and InfoNCE per absolute position (`src/othello_thesis/training/_jepa_all_positions.py`, selected lines 304–335 and 465–473).

G.5.4 Historical VICReg and Smooth L1 objectives

The shared all-position runner supports the historical VICReg and Smooth L1 variants for reproducing the ablations. VICReg combines a matching term, a variance-preservation term and

a decorrelation term, while Smooth L1 applies direct regression in representation space.

```

if loss_type == "vicreg":
    inv = (
        ((pred - target).square() * mask_f).sum(dim=(0, 2))
        / (denom * d_model)
    )
    var = 0.5 * (
        torch.relu(gamma - pred_std).mean(dim=-1)
        + torch.relu(gamma - target_std).mean(dim=-1)
    )
    cov = pred_cov_penalty + target_cov_penalty
    loss_at_position = (
        float(jcfg.vicreg_lambda) * inv
        + float(jcfg.vicreg_mu) * var
        + float(jcfg.vicreg_nu) * cov
    )
elif loss_type == "smooth_l1":
    elementwise = torch.nn.functional.smooth_l1_loss(
        pred, target,
        beta=float(jcfg.smooth_l1_beta),
        reduction="none",
    )
    loss_at_position = (
        (elementwise * mask_f).sum(dim=(0, 2))
        / (denom * d_model)
    )

```

Code Listing G.5.4. Historical VICReg and Smooth L1 branches of the all-position runner (`src/othello_thesis/training/_jepa_all_positions.py`, selected lines 415–453).

G.5.5 Multimodal distance-margin loss of v6

v6 produces ten candidate representations. For each target from a game with the same prefix, the loss pulls the closest prediction towards it. For targets with a different prefix it enforces a minimum distance. There is, however, a special case: different prefixes can have the same next move and therefore the same target embedding. To avoid contradictory attractions and repulsions, in the repulsion term that pair uses a phantom target from a token that does not appear in the batch. If no such token exists, the conflicting negative pair is skipped. The excerpt includes this check and the normalisation, as implemented in the archived function.

```

B, M, d = p.shape
if normalize:
    p = F.normalize(p, dim=-1)
    z = F.normalize(z, dim=-1)
    if z_phantom is not None:
        z_phantom = F.normalize(z_phantom, dim=-1)
same_prefix = prefix_ids.unsqueeze(0) == prefix_ids.unsqueeze(1)
same_action = next_actions.unsqueeze(0) == next_actions.unsqueeze(1)
collision = (same_prefix.float() @ same_action.float()) > 0.0
pull_mask = same_prefix
neg_mask = ~same_prefix
min_sq, _ = _pairwise_min_mode_sq_dist(p, z)
min_dist = min_sq.clamp_min(1e-12).sqrt()
pull_count = pull_mask.float().sum().clamp_min(1.0)

```

```

pull_loss = (min_sq * pull_mask.float()).sum() / pull_count
if dedup_negative_actions and z_phantom is not None:
    phantom_sq = ((p - z_phantom.unsqueeze(0).unsqueeze(0)) ** 2).sum(dim=-1)
    phantom_min_sq = phantom_sq.min(dim=1).values
    phantom_min_dist = phantom_min_sq.clamp_min(1e-12).sqrt()
    phantom_dist_grid = phantom_min_dist.unsqueeze(1).expand(B, B)
    push_dist = torch.where(collision, phantom_dist_grid, min_dist)
else:
    push_dist = min_dist
    if dedup_negative_actions and z_phantom is None:
        neg_mask = neg_mask & (~collision)
push_terms = F.relu(margin - push_dist) ** 2
push_count = neg_mask.float().sum().clamp_min(1.0)
push_loss = (push_terms * neg_mask.float()).sum() / push_count
total = pull_loss + lambda_push * push_loss

```

Code Listing G.5.5. Core of the multimodal distance-margin loss of v6 (JEPA_Experimentation/archive/source_tree/othello_research/objectives/jepa_hard_disjoint_action.py, selected lines 233–280).

G.5.6 Restricting the active implementation to the final setup

To prevent a historical objective from being activated by mistake in the main experimental setup, the active code rejects any different loss, view, predictor or horizon. The internal label `jepa_v1` is kept only for compatibility with the six canonical checkpoints and does not indicate that the original experimental v1 is used.

```

def normalize_jepa_loss_type(
    loss_type: str, variant: str = "jepa_v1"
) -> str:
    normalize_jepa_variant(variant)
    if loss_type != "infonce":
        raise ValueError(
            "The main experiment supports only loss_type='infonce'; "
            "historical losses are available under JEPA_Experimentation."
        )
    return "infonce"

def normalize_jepa_view_mode(view_mode: str) -> str:
    if view_mode != "hard_disjoint_future":
        raise ValueError(
            "The main experiment supports only "
            "view_mode='hard_disjoint_future'; historical views are "
            "available under JEPA_Experimentation."
        )
    return "hard_disjoint_future"

if cfg.predictor_type != "linear":
    raise ValueError(
        "The final JEPA objective requires predictor_type='linear'."
    )

if cfg.prediction_horizon != 1:
    raise ValueError(
        "The final JEPA objective requires prediction_horizon=1."
    )

```

Code Listing G.5.6. Checks that lock the active implementation to the final JEPA objective (`src/othello_thesis/objectives/jepa.py`, selected lines 38–57 and 204–207).

G.6 Frozen move-prediction heads

The encoder remains frozen and only the parameters of the linear or MLP head are trained, in `fp32`.

```
class _MLPReadout(nn.Module):
    def __init__(self, d_in, d_out, hidden_dim, dropout):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, hidden_dim), nn.GELU(),
            nn.Dropout(dropout), nn.Linear(hidden_dim, d_out),
        )

    def forward(self, x):
        return self.net(x)

def build_readout(d_model, vocab_size, head_type, *,
                  hidden_dim=512, dropout=0.1):
    if head_type == "linear":
        return nn.Linear(d_model, vocab_size)
    if head_type == "mlp":
        return _MLPReadout(
            d_model, vocab_size, hidden_dim, dropout
        )
    raise ValueError("head_type must be 'linear' or 'mlp'")

class FrozenEncoderNextMoveHead(nn.Module):
    def __init__(self, encoder, head_type, *, hidden_dim=512,
                  dropout=0.1, encoder_precision="bf16"):
        super().__init__()
        self.encoder = _base_model(encoder)
        for parameter in self.encoder.parameters():
            parameter.requires_grad_(False)
        self.encoder.eval()
        self.head = build_readout(
            int(self.encoder.config.d_model),
            int(self.encoder.config.vocab_size),
            head_type, hidden_dim=hidden_dim, dropout=dropout,
        )
        self.head.float()

# train_frozen_next_move_head
optimizer = torch.optim.AdamW(
    model.head.parameters(), lr=learning_rate,
    weight_decay=weight_decay,
)

for x_cpu, y_cpu in loader:
    x = x_cpu.to(device, non_blocking=True)
    y = y_cpu.to(device, non_blocking=True)
    optimizer.zero_grad(set_to_none=True)
    logits, loss = model(x, y)
```

```

loss.backward()
torch.nn.utils.clip_grad_norm_(
    model.head.parameters(), config.head_gradient_clip
)
optimizer.step()

```

Code Listing G.6. Definition and training of the frozen head (src/othello_thesis/evaluation/unified.py, selected lines 120–179, 810–811 and 848–861).

G.7 Board-state probes

Each probe outputs three classes per square. The linear and MLP probes are trained independently for each layer and encoding.

```

class PositionProbe(nn.Module):
    def __init__(self, d_model, n_squares, *, probe_type,
                  hidden_dim=512, dropout=0.1,
                  input_layernorm=True):
        super().__init__()
        self.n_squares = int(n_squares)
        self.input_norm = (
            nn.LayerNorm(d_model, elementwise_affine=False)
            if input_layernorm else nn.Identity()
        )
        output_dim = self.n_squares * 3
        if probe_type == "linear":
            self.proj = nn.Linear(d_model, output_dim)
        else:
            self.proj = nn.Sequential(
                nn.Linear(d_model, hidden_dim), nn.GELU(),
                nn.Dropout(dropout),
                nn.Linear(hidden_dim, output_dim),
            )

    def forward(self, features):
        logits = self.proj(self.input_norm(features.float()))
        return logits.view(features.shape[0], self.n_squares, 3)

optimizer = torch.optim.AdamW(
    bank.parameters(), lr=float(lr), weight_decay=weight_decay
)
for start in range(0, train_features.n_samples, batch_size):
    batch_indices = order[start : start + batch_size]
    labels_by_mode = {
        "absolute": train_features.absolute_labels[batch_indices].to(
            target_device
        ),
        "relative": train_features.relative_labels[batch_indices].to(
            target_device
        ),
    }
    optimizer.zero_grad(set_to_none=True)
    losses = []
    for layer in train_features.layers:
        layer_features = train_features.features[layer][
            batch_indices

```

```

].to(target_device)
for mode, probe_dict in (
    ("absolute", bank.absolute),
    ("relative", bank.relative),
):
    logits = probe_dict[str(layer)](layer_features)
    losses.append(
        _classification_loss(logits, labels_by_mode[mode])
    )
torch.stack(losses).sum().backward()
for _, _, probe in bank.iter_independent_probes():
    torch.nn.utils.clip_grad_norm_(
        probe.parameters(), max_norm=gradient_clip,
        error_if_nonfinite=True,
    )
optimizer.step()

```

Code Listing G.7. Definition and training of the board-state probe (src/othello_thesis/evaluation/position_probe.py, selected lines 271–308, 684–686 and 701–733).

G.8 Causal intervention in the residual stream

The implementation organises the Li, Nanda and adapted methods in a shared runner, so that they use the same counterfactual cases, the same head per model and the same metric computation. The three excerpts that follow isolate the defining operation of each intervention; the shared case construction, batch processing and result storage are not repeated.

G.8.1 Li method: optimising the internal representation

The method uses a nonlinear probe of the absolute state with a hidden layer of 128 units. At each selected depth, the representation is optimised for 1,000 Adam steps with learning rate 0.001. The square that changes receives weight 1, while the other squares receive a preservation weight of 0.2.

```

edited = activation.detach().float().clone().requires_grad_(True)
desired_labels = desired_labels.to(edited.device, dtype=torch.long)
changed_squares = changed_squares.to(edited.device, dtype=torch.long)
weights = torch.full(
    desired_labels.shape,
    float(preservation_weight),
    device=edited.device,
    dtype=torch.float32,
)
weights[
    torch.arange(edited.shape[0], device=edited.device),
    changed_squares,
] = 1.0
optimizer = torch.optim.Adam([edited], lr=float(learning_rate))

for _ in range(steps):
    optimizer.zero_grad(set_to_none=True)
    logits = probe(edited)
    losses = F.cross_entropy(
        logits.reshape(-1, 3),

```

```

        desired_labels.reshape(-1),
        reduction="none",
    ).reshape_as(weights)
    loss = (weights * losses).mean()
    if not torch.isfinite(loss):
        raise RuntimeError(
            "Li activation optimization produced non-finite loss"
        )
    loss.backward()
    optimizer.step()

```

Code Listing G.8.1. Core of the representation optimisation in the Li method (src/othello_thesis/evaluation/causal_suite.py, selected lines 323–376).

G.8.2 Nanda method: intervention on selected residual branches

The direction is the normalised weight vector of the target class from a raw linear relative probe. In the Transformer it is added to the attention output of the selected blocks with a fixed strength of 2.3. In Mamba the same operation is applied to the output of the mixer residual branch.

```

def nanda_target_directions(
    probe_bank, cases, *, layer, device
):
    probe = probe_bank.relative[str(layer)]
    if not isinstance(probe.input_norm, nn.Identity):
        raise TypeError(
            "Nanda intervention requires an unnormalized probe"
        )
    weights = probe.proj.weight.view(
        probe.n_squares, 3, probe.proj.weight.shape[1]
    )
    directions = torch.stack([
        weights[case.square, case.target_class]
        for case in cases
    ]).to(device=device, dtype=torch.float32)
    return directions / directions.norm(
        dim=-1, keepdim=True
    ).clamp_min(1e-8)

for block_index, block in enumerate(encoder.blocks):
    if hasattr(block, "attn"):
        branch = block.attn(block.ln_1(hidden))
        if block_index in patch_blocks:
            branch = _apply_last_position_edit(
                branch, rows=rows, last_indices=last_indices,
                directions=directions, scale=scale,
            )
        hidden = hidden + branch
        hidden = hidden + block.mlp(block.ln_2(hidden))
    elif hasattr(block, "mixer"):
        branch = block.resid_dropout(
            block._mix(block.ln_1(hidden))
        )
        if block_index in patch_blocks:
            branch = _apply_last_position_edit(
                branch, rows=rows, last_indices=last_indices,

```

```

        directions=directions, scale=scale,
    )
    hidden = hidden + branch
    if block.use_mlp:
        hidden = hidden + block.mlp(block.ln_2(hidden))

```

Code Listing G.8.2. Extraction of the target direction and its application to the Transformer and Mamba branches (src/othello_thesis/evaluation/causal_suite.py, selected lines 379–411 and 438–463).

G.8.3 Adapted method: successive intervention after every block

The adapted method subtracts the vector of the source class from that of the target class, normalises the direction and applies it at the last position after every block. The size of the change is scaled by the standard deviation of the current representation and by the selected strength α . The same code branch produces the geometrically matched random control, by applying the same random coordinate permutation and sign flip to all layers. The older control with an independent random direction per layer is kept only as a sensitivity check and is not used in the main tables.

```

def _probe_directions(probe_bank, cases, *, layer, device):
    probe = probe_bank.relative[str(layer)]
    if not isinstance(probe.input_norm, nn.Identity):
        raise TypeError(
            "Causal intervention requires a raw, unnormalized probe"
        )
    weights = probe.proj.weight.view(
        probe.n_squares, 3, probe.proj.weight.shape[1]
    )
    directions = torch.stack([
        weights[case.square, case.target_class]
        - weights[case.square, case.source_class]
        for case in cases
    ]).to(device=device, dtype=torch.float32)
    return directions / directions.norm(
        dim=-1, keepdim=True
    ).clamp_min(1e-8)

random_permutations: torch.Tensor | None = None
random_signs: torch.Tensor | None = None
for layer, block in enumerate(encoder.blocks, start=1):
    hidden = block(hidden)
    if control == "null":
        continue
    directions = _probe_directions(
        probe_bank, cases, layer=layer, device=device
    )
    if control == "random":
        if random_permutations is None or random_signs is None:
            random_permutations, random_signs = (
                _geometry_matched_randomizer(
                    directions, seed=random_seed
                )
            )
        directions = _apply_geometry_matched_randomizer(
            directions, random_permutations, random_signs

```

```

    )
    elif control == "independent_random":
        directions = _random_directions_like(
            directions, seed=random_seed + layer
        )
    selected = hidden[rows, last_indices].float()
    residual_scale = selected.std(
        dim=-1, unbiased=False
    ).clamp_min(1e-6)
    edited = selected + alpha * residual_scale[:, None] * directions
    hidden[rows, last_indices] = edited.to(hidden.dtype)

```

Code Listing G.8.3. Computation and successive application of the adapted intervention direction (src/othello_thesis/evaluation/causal_intervention.py, selected lines 215–236 and 320–351).